MODULE 00 · EXAM STRATEGY - ALL DOMAINS

How This Course Works

HashiCorp Certified: Terraform Associate (004). Twenty-nine modules, just under seven hours.

EXAM TA-004 TERRAFORM 1.12 RUNTIME 6h 58m

admin@sthithapragna.com

A course built to make you **understand** Terraform well enough to pass, rather than to recognise answers.

admin@sthithapragna.com

What is in the course

PART	WHAT IT COVERS
Modules 1 to 23	The eight exam domains, in the order they build on each other
Module 24	Confusion pairs. Mechanisms that get mistaken for each other
Module 25	Version currency. What changed after the version the exam tests
Module 26	Exam mechanics. Format, timing and strategy
Practice exams 1 to 3	Sixty original questions, mixed domains, timed

ON THE EXAM

Modules 24 to 26 add no new Terraform. They convert what you know into marks, which is a different skill.

Contents: Domains 1 to 3

STARTS	MODULE
0:00	1. Infrastructure as Code, and Why Terraform
13:50	2. Architecture: Core, Providers, the Registry
30:26	3. Provider Versioning and the Lock File
45:17	4. State: What It Is, Why It Exists, What It Costs
1:00:25	5. The Core Workflow: init, plan, apply, destroy
1:17:31	6. Reading a Plan. fmt, validate, show, output, console

ON THE EXAM

State is taught here, in Domain 2, rather than waiting for Domain 6. Everything after module 4 depends on it.

Contents: Domain 4

STARTS	MODULE
1:31:54	7. Resources, Data Sources, References, Addressing
1:47:04	8. Variables, Outputs, Locals, and Precedence
2:03:39	9. Complex Types, Expressions and Functions
2:18:19	10. count, for_each and Dynamic Blocks
2:33:06	11. Dependencies and Lifecycle Rules
2:48:24	12. Custom Conditions and Check Blocks
3:01:38	13. Sensitive Data, Ephemeral Values, Write-Only

ON THE EXAM

Seven modules and nearly two hours. Domain 4 is the largest share of the exam and the part people find hardest.

Contents: Domains 5 to 7

STARTS	MODULE
3:17:48	14. Modules: Sourcing, Scope, Composition
3:32:46	15. Module Versioning and the Registry
3:45:08	16. Backends and State Locking
3:59:32	17. Drift, Refresh-Only, and the Refactoring Blocks
4:14:40	18. State Commands, and CLI Workspaces
4:28:46	19. Importing Existing Infrastructure
4:40:51	20. Logging, Debugging and Troubleshooting

ON THE EXAM

Modules 16 to 18 return to state, now that you have the language to describe what is happening to it.

Contents: Domain 8, and the rest

STARTS	MODULE
4:51:50	21. HCP Terraform: Workflows and Remote Operations
5:05:25	22. HCP Terraform: Workspaces, Projects, Variable Sets
5:19:18	23. HCP Terraform: Governance, Policy and Editions
5:32:57	24. Confusion Pairs
5:46:57	25. Version Currency: 1.12 Against Today
5:56:39	26. Exam Mechanics and Strategy

ON THE EXAM

Modules 24 to 26 add no new Terraform. They convert what you already know into marks.

Contents: The practice exams

STARTS	MODULE
6:07:12	Practice Exam 1 of 3 - 20 questions
6:23:57	Practice Exam 2 of 3 - 20 questions
6:40:30	Practice Exam 3 of 3 - 20 questions
6:58:25	End

> ON THE EXAM
>
> Sixty original questions. Take paper 1 straight after module 23, not at the end.

How a teaching module is built

- **Roughly twenty-five slides, twelve to fifteen minutes.** One sitting, not a lecture series.
- **It opens with the single idea** the module turns on, then develops it.
- **Every slide carries an 'on the exam' line** saying how that point is actually tested.
- **It closes with four or five worked scenarios**, then a recap you can rewatch alone.
- There are **109 worked scenarios** across the teaching modules, before the practice papers.

> ON THE EXAM
>
> The recap slide at the end of each module is designed to be watched on its own the week before the exam.

The scenarios, and how to use them

- **A situation, a question, four options, then silence.** Answer before the walkthrough starts.
- **Do not skip the pause.** Watching someone else reason is not the same as reasoning.
- **Every wrong option is explained**, not just the right one. Wrong answers are where the learning is.
- **Most wrong options are true statements** about something adjacent. That is what makes them plausible.
- **Several scenarios have more than one workable answer.** One sentence in the situation decides.

> ON THE EXAM
>
> If you find yourself agreeing with a wrong option's reasoning, that is the most valuable moment in the course.

The eight domains, and where they live

DOMAIN	MODULES
1. Infrastructure as Code concepts	Module 1
2. Terraform's purpose and architecture	Modules 2 to 4
3. Terraform basics and the core workflow	Modules 5 and 6
4. The configuration language	Modules 7 to 13
5. Modules	Modules 14 and 15
6. State management	Modules 16 to 18
7. Maintaining and troubleshooting	Modules 19 and 20
8. HCP Terraform	Modules 21 to 23

ON THE EXAM

Domain 4 gets seven modules because it is the largest and the hardest. State is taught early, in Domain 2, because everything later depends on it.

What is new in the 004 exam

- **depends_on and create_before_destroy lifecycle rules.** Module 11.
- **Validating configuration with custom conditions.** Module 12.
- **Ephemeral values and write-only arguments.** Module 13.
- **Organising HCP Terraform workspaces and projects.** Module 22.
- **If you studied for the 003 exam, these four are new.** Two of them describe features that did not exist then.

> ON THE EXAM
>
> Material written for the 003 exam will not cover these at all, and much of it predates the features entirely.

Things this course corrects

- **The S3 and DynamoDB locking pattern is deprecated.** Nearly every guide still teaches it.
- **The dependency lock file covers providers only.** Modules are not recorded in it, whatever you have read.
- **sensitive controls display, never storage.** This one appears in five separate modules.
- **Variable validation can reference other variables** since 1.9. Older guides say it cannot.
- Each correction is sourced from HashiCorp's own documentation, and the module says where.

> ON THE EXAM
>
> Everything on this list still appears in current, widely-read material. Working and current are not the same thing.

How to work through it

- **In order.** Later modules assume earlier ones, and reference them by number.
- **One or two modules a sitting.** Each is a self-contained argument; three in a row is diminishing returns.
- **Take practice exam 1 after module 23**, not at the end. It shows you where to spend the last week.
- **Then modules 24 to 26**, then papers 2 and 3 spaced a few days apart.
- **Rewatch the recap slides** in the final days. They exist for exactly that.

> ON THE EXAM
>
> The single most useful moment is your first practice paper. Take it before you feel ready, because that is what it is for.

The practice exams

- **Sixty original questions** across three papers. None is taken from any question bank.
- **Twenty per paper, mixed domains**, weighted to match the exam's likely emphasis.
- **Nine seconds to answer each.** The real exam gives you about a minute per question.
- **Four questions deliberately invert an earlier one**, so no mechanism becomes learnable as always-wrong.
- **Score by domain, not by total.** Where you miss matters far more than how many.

> ON THE EXAM
>
> The real exam gives you an objective-level report. Scoring these papers by domain is practice for reading it.

What this course does not do

- **It is not a substitute for writing Terraform.** This exam rewards people who have used it.
- **There are no labs.** Every module names what to try; do that in your own account.
- **It does not cover Terraform beyond 1.12**, except in module 25, which explains what changed.
- **It teaches no question bank.** If you want memorised answers, this is the wrong course.
- **It assumes you can read code**, and does not assume you have written Terraform before.

> ON THE EXAM
>
> Pair this with a real account and a small project. The combination passes; either alone is weaker.

Before module 1

- **Install Terraform**, and check the version. Anything from 1.10 onwards behaves as described here.
- **Get a free-tier cloud account** you can safely destroy things in.
- **Bookmark the exam content list**, and use its links. They point at 1.12 documentation deliberately.
- **Do not book the exam yet.** Book it after your first practice paper, when you know where you stand.
- **Module 26 covers booking, format and timing** when you get there.

> ON THE EXAM
>
> The exam is sixty minutes, one attempt, and no free retake. That is a good reason to book it once you have evidence.

Module 1 asks a question with a less obvious answer than it looks: **why does Infrastructure as Code exist at all?**

admin@sthithapragna.com

MODULE 01 · DOMAIN 1 - INFRASTRUCTURE AS CODE WITH TERRAFORM

Infrastructure as Code, and Why Terraform

What IaC actually gives you, what it does not, and where Terraform sits among the alternatives.

DOMAIN 1 of 8 COURSE WEIGHT 6% MODULE 1 of 27

admin@sthithapragna.com

Infrastructure as Code does not make infrastructure better. It makes infrastructure **reviewable, repeatable and diffable**.

What Infrastructure as Code is

- **Machine-readable definition files** that describe infrastructure, replacing manual console clicks and one-off scripts.
- **Stored in version control** alongside application code. This is the defining practice, not an optional extra.
- The file is the **source of truth**. A tool reads it and makes reality match.
- HashiCorp's framing: define infrastructure in a **human-readable** configuration, then provision it programmatically.
- It is **not** a deployment pipeline, not a monitoring system, and not a cost tool.

> ON THE EXAM
>
> If a question asks the primary purpose of IaC, the answer is programmatic creation and configuration of resources. Nothing about pipelines or price.

Declarative or imperative?

DECLARATIVE

You describe the destination

- State what should exist, not how
- The tool computes the steps
- Re-running converges to the same result
- Terraform, CloudFormation, ARM

IMPERATIVE

You describe the journey

- State the steps, in order
- You own the sequencing
- Re-running may repeat the work
- Shell scripts, raw API calls

WHERE TERRAFORM SITS

Terraform is declarative. You never write the order of operations; the dependency graph derives it.

The advantages the exam actually rewards

- **Versioning.** Every change has an author, a date, a reason and a rollback point.
- **Reuse and sharing.** One definition deploys to many regions, environments or teams.
- **Review before apply.** A proposed change can be read and approved before it touches anything.
- **Reduced configuration drift.** A declarative definition keeps deployed reality close to intent.
- **Automation and self-service.** Humans stop being the bottleneck for routine provisioning.

> ON THE EXAM
>
> Versioning, reuse and sharing is the single most repeated correct answer in this domain. Learn that phrase.

What IaC does NOT give you

- **It does not secure your credentials.** Config as code does nothing for secret handling.
- **It does not make resources cheaper.** The same instance costs the same.
- **It does not prevent manual modification.** Console access still changes things.
- **It does not guarantee zero downtime.** Replacing a resource can take it offline.
- **It does not autoscale anything.** Scaling is a property of what you built.

> ON THE EXAM
>
> Every one of these is a real distractor. They sound like benefits because they are benefits of something else.

Mutable or immutable infrastructure?

MUTABLE

Change the thing in place

- Patch and reconfigure live servers
- Each server accumulates its own history
- Drift grows quietly over time
- Debugging asks: what is different here?

IMMUTABLE

Replace the thing entirely

- Build a new version, swap it in
- Every instance is identical by construction
- No accumulated history to diverge
- Upgrades become far less complex

THE TIEBREAKER

The examinable advantage of immutable infrastructure is reduced complexity, not speed and not automation.

Terraform against writing API calls yourself

- **Terraform calls the same provider APIs** you would call. That part is genuinely no different.
- **The difference is the configuration.** Version-controlled, repeatable, and expressed as desired state.
- Terraform tracks **what it already built**, so a second run changes only what differs.
- **It does not replace provider APIs** with its own protocol. Providers are thin wrappers over those APIs.
- **Merely a wrapper, so no real difference** is a distractor. The wrapper is not the point; the state and the graph are.

> ON THE EXAM
>
> Both extremes are wrong. Terraform does not bypass the API, and it is not just a wrapper either.

The workflow this all produces

01

Write

Describe desired infrastructure in configuration files.

02

Plan

Terraform shows exactly what it would create, change or destroy.

03

Apply

The proposed change is executed against the provider APIs.

04

Review

The change lives in version control with an author and a reason.

ON THE EXAM

The plan step is why IaC enables review. It turns an intention into a readable, checkable artefact before anything happens.

Multi-cloud, hybrid cloud, service-agnostic

- **One workflow, many providers.** The same write, plan, apply cycle throughout.
- **Over three thousand providers:** clouds, on-premises, SaaS, DNS, monitoring.
- **Hybrid cloud** means public cloud plus on-premises in one configuration.
- **Service-agnostic** means it only needs a provider that can manage the thing.
- It does **not** make configurations portable. AWS resources stay AWS resources.

> ON THE EXAM
>
> One workflow across providers is the claim. One configuration that runs anywhere is not, and never was.

Terraform or a provider-native tool?

PROVIDER-NATIVE FITS WHEN

One vendor, and staying there

- Everything lives in a single cloud
- You want the vendor to hold state for you
- Deep, same-day support for new services
- CloudFormation, ARM, Deployment Manager

TERRAFORM FITS WHEN

The estate crosses a boundary

- More than one cloud, or cloud plus on-premises
- SaaS and infrastructure in one workflow
- One language and one skill set for all of it
- You accept owning the state file

THE TIEBREAKER

A native tool cannot provision another vendor's platform. That single limitation decides most questions in this pairing.

The trap: secrets and version control

- **Configuration belongs in version control.** Always the right answer.
- **Secrets do not.** Passwords, tokens and keys are never committed alongside it.
- A repository keeps **history**. Deleting a secret later does not remove it.
- Use a **secrets manager**, environment variables, or a run-time fetch.
- Terraform's answer is the **Vault provider**, plus ephemeral values. Module thirteen.

> ON THE EXAM
>
> Two near-identical questions, opposite answers. Configuration in version control is best practice. Secrets in version control never is.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Version, reuse and share configuration	The headline advantage of IaC over a GUI
Deploy into a second cloud or a SaaS platform	A vendor-native tool is the problem, not the answer
Replace rather than patch, identical every time	Immutable infrastructure, and less complex upgrades
Changes should be reviewed before they happen	The plan step, and code review on the repository
Programmatically create and configure resources	The primary purpose of IaC. Not pipelines, not cost
Stop anyone editing resources by hand	IaC cannot promise this. It detects, it does not prevent

ON THE EXAM

This domain is decided by matching a phrase to a definition. Slow down and read the exact claim being made.

> Halden Logistics runs its platform on a single public cloud. Engineers provision resources through the web console. Twice this quarter, an outage was traced to a setting somebody changed by hand and could not account for afterwards.
>
> The engineering director proposes adopting Terraform, and tells the board it will make it impossible for engineers to change infrastructure outside the approved process.

How should the platform lead respond to that claim?

A Agree. Adopting Terraform prevents changes made outside the configuration

B Correct it. Terraform surfaces such changes as drift, but cannot prevent them

C Agree, provided the configuration is stored in version control

D Correct it. Terraform prevents console changes only for immutable resources

◆ PAUSE AND DECIDE

How should the platform lead respond to that claim?

A Terraform prevents it

Overclaims. Terraform holds no authority over the provider's API. Prevention needs provider-side permissions or policy, not a tool choice.

B It surfaces drift, not prevents

Correct. Terraform compares desired state against reality and reports the difference. Detection is real and valuable; prevention is a different control.

C True with version control

Version control adds history and review to the configuration. It has no reach at all over what somebody does in a console.

D Only for immutable resources

Invents a rule. Immutability describes how you choose to change resources, not whether the provider accepts edits from elsewhere.

THE SIGNAL

IaC detects unmanaged change. Preventing it requires taking away the permission to make it.

> Meridian Health standardised on a single cloud vendor's native infrastructure-as-code service two years ago. The templates are mature and the team is comfortable with them.
>
> A new product requires a managed database in a second cloud, and must register its endpoints with an external monitoring SaaS at provisioning time. The team wants one review and approval process covering the whole deployment.

Which aspect of the current approach presents the difficulty?

A Native tools cannot store their templates in version control

B Native tools cannot provision resources outside their own vendor

C Native tools cannot express dependencies between resources

D Native tools cannot be reused across regions

◆ PAUSE AND DECIDE

Which aspect of the current approach presents the difficulty?

A. Cannot use version control

False. A native template is a text file and versions perfectly well. Version control is a practice, not a tool feature.

B. Cannot leave their own vendor

Correct. Two boundaries are crossed here, a second cloud and a SaaS platform, and the native service reaches neither.

C. Cannot express dependencies

False, and it is one of the things native tools do well. Dependency ordering is core to any declarative provisioning tool.

D. Cannot be reused across regions

False. Reuse across regions is a standard, well-supported pattern. The scenario never mentions regions, which is the tell.

THE SIGNAL

The scenario names two boundary crossings and one process requirement. The boundaries decide it; the process requirement explains why it matters.

> A team runs forty application servers. Patching is done in place by a configuration management tool. Over eighteen months, six servers have developed behaviour the team cannot reproduce anywhere else, and troubleshooting usually begins by comparing one server against another.
>
> An architect proposes moving to an immutable model: build a new machine image for every release and replace the fleet.

What is the principal advantage the team gains?

A. Upgrades complete more quickly than in-place patching

B. Upgrades become less complex, because instances cannot diverge

C. Upgrades happen automatically without team involvement

D. Upgrades can be applied in place with no downtime

◆ PAUSE AND DECIDE

What is the principal advantage the team gains?

A. Quicker upgrades

Usually the opposite. Building an image and replacing forty machines takes longer than patching them where they stand.

B. Less complex upgrades

Correct. Every instance is built the same way from the same image, so the class of problem described in the scenario stops occurring.

C. Automatic upgrades

Automation is orthogonal. In-place patching here is already automated by a configuration management tool, and it still produced the problem.

D. In-place with no downtime

Describes mutable infrastructure, which is what they are leaving. Replacement can also mean a brief interruption per instance.

THE SIGNAL

Six servers nobody can reproduce is the planted detail. It names divergence, and divergence is exactly what replacement removes.

> A team is moving its Terraform configuration into a shared Git repository so that every engineer can run the same deployment. The configuration needs a database administrator password.
>
> An engineer suggests committing a variables file containing the password, so that anyone who clones the repository can apply immediately without extra setup. She notes the repository is private and access is limited to the eight people on the team.

How should the team proceed?

A Commit the file. A private repository with eight members is an acceptable boundary

B Keep the password out of the repository and supply it from a secrets manager or environment

C Keep the Terraform configuration out of version control entirely, since it references a secret

D Commit the file, then remove the password in a follow-up commit before granting wider access

◆ PAUSE AND DECIDE

How should the team proceed?

A Private repo is enough

Access scope is not the issue. The secret is now in permanent history, in plaintext, and repository membership changes over time.

B Keep it out entirely

Correct. Configuration is versioned; the secret arrives at run time from a secrets manager, an environment variable, or the Vault provider.

C Don't version the config

Overcorrects, and throws away the entire practice. Referencing a secret is normal; storing its value is the problem.

D Commit then remove later

The most dangerous option, because it feels like a plan. A later commit does not erase history. That password is compromised from the moment it lands.

THE SIGNAL

Two questions that look identical have opposite answers. Configuration in version control, yes. Secrets in version control, never.

Module 1 recap

- **Version, reuse and share.** The headline advantage of code over a graphical interface.
- **Primary purpose:** programmatically create and configure resources. Not pipelines, not cost, not scaling.
- IaC **detects** unmanaged change. It never prevents it.
- **Immutable means less complex,** not quicker. Replacement removes divergence, and often takes longer.
- **One workflow across providers** is the multi-cloud claim. Portable configuration is not.
- **Configuration in version control, always. Secrets, never.**

> ON THE EXAM
>
> Domain 1 is definitional. It is the cheapest domain to score full marks in, and the easiest to lose marks in by reading quickly.

MODULE 02 · DOMAIN 2 - TERRAFORM FUNDAMENTALS

Architecture: Core, Providers, the Registry

How Terraform reaches an API it knows nothing about, and how you tell it which plugin to use.

DOMAIN 2 of 8 COURSE WEIGHT 12% MODULE 2 of 27

admin@sthithapragna.com

Terraform core knows nothing about any cloud. **Every** piece of platform knowledge lives in a plugin.

Two halves, released separately

- **Terraform core.** Parses configuration, builds the dependency graph, manages state, decides what must change.
- **Providers.** Plugins that translate those decisions into real API calls against a specific platform.
- Providers are separate programs, written in Go, that core launches and talks to over a documented plugin protocol.
- **They version independently.** Terraform 1.12 and AWS provider 5.90 are two different version numbers with two different meanings.
- A provider is **not** part of the Terraform binary and is not downloaded until you run **terraform init**.

> ON THE EXAM
>
> Independent versioning is why required_version and required_providers are two separate settings. They constrain different things.

What happens when you declare a resource

01
Declare
You write a resource with a type such as aws_instance.

02
Resolve
Core reads the first word of the type to find the provider's local name.

03
Launch
Core starts the installed plugin and hands it the configuration.

04
Translate
The provider turns create, read, update or delete into real API calls.

05
Return
The provider reports back what exists, and core records it in state.

ON THE EXAM

The first word of a resource type is not decoration. It is how Terraform chooses which plugin handles that resource.

What a provider is responsible for

- **Defining resource types and data sources.** Everything you can manage comes from some provider.
- **Authenticating** to the platform, and knowing its endpoints and regions.
- Mapping Terraform's lifecycle onto that platform's API, including what forces a replacement rather than an update.
- **Over three thousand providers** exist: clouds, on-premises systems, SaaS, DNS, monitoring, databases, version control.
- A provider does **not** hold state, plan changes, or order operations. Those are core's job.

> ON THE EXAM
>
> If a behaviour is platform-specific, it is the provider's. If it is about graphs, plans or state, it is core's.

The three registry tiers

TIER	WHO MAINTAINS IT
Official	HashiCorp. Published in the hashicorp namespace
Partner	A third party in HashiCorp's Technology Partner Program, with a verified publisher
Community	An individual or organisation from the community. No formal verification

ON THE EXAM

The tier tells you who supports it, not how it installs. All three install identically with terraform init.

A source address has three parts

- **[HOSTNAME/]NAMESPACE/TYPE**. For example, registry.terraform.io/hashicorp/aws.
- **Hostname** is optional and defaults to registry.terraform.io, the public registry.
- **Namespace** is the publishing organisation. **Type** is the platform, and must be unique in that namespace.
- The address is the provider's **global identifier**. Terraform normalises it and uses it internally to tell providers apart.
- Republish the same provider under a different namespace and Terraform treats it as an **entirely different provider**.

> ON THE EXAM
>
> Source addresses are case-insensitive, but write types in lowercase. hashicorp/aws and registry.terraform.io/hashicorp/aws are the same provider.

Source address or local name?

SOURCE ADDRESS

Global, and used once

- Identifies the provider worldwide
- Appears only in required_providers
- Written as namespace and type
- Same for everyone, everywhere

LOCAL NAME

Per-module, and used everywhere else

- Your name for it inside this module
- Must be unique within the module
- Used by provider blocks and resource prefixes
- You may choose it, but rarely should

THE TIEBREAKER

The local name is the key in required_providers. The source address is its value. Everything after that refers to the local name.

required_providers: declare before you configure

- Nested inside the top-level **terraform** block. Every module should declare the providers it requires.
- Each entry takes a **source** and, strongly recommended, a **version** constraint.
- **Omit source and Terraform implies registry.terraform.io/hashicorp/ plus your local name.** A backwards-compatibility rule from 0.13.
- That implication is why a community provider silently resolves to a HashiCorp one that may not exist.
- **terraform init** reads this block, downloads the plugins, and records exact versions in the lock file.

> ON THE EXAM
>
> Always write source explicitly. The implied hashicorp namespace is a transitional aid, not a convenience.

required_providers or a provider block?

REQUIRED_PROVIDERS

Which plugin, and which version

- Inside the terraform block
- Names the source address
- Constrains the version
- Read by terraform init to install

PROVIDER BLOCK

How that plugin should behave

- Top-level block in your configuration
- Region, endpoints, credentials, defaults
- Optional if the provider needs no settings
- Read at plan and apply time

THE TIEBREAKER

Declaration installs. Configuration parameterises. A provider block alone will not install anything.

Configuring a provider, and authenticating it

- The **provider** block is labelled with the **local name**, not the source address.
- Arguments are provider-specific. Region, endpoint and profile are typical.
- **Never hardcode credentials in a provider block.** The configuration is version-controlled; the credentials would be too.
- Prefer **environment variables**, the platform's own credential chain, or a dynamically issued short-lived credential.
- HCP Terraform's answer is **dynamic credentials**: short-lived, issued per run, never stored. Module twenty-one.

> ON THE EXAM
>
> Same rule as module one's secrets scenario, one level lower. Configuration in version control, credentials never.

Aliases: several configurations of one provider

- A provider block with an **alias** argument creates an **additional** configuration alongside the default.
- The block without an alias is the **default**, used by any resource that does not say otherwise.
- **This is the answer to two regions, two accounts, or two subscriptions** in one configuration.
- A resource selects one with the **provider** meta-argument, written as local name dot alias.
- Aliases do not install anything extra. It is **one plugin**, configured more than once.

> ON THE EXAM
>
> Objective 2c is about writing configuration with multiple providers. Aliases are how you use the same provider twice.

Which configuration does a resource use?

THE SITUATION	WHAT TERRAFORM DOES
`No provider meta-argument`	The default configuration for the type prefix
`provider = aws.tokyo`	The aliased configuration named
`Two providers, same type name`	You must set provider on every resource
`Child module needs an alias`	Pass it with the providers meta-argument
`Child module defines its own`	Discouraged. Pass configuration in instead

ON THE EXAM

Modules inherit default configurations only. Aliased ones must be passed in.

When two providers want the same name

- Local names must be unique per module. Two providers of type **http** from different namespaces will collide.
- HashiCorp's recommendation: **compound local names** joining namespace and type, such as hashicorp-http and mycorp-http.
- Terraform can then no longer infer the provider from the resource type prefix.
- So every affected resource and data source must carry an explicit **provider** meta-argument.
- More typing, and deliberately so. Ambiguity here is far more expensive than verbosity.

> ON THE EXAM
>
> Renaming breaks the type-prefix shortcut. That is the cost, and it is why preferred local names are the default advice.

Two providers that break the pattern

- **The built-in provider.** Its source address is terraform.io/builtin/terraform and it powers terraform_remote_state.
- It is **inside the Terraform binary**, so you never declare it and terraform init never downloads it.
- **hashicorp/terraform is a different, obsolete thing.** It should never appear in required_providers.
- **In-house providers** need not be published. Use a hostname on a domain you control, such as terraform.example.com.
- Distribute them through a **filesystem mirror** or a private registry. The address needs a hostname; the hostname need not resolve.

> ON THE EXAM
>
> A source address is an identifier first and a download location second. It does not have to point at a real service.

Where the plugins actually land

- **terraform init** downloads providers into the **.terraform** directory inside your working directory.
- **.terraform is disposable and belongs in .gitignore.** It is a working directory, not part of your configuration.
- The **lock file** is the opposite: .terraform.lock.hcl is committed. Module three covers it properly.
- Provider binaries are large. Downloading them per project, per branch, adds up quickly.
- A **plugin cache directory** shares one copy across every configuration on the machine.

> ON THE EXAM
>
> Two files named almost identically, opposite treatment. Ignore the .terraform directory. Commit the .terraform.lock.hcl file.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Two regions or two accounts	Provider aliases, and the provider meta-argument
Wrong provider installed, or not found	source omitted; the hashicorp namespace implied
Proprietary system, never published	In-house provider from a filesystem mirror
Which versions are acceptable	required_providers, not the provider block
Which region the provider uses	The provider block, not required_providers
Child module needs the alias	The providers meta-argument on the module block

ON THE EXAM

Ask whether the requirement is about installing a plugin or configuring one.

> Coriander Media runs its main platform in one cloud region. A new regulatory requirement means customer records for one market must be stored in a second region, in the same cloud account, while the rest of the platform stays where it is.
>
> The team wants both sets of resources described in a single configuration, planned and applied together, so the dependency between them is visible.

How should the team configure this?

A Declare the provider twice in required_providers, once per region

B Add a second provider block with an alias, and set the provider meta-argument on the second region's resources

C Create a second root module and apply it separately

D Use a CLI workspace per region, switching between them before each apply

◆ PAUSE AND DECIDE

How should the team configure this?

A Declare it twice

required_providers holds one entry per local name. It says which plugin to install, and region is not part of that. Duplicate keys are invalid.

B Alias plus meta-argument

Correct. One plugin, two configurations. Both regions appear in one plan, so the dependency between them is real and visible.

C A second root module

Works, and splits the plan in two. The scenario explicitly asks for one plan, which is the sentence that rules this out.

D A workspace per region

CLI workspaces give one configuration several independent states. Only one is active at a time, so the two regions could never appear in one plan.

THE SIGNAL

Planned and applied together, with the dependency visible, is the deciding sentence. Anything that splits the plan fails it.

> An engineer adds a DNS provider published by a vendor called Northwind to a module. She writes a required_providers entry with the local name dns and a version constraint, then runs terraform init.
>
> Terraform reports that it has installed a provider, but every Northwind resource type in her configuration is rejected as unknown. The version installed does not match anything on Northwind's release page.

What is the most likely cause?

A. Community providers cannot be installed automatically by terraform init

B. The source argument was omitted, so Terraform resolved hashicorp/dns instead

C. The dependency lock file pinned an older version and must be deleted

D. The local name must always match the vendor's namespace, not the type

◆ PAUSE AND DECIDE

What is the most likely cause?

A Cannot install community providers

True before Terraform 0.13, and false ever since. Official, partner and community providers all install identically from the registry.

B source omitted, hashicorp implied

Correct. With no source, Terraform implies registry.terraform.io/hashicorp/ plus the local name, so it installed hashicorp/dns.

C Stale lock file

A lock file constrains versions of the provider it already recorded. It cannot cause a different provider to be chosen.

D Local name must match namespace

Invents a rule. Local names are yours to choose and conventionally match the type, never the namespace.

THE SIGNAL

It installed successfully and the resource types are unknown. That combination means the wrong provider, not a broken install.

A bank has written its own Terraform provider for an internal mainframe scheduling system. The provider must be usable by about forty teams across the organisation.

Security has stated that neither the provider binary nor its documentation may be published anywhere outside the bank's own network. The platform team would rather not operate another internal service if it can be avoided.

What should the platform team do?

A. Publish it to the public registry under a private namespace

B. Give it a source address on a bank-controlled hostname and distribute it via a filesystem mirror

C. Omit it from required_providers and have each team place the binary manually

D. Rewrite the functionality as a module instead, since providers must be publicly published

◆ PAUSE AND DECIDE

What should the platform team do?

A Public registry, private namespace

The public registry has no private namespaces. A namespace is an organisation name, not an access control, and everything published there is public.

B Own hostname plus filesystem mirror

Correct. The address identifies the provider; it need not resolve. A mirror avoids running a registry, and a real one can be added later at the same address.

C Omit and place manually

Omitting source implies the hashicorp namespace, which is scenario two's failure. Providers must be declared to be used.

D Rewrite as a module

A module composes existing resource types. It cannot invent one, so it cannot reach a system no provider knows about.

THE SIGNAL

A source address is an identifier that happens to describe a download location. The hostname does not have to exist.

> A configuration needs HashiCorp's http provider to fetch a document, and a second http provider published by a monitoring vendor to register a health check. Both providers use http as their preferred local name.
>
> An engineer adds both to required_providers and terraform init fails before downloading anything.

What must the engineer do to make this work?

A Give each an alias in its provider block, keeping the local name http for both

B Use distinct compound local names, then set the provider meta-argument on every affected resource

C Split the two providers into separate child modules so the names never meet

D Keep only the HashiCorp provider, since duplicate type names are not supported

◆ PAUSE AND DECIDE

What must the engineer do to make this work?

A Alias them both

Aliases distinguish configurations of one provider. These are two different plugins with two different source addresses, so the collision is unresolved.

B Compound names plus meta-argument

Correct, and both halves are required. Unique names let init proceed; the meta-argument replaces the type-prefix inference that renaming destroyed.

C Separate child modules

Local names are per-module, so this genuinely resolves the collision. It also splits related resources apart to satisfy a naming rule, which is the tail wagging the dog.

D Drop one provider

Duplicate type names across namespaces are explicitly supported. This abandons a stated requirement to avoid a solved problem.

THE SIGNAL

Option A is the trap. Aliases separate configurations of one provider; they cannot separate two different providers.

Module 2 recap

- **Core knows nothing platform-specific.** Providers are separate plugins, separately versioned.
- **Source address is global; local name is per-module.** The local name is the key, the source is the value.
- **Omit source and you get the hashicorp namespace.** Always write it explicitly.
- **required_providers installs. The provider block configures.** Different jobs, constantly confused.
- **Aliases give one provider several configurations.** Two regions or two accounts in one plan.
- Modules inherit **default** configurations only. Aliased ones must be passed in.

> ON THE EXAM
>
> Official, Partner and Community describe who supports a provider. They install identically.

MODULE 03 · DOMAIN 2 - TERRAFORM FUNDAMENTALS

Provider Versioning and the Lock File

Constraints say what is acceptable. The lock file says what was chosen. Two different jobs.

DOMAIN 2 of 8 COURSE WEIGHT 12% MODULE 3 of 27

admin@sthithapragna.com

A constraint says what is **acceptable**. The lock file says what was **chosen**.

Providers use semantic versioning

- **MAJOR.MINOR.PATCH.** A major bump signals breaking changes, minor adds features, patch fixes bugs.
- Provider versions move fast. A widely used cloud provider may release several times a month.
- This is why constraints matter. Without one, Terraform accepts **any** version, including tomorrow's breaking release.
- The **version** argument is optional. HashiCorp strongly recommends writing one for every provider.
- Terraform selects the **newest version that satisfies every constraint** in the whole configuration, including child modules.

> ON THE EXAM
>
> Constraints from the root module and every child module are combined. All of them must be satisfied by one selected version.

The constraint operators

OPERATOR	MEANING
`= 1.4.0, or just 1.4.0`	Exactly this version. Nothing else
`!= 1.4.0`	Any version except this one
`>= 1.2.0`	This version or newer. The usual floor
`<= 1.9.0`	This version or older
`> 1.2.0 and < 2.0.0`	A range. Combine with a comma; all conditions must hold
`~> 1.4.0`	Pessimistic. Allows the rightmost part to increase, nothing above it

ON THE EXAM

Comma-separated constraints are combined with AND, never OR. Every one must be satisfied.

The pessimistic operator, and where it stops

- **~> allows the rightmost component you wrote to increase.** Everything to its left is frozen.
- **~> 1.4.0** accepts 1.4.1 and 1.4.9. It rejects 1.5.0. Patch releases only.
- **~> 1.4** accepts 1.5 and 1.9. It rejects 2.0. Minor releases allowed.
- How many components you write changes the meaning entirely. This is the detail questions are built on.
- It is **not** the same as greater than or equal to. There is always an implied ceiling.

> ON THE EXAM
>
> Count the components. Three components allow patch only; two components allow minor. Same operator, different answer.

Working an example

CONSTRAINT WRITTEN	CHOSEN FROM 1.4.0, 1.4.7, 1.5.2, 2.0.1
`~> 1.4.0`	1.4.7. Patch may rise, minor may not
`~> 1.4`	1.5.2. Minor may rise, major may not
`>= 1.4.0`	2.0.1. No ceiling at all, so the newest wins
`>= 1.4.0, < 2.0.0`	1.5.2. The comma adds a ceiling by hand
`1.4.0`	1.4.0. An exact pin, not a floor
`No version argument`	2.0.1. Any version qualifies, including breaking ones

ON THE EXAM

Terraform always takes the newest version that satisfies every constraint. The question is only ever where the ceiling sits.

required_version or required_providers?

REQUIRED_VERSION

Constrains Terraform itself

- Sits directly in the terraform block
- One setting, no source, no names
- Checked before anything else runs
- Refuses to proceed on the wrong CLI

REQUIRED_PROVIDERS VERSION

Constrains a plugin

- Sits inside required_providers
- One entry per provider, beside its source
- Resolved during terraform init
- Selects which plugin version to download

THE TIEBREAKER

Both live in the terraform block and both take the same operators. They constrain completely different software.

Where to be strict, and where not to be

- **Every module should declare at least a minimum**, using >= for the oldest version it is known to work with.
- **A root module should also set a maximum**, typically with ~>, to prevent an accidental jump to a breaking release.
- **A reusable module should not set a maximum.** HashiCorp's guidance is explicit about this.
- A ceiling inside a shared module forces every consumer to upgrade the module before they can upgrade anything else.
- Instead: state a minimum, document known incompatibilities, and **let the root module decide the ceiling**.

> ON THE EXAM
>
> The right constraint depends on where the code sits. Root modules protect themselves. Shared modules must not constrain their consumers.

The dependency lock file

- **.terraform.lock.hcl**, created and updated by **terraform init**. Terraform 0.14 onwards.
- **Commit it.** That is the point: everyone and every pipeline selects identically.
- It belongs to the **configuration as a whole**, not one per child module.
- HCP Terraform, Terraform Enterprise and the CLI all obey it.
- Do **not** hand-edit it. Manual edits may be lost, and hashes must match.

> ON THE EXAM
>
> Contrast with module two: the .terraform directory is ignored, the .terraform.lock.hcl file is committed. Similar names, opposite treatment.

Inside a lock file entry

FIELD	WHAT IT RECORDS
`Block label`	The provider's fully-qualified source address
`version`	The exact version that was selected
`constraints`	The constraint from your configuration that produced that selection
`hashes`	Checksums of the provider packages, verified on every install
`h1:` prefix	The current hashing scheme, computed over the package contents
`zh:` prefix	The legacy zip hash. One per platform the provider was built for

ON THE EXAM

Both version and constraints are recorded, so you can see the decision and the rule that produced it side by side.

What the lock file does and does not track

LOCKED

Providers

- Exact version recorded
- Checksums recorded and verified
- Same selection on every machine
- Changes only when you ask

NOT LOCKED

Modules

- No version recorded, ever
- Terraform picks newest matching the constraint
- Two machines can select differently
- Pin exactly if you need reproducibility

THE TIEBREAKER

HashiCorp is explicit: the lock file tracks only provider dependencies. Many study guides state otherwise, and they are wrong.

init, and the -upgrade flag

- **terraform init** reads the lock file first and reuses the recorded version whenever it still satisfies your constraints.
- **New providers** are resolved and added. Existing ones are left exactly where they are.
- Change a constraint so the locked version no longer fits, and init **fails** rather than silently choosing a new one.
- The error names the fix: **terraform init -upgrade**, which re-resolves every provider against the current constraints.
- That refusal is deliberate. Upgrading a provider is a decision, so Terraform makes you state it.

> ON THE EXAM
>
> Plain init is conservative by design. Upgrading within your constraints requires -upgrade, every time.

Mixed platforms, and terraform providers lock

- **init records hashes only for the platform it ran on.** Other platforms then fail.
- **terraform providers lock** writes checksums **without installing anything**.
- Use **-platform=os_arch**, once per platform your team uses.
- It also matters with a **mirror**, which never contacts the origin registry.
- Pre-recorded checksums verify the **mirror** against what the publisher signed.

> ON THE EXAM
>
> A checksum failure on a colleague's machine is usually a missing platform hash, not a corrupted provider.

When a checksum does not match

- Every install verifies the downloaded package against the hashes already in the lock file.
- **Usually benign:** a platform whose hash was never recorded. Fix with terraform providers lock and the right -platform.
- **Occasionally serious:** the package genuinely differs from the one the publisher signed.
- Never resolve it by deleting the lock file. That discards the evidence and re-records whatever you were just served.
- **terraform providers lock** prints signing information, but cannot judge whether a publisher is trustworthy.

> ON THE EXAM
>
> Deleting the lock file makes the error disappear without answering the question the error was asking.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Locked provider does not match the constraint	terraform init -upgrade
Everyone must select the same provider version	Commit the lock file
A module changed without anyone editing configuration	Modules are not locked. Pin an exact version
Checksum error only on some machines	terraform providers lock with -platform
A shared module used by many teams	Set a minimum only. No ceiling
Must never move to the next major version	The pessimistic operator, or an explicit upper bound

ON THE EXAM

Ask first whether the situation is about the acceptable range or the recorded selection. That splits nearly all of these.

A team raises a provider constraint in required_providers from a version that has a known bug to a newer one, commits the change, and pushes.

The pipeline fails during initialisation. The error states that the locked provider version does not match the configured version constraint. Nothing else in the configuration changed, and the same command worked yesterday.

What should the team do?

A Delete .terraform.lock.hcl from the repository and re-run the pipeline

B Run terraform init -upgrade, and commit the updated lock file

C Add the new version to required_version in the terraform block

D Run terraform apply -refresh=false to bypass provider resolution

◆ PAUSE AND DECIDE

What should the team do?

A Delete the lock file

Produces a working pipeline and throws away every recorded checksum. It treats a message asking for a decision as an obstacle.

B init -upgrade, then commit

Correct, and both halves matter. The flag re-resolves against the new constraint; committing the result is what makes the selection reproducible.

C Set required_version

Constrains the Terraform binary, not the provider. The pairing from earlier in this module, appearing as a distractor.

D Apply with -refresh=false

Refresh controls whether Terraform re-reads remote state before planning. It has nothing to do with provider selection, and init must succeed first anyway.

THE SIGNAL

The error message names its own fix. Terraform refuses on purpose, because changing a provider version should be deliberate.

A platform team commits .terraform.lock.hcl for every configuration and states in its standards document that all deployments are now reproducible.

Three weeks later, a plan in an untouched configuration proposes replacing a load balancer. No engineer changed any file. The configuration uses a registry module with the constraint version = "~> 4.0", and the module authors released 4.3.0 that morning.

Why did the plan change?

A The lock file recorded the module version but the constraint overrode it

B The lock file does not track modules, so Terraform selected the newest version matching ~> 4.0

C A provider upgrade was applied automatically during terraform init

D Committed lock files expire after a set period and must be regenerated

◆ PAUSE AND DECIDE

Why did the plan change?

A Constraint overrode the lock

Assumes a module entry exists in the lock file. There has never been one, so there was nothing for a constraint to override.

B Modules are not locked

Correct. The lock file covers providers only. Terraform re-resolves module versions on every init and takes the newest that fits.

C Automatic provider upgrade

Backwards. Plain init is conservative and reuses the locked provider version. That is exactly what the lock file is for.

D Lock files expire

Invents a mechanism. A lock file has no lifetime and no expiry; it changes only when a command rewrites it.

THE SIGNAL

The team's belief was half right. Providers were reproducible. Modules never were, and the standards document hid the gap.

> An engineering team is split between developers on Apple silicon laptops and a build pipeline running on Linux. Whoever runs terraform init first commits a lock file that then fails checksum verification for the other group.
>
> The team currently works around it by deleting the lock file whenever the error appears.

What should the team do instead?

A. Add .terraform.lock.hcl to .gitignore so each environment keeps its own

B. Run terraform providers lock with a -platform argument for each platform, and commit the result

C. Standardise every developer onto Linux virtual machines

D. Pin every provider to an exact version so checksums stop changing

◆ PAUSE AND DECIDE

What should the team do instead?

A Gitignore the lock file

Removes the error and the guarantee together. Each environment then selects independently, which is precisely what the file prevents.

B providers lock per platform

Correct. The command records official checksums for every platform named, without installing anything, so one committed file satisfies both groups.

C Standardise on Linux

Solves it by removing the variation, at real cost to developers, when the tool already has a purpose-built command for exactly this.

D Pin exact versions

Confuses version with platform. The version was never in dispute; the missing hash is for a different operating system and architecture.

THE SIGNAL

Option D is the trap. The mismatch is about which platforms have recorded hashes, not about which version was chosen.

> An engineer maintains a networking module published to an internal registry and used by nineteen teams. She has tested it against version 5.40 of the cloud provider and wants to prevent teams hitting breaking changes in the next major release.
>
> She proposes adding version = "~> 5.40" to the module's own required_providers block.

What should a reviewer advise?

A. Approve it. A pessimistic constraint is the correct way to prevent major-version breakage

B. Advise a minimum only, and let each root module set its own ceiling

C. Advise pinning an exact version, so all nineteen teams are guaranteed identical behaviour

D. Advise removing the version argument, since the lock file already controls selection

◆ PAUSE AND DECIDE

What should a reviewer advise?

A **Approve the ceiling**

Right technique, wrong place. HashiCorp advises against maximum constraints in modules intended for reuse, because the ceiling propagates to every consumer.

B **Minimum only**

Correct. Declare the oldest version known to work, document incompatibilities, and leave the ceiling to the root module that owns the deployment.

C **Pin exactly**

The strictest version of the same mistake. Nineteen teams could then never move the provider at all until the module was re-released.

D **No version at all**

The lock file records a selection; it does not decide which versions are acceptable. Without a constraint, any version qualifies.

THE SIGNAL

Constraints combine across every module. A ceiling in a shared module becomes a ceiling in every configuration that uses it.

Module 3 recap

- **Constraints define a range. The lock file records a selection.** Nearly every question separates on that.
- **~> freezes everything left of the component you wrote.** Count the components before answering.
- **required_version is Terraform. required_providers version is a plugin.**
- **The lock file tracks providers only.** Modules re-resolve on every init. Pin exactly if you need them still.
- **Plain init is conservative.** Moving within your constraints needs terraform init -upgrade.
- **Shared modules set a floor, never a ceiling.** Root modules set both.

> ON THE EXAM
>
> Commit .terraform.lock.hcl. Ignore the .terraform directory. Never hand-edit either.

MODULE 04 · DOMAIN 2 - TERRAFORM FUNDAMENTALS

State: What It Is, Why It Exists, What It Costs

The binding between your configuration and the real world, and the four jobs it does.

DOMAIN 2 of 8 COURSE WEIGHT 12% MODULE 4 of 27

admin@sthithapragna.com

State's primary purpose is to store **bindings** between remote objects and the resource instances in your configuration.

admin@sthithapragna.com

Four reasons state exists

- **Mapping to the real world.** Which remote object corresponds to which resource instance.
- **Metadata.** Resource dependencies, and which provider configuration was last used.
- **Performance.** A cached copy of every resource's attributes, so plans need not query everything.
- **Syncing.** A shared, lockable record so a team does not apply over each other.
- HashiCorp's position: state is **required**. Removing it would move enormous complexity somewhere else, not remove it.

> ON THE EXAM
>
> These four are the documented list, and questions are often built directly from them. Learn them as a set.

Mapping: why not just use tags?

- Early Terraform prototypes had **no state file** and identified resources by cloud tags instead.
- **Not all resources support tags,** and not all providers support them at all.
- So Terraform keeps its own structure, recording each object's identity when it creates it.
- **Each remote object must bind to exactly one resource instance.** Terraform guarantees this when it creates the object.
- Import the same object twice and the mapping is **ambiguous**. Terraform may then act unexpectedly against it.

> ON THE EXAM
>
> The one-to-one rule is why importing carefully matters. Creation is safe by construction; import is your responsibility.

Metadata: the part that surprises people

- State records **resource dependencies**, not just identities.
- Normally Terraform derives dependency order from the configuration, which is where the graph comes from.
- **But delete a resource block and its dependencies vanish from the configuration too.**
- Terraform still has to destroy that object, and still has to destroy things **in the right order**.
- So state retains the **most recent set of dependencies**. It also records which provider configuration was last used.

> ON THE EXAM
>
> Dependency metadata exists for deletion. Once the configuration is gone, state is the only place that ordering survives.

Performance: the attribute cache

- State caches the **attribute values** of every resource, not only its identity.
- HashiCorp calls this the **most optional** part of state. It is a performance feature, nothing more.
- **By default, every plan and apply refreshes every resource** by querying the provider.
- On a large estate that becomes slow, because it is one or more API calls per resource, every time.
- **-refresh=false** skips the query and plans from the cache. Faster, and the plan may be based on stale data.

> ON THE EXAM
>
> Refreshing is the default because correctness beats speed. Turning it off is a deliberate trade, not an optimisation.

Three things called refresh

WHAT YOU WRITE	WHAT IT DOES
`Default plan or apply`	Refreshes every resource, then compares against configuration
`-refresh=false`	Skips the query. Plans from cached attributes, which may be stale
`plan -refresh-only`	Compares state against reality only. Proposes no infrastructure changes
`apply -refresh-only`	Writes those findings into state, after you approve them
`terraform refresh`	Deprecated since 0.15.4. Updates state with no preview and no prompt

ON THE EXAM

refresh=false skips reality. refresh-only looks at reality and nothing else. Opposite meanings, one shared word.

Syncing: state is a shared resource

- **State on a laptop is state one person has.** A second engineer plans against a different picture of reality.
- **Remote state** puts one authoritative copy where everyone and every pipeline reads it.
- **State locking** stops two applies running at once, which is how state gets corrupted.
- Locking is per backend. Not every backend supports it, and that is a real selection criterion.
- Backends, locking and migration are module sixteen. Here, only the reason they exist.

> ON THE EXAM
>
> Two people applying against separate local state files will each destroy what the other created. Sharing state is not optional for a team.

The three inputs to every plan

01

Configuration

What you have declared should exist. The desired state.

02

State

What Terraform believes exists, and which object is which.

03

Reality

What the provider reports right now, read during refresh.

04

Plan

The difference, expressed as creates, updates, replacements and destroys.

ON THE EXAM

A plan is a three-way comparison, not a two-way one. Most confusing plan output comes from forgetting the middle input.

Desired state or current state?

DESIRED STATE

Your configuration files

- Written by you, in HCL
- Version-controlled and reviewed
- Describes the end result
- Changing it changes nothing on its own

CURRENT STATE

The state file, refreshed

- Written by Terraform, never by hand
- Records identities and attributes
- Describes what exists now
- Changing it changes nothing either

THE TIEBREAKER

Neither file touches infrastructure. Only apply does. Editing state to 'fix' something changes what Terraform believes, not what is true.

What the file actually looks like

- **JSON**, in a documented but internal format. It is not a configuration file and not written in HCL.
- Carries a **version** for the format, a **serial** that increments on change, and a **lineage** identifying the state's history.
- By default it is **terraform.tfstate** in the working directory, with **terraform.tfstate.backup** holding the previous version.
- **The format is internal.** HashiCorp reserves the right to change it, so never write tooling that parses it directly.
- Read it with **terraform show** or the state commands, which give you a supported interface.

> ON THE EXAM
>
> That it is JSON does not make it a file you may edit. Serial and lineage exist so Terraform can detect a state that has been swapped or rolled back.

State holds your secrets in plain text

- **Any attribute a provider returns is stored, including passwords, keys and tokens.** In plain text.
- **Marking a variable sensitive does not change this.** It hides values in CLI output. State is unaffected.
- So HashiCorp's guidance is blunt: **treat state itself as sensitive data**.
- Practically: encrypt it at rest, restrict who can read the backend, and never commit it to a repository.
- **Ephemeral values and write-only arguments** keep specific secrets out of state entirely. Module thirteen.

> ON THE EXAM
>
> sensitive controls what is printed. It has never controlled what is stored. That gap is the single most common misconception about state.

When state is lost

- **Terraform stops knowing your infrastructure exists.** The resources keep running.
- The next plan proposes to **create everything from scratch**.
- **terraform.tfstate.backup** holds the previous version. A bad write, not a deleted backend.
- Real protection is a **versioned remote backend**.
- Last resort: **import every resource back**. Slow and manual. Module nineteen.

> ON THE EXAM
>
> Losing state does not destroy anything. It destroys Terraform's knowledge, which is worse, because the next apply looks reasonable.

What state is not

A TEMPTING BELIEF	THE REALITY
State is a backup of my infrastructure	It records identities and attributes, and recreates nothing
Marking values sensitive protects state	sensitive affects output only. State is plain text
State is optional for small projects	Terraform always uses state. Local state is still state
I can hand-edit state to fix a problem	You change what Terraform believes. Reality does not move
The lock file and the state file are related	Unrelated. Provider selections versus infrastructure

ON THE EXAM

These distractors are all true things about state, stretched slightly too far.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Plans take far too long on a large estate	The refresh step, and -refresh=false as a trade
A password appeared somewhere it should not	State stores attributes in plain text
Two engineers overwrote each other's work	Local state, and the need for remote state plus locking
A resource was destroyed in the wrong order	Dependency metadata, which state retains for deletion
The same object was imported twice	The one-to-one binding rule, now ambiguous
Why can Terraform not just query the cloud?	Tags are not universal, and deletion ordering would be lost

ON THE EXAM

Ask which of the four purposes the scenario is stressing. Mapping, metadata, performance or syncing.

A team removes an application server and the private subnet it sat in from their configuration in a single commit, deleting both resource blocks. The subnet cannot be deleted while the server still exists.

The apply succeeds. The server is destroyed first, then the subnet. A junior engineer asks how Terraform knew the ordering, given that neither resource block still exists to describe the relationship.

What made the correct ordering possible?

A. Terraform read the dependency from the previous commit in version control

B. Terraform retains the most recent set of resource dependencies in state

C. The provider rejected the subnet deletion and Terraform retried in a different order

D. Terraform destroys resources in reverse alphabetical order by resource address

◆ PAUSE AND DECIDE

What made the correct ordering possible?

A Read from version control

Terraform has no awareness of Git. It sees only the files in the working directory, and those no longer describe either resource.

B Dependencies retained in state

Correct, and this is precisely why state stores dependency metadata. Configuration cannot describe the ordering of something it no longer contains.

C Failed and retried

Terraform does not discover ordering by trial and error. A dependency violation surfaces as an error, not as an automatic reordering.

D Reverse alphabetical

Invents a rule, and a plausible-sounding one. Ordering always comes from the dependency graph, never from names.

THE SIGNAL

The graph normally comes from configuration. For deletion it cannot, which is exactly the gap dependency metadata fills.

A configuration manages about four thousand resources. A plan now takes over twenty minutes before printing anything, and engineers have started skipping plans during incidents.

A colleague suggests always running plan with -refresh=false, and says it makes plans nearly instant.

How should the team evaluate this suggestion?

A Adopt it as the default. Refresh is a performance feature and adds no correctness

B Understand it as a trade: faster plans, computed against cached attributes that may be stale

C Reject it. Terraform cannot produce a plan without refreshing first

D Adopt it, because state is refreshed during apply regardless of the plan flag

◆ PAUSE AND DECIDE

admin@sthithapragna.com

How should the team evaluate this suggestion?

A No correctness cost

Inverts what the cache is for. The cache is the performance feature; refreshing is what makes the plan reflect reality.

B A trade, understood

Correct. The flag is legitimate and the cost is real: any change made outside Terraform since the last refresh is invisible to that plan.

C Cannot plan without refresh

False. Terraform plans happily from cached attributes, which is exactly what the flag asks it to do.

D Apply refreshes anyway

Mixes two things. A saved plan is applied as written; the value of seeing an accurate plan is lost whatever apply does afterwards.

THE SIGNAL

Skipping plans during incidents is the planted detail. It says drift is likely, which is exactly when a stale plan is most dangerous.

admin@sthithapragna.com

A configuration creates a managed database with a generated administrator password. The input variable holding it is declared with sensitive = true, and plan output shows the value as sensitive rather than printing it.

During a security review, an auditor opens the state file in the remote backend and reads the password in plain text. The team is surprised, having believed the sensitive marking protected it.

What is the correct explanation?

A The sensitive marking failed because it was applied to a variable rather than an output

B sensitive affects displayed output only. State stores attribute values in plain text regardless

C The backend was misconfigured; correctly configured backends encrypt sensitive attributes individually

D The password was written to state because the plan was run with -refresh=false

◆ PAUSE AND DECIDE

What is the correct explanation?

A Wrong place for sensitive

Variables and outputs both accept sensitive, and it behaves the same way in both. The marking worked exactly as designed.

B Output only, never storage

Correct. sensitive suppresses display. Terraform stores whatever the provider returns, in plain text, which is why HashiCorp says to treat state as sensitive.

C Backend misconfiguration

No backend encrypts individual attributes. Encryption at rest protects the whole file, and anyone authorised to read it still sees plain text.

D Caused by -refresh=false

Unrelated. Refresh controls whether attributes are re-read; it has no bearing on whether they are stored.

THE SIGNAL

Nothing malfunctioned. The team's mental model was wrong, and the tool behaved exactly as documented.

admin@sthithapragna.com

> A newly hired architect proposes removing state files entirely. Her argument is that the cloud provider already knows what exists, so Terraform should query the provider on every run and compare the results with the configuration directly.
>
> She points out this would eliminate state locking, state corruption, and the risk of secrets sitting in a state file.

What is the strongest technical objection?

A Provider APIs are too slow to query, so this would be impractical at any scale

B Terraform could not identify which objects it manages, nor order deletions once configuration is removed

C The Terraform language has no way to express a comparison against live infrastructure

D Providers do not expose read operations, only create, update and delete

◆ PAUSE AND DECIDE

What is the strongest technical objection?

A APIs too slow

A real cost and the weakest objection, since it is only about speed. It is also the problem the attribute cache already exists to reduce.

B Identity and deletion ordering

Correct. It names both failures HashiCorp documents: tags are not universally supported, and deletion ordering has no source once configuration is gone.

C Language cannot express it

False. Data sources read live infrastructure routinely. Reading is not the difficulty; knowing which readings are yours is.

D No read operations

False, and refresh depends on reads. Every provider implements them.

THE SIGNAL

Her three complaints are all genuine. Every one of them is a cost of state, not an argument that state is unnecessary.

admin@sthithapragna.com

Module 4 recap

- **State stores bindings** between remote objects and resource instances. That is its primary purpose.
- **Four documented reasons:** mapping, metadata, performance, syncing.
- **Dependency metadata exists for deletion,** when configuration can no longer describe the ordering.
- **Refresh is the default and it is the correctness step.** The cache is the optimisation.
- **State holds secrets in plain text.** sensitive controls printing, never storage.
- **Editing state changes belief, not reality.** Only apply changes infrastructure.

> ON THE EXAM
>
> A plan compares three things, not two: configuration, state, and what the provider reports.

MODULE 05 · DOMAIN 3 - CORE TERRAFORM WORKFLOW

The Core Workflow: init, plan, apply, destroy

The four commands, the flags that change what they do, and the two that HashiCorp warns you about.

DOMAIN 3 of 8 COURSE WEIGHT 16% MODULE 5 of 27

admin@sthithapragna.com

Write, plan, apply. Everything else is a **variation** on those three steps.

admin@sthithapragna.com

Write, plan, apply

01

Write

Author configuration describing the infrastructure you want.

02

Init

Prepare the working directory. Backend, providers, modules.

03

Plan

Preview the changes. Nothing is altered.

04

Apply

Execute the plan against the provider APIs.

ON THE EXAM

Init runs once per working directory, and again whenever its inputs change. Plan and apply are the loop you live in.

What terraform init actually does

- **Initialises the backend**, so Terraform knows where state lives before anything else happens.
- **Downloads and installs providers** named in required_providers, writing the dependency lock file.
- **Downloads modules** referenced by module blocks, into the .terraform directory.
- **Safe to run repeatedly.** It will not destroy anything and will not change existing state.
- It does **not** create infrastructure, read your variables, or contact your cloud provider's resource APIs.

> ON THE EXAM
>
> Three responsibilities: backend, providers, modules. Almost every init question is about one of those three changing.

When you must run init again

- **You added a provider,** or changed a version constraint so the locked version no longer fits.
- **You added or changed a module block,** including its source or version.
- **You changed the backend configuration,** or added one where there was none.
- You cloned the repository, or deleted the **.terraform** directory. It is disposable, so recreate it.
- Other commands **detect this and tell you**. The error asking you to run init is not a bug.

> ON THE EXAM
>
> Backend, providers, modules. If one of those three changed, init needs running. Nothing else requires it.

The init flags worth knowing

FLAG	WHAT IT DOES
`-upgrade`	Re-resolve providers and modules against current constraints, ignoring the lock
`-reconfigure`	Reinitialise the backend, discarding existing configuration. No state migration
`-migrate-state`	Reinitialise the backend and offer to copy existing state across
`-backend-config=...`	Supply backend settings from a file or key-value pair at init time
`-input=false`	Never prompt. Essential in automation, where there is nobody to answer
`-get=false`	Skip module installation. Rarely useful outside constrained environments

ON THE EXAM

-reconfigure and -migrate-state both change backends. Only one offers to bring your state with it.

What terraform plan does, precisely

- **Reads the current state of existing remote objects,** so state is up to date. This is the refresh.
- **Compares the configuration against the prior state,** noting every difference.
- **Proposes change actions** that would make reality match the configuration.
- **Plan alone carries out nothing.** It is a preview, and running it is always safe.
- If nothing differs, Terraform reports that no actions need to be taken.

> ON THE EXAM
>
> Three operations in order: refresh, compare, propose. That is module four's three-input model as a command.

Speculative plan or saved plan?

SPECULATIVE

terraform plan, with no -out

- A description of the effect
- No intent to apply it
- Ideal before code review
- Cannot be handed to apply

SAVED

terraform plan -out=tfplan

- An opaque file on disk
- Passed to apply as an argument
- Applied exactly as written, no prompt
- Intended mainly for automation

THE TIEBREAKER

A saved plan applies without asking, because you already approved it when you generated it. That is the point and the risk.

A saved plan file is a sensitive artefact

- **The plan file contains your full configuration**, every planned value, and all the input variables.
- **Sensitive data is stored in cleartext,** even where Terraform obscured it in terminal output.
- HashiCorp's guidance: treat saved plan files as **potentially sensitive artefacts**.
- So do not publish them as build artefacts, and do not retain them longer than the apply needs.
- **Never name it with a .tf suffix.** Terraform would try to parse it as configuration and every later command breaks.

> ON THE EXAM
>
> The same lesson as state in module four. Obscured on screen has never meant absent from the file.

The three planning modes

MODE	ITS GOAL
`Normal`	Change the remote system to match your configuration
`-destroy`	Destroy every remote object, leaving an empty state
`-refresh-only`	Update state and outputs to match reality. Change no infrastructure

> ON THE EXAM
>
> Mutually exclusive. Choosing one disables normal mode, and you cannot combine two.

Options that change the plan

OPTION	EFFECT
`-refresh=false`	Skip the refresh. Faster, and external changes become invisible
`-replace=ADDRESS`	Plan to destroy and recreate one instance. Available from v0.15.2
`-target=ADDRESS`	Restrict planning to an address and everything it depends on
`-var 'name=value'`	Set one input variable. No spaces around the equals sign
`-var-file=FILE`	Set many from a .tfvars file. Preferred over -var for anything complex

ON THE EXAM

These work on both plan and apply. -replace cannot be combined with -destroy, and -refresh=false is invalid in refresh-only mode.

-target: read the warning

- It selects the address you name **plus everything it depends on**, directly or indirectly.
- HashiCorp: for **exceptional circumstances only**, such as recovering from mistakes or working around limitations.
- **Not recommended for routine operations.** It causes undetected drift and confusion about what state means.
- The documented alternative: **break a large configuration into several smaller ones** applied independently.
- Then share values between them with **data sources**, so the architecture stays connected.

> ON THE EXAM
>
> If a scenario uses -target as a routine habit, that habit is the wrong answer. It is a recovery tool.

-replace, and the command it superseded

- **-replace=ADDRESS** plans to destroy and recreate one instance, even though nothing in configuration changed.
- The use case: a remote object has become **degraded** and you want a fresh one built identically.
- It appears **in the plan**, so you see the replacement before approving it. Repeat the flag for several objects.
- **terraform taint** did this by editing state directly, with no preview. It is deprecated.
- -replace arrived in **v0.15.2**. Use taint only on versions older than that.

> ON THE EXAM
>
> The improvement is visibility. taint changed state and hoped; -replace shows you the replacement in a plan first.

terraform apply, in two forms

- **With no plan file:** apply generates a fresh plan, shows it, and prompts you to approve.
- **With a plan file:** apply executes that plan exactly as written, and does not prompt.
- **-auto-approve** skips the prompt in the first form. Standard in automation, dangerous at a keyboard.
- Apply accepts the planning modes and options too, so **apply -destroy** and **apply -refresh-only** are both valid.
- A saved plan can go **stale**. Other changes may have landed since it was generated.

> ON THE EXAM
>
> Apply never prompts for a saved plan. The approval is considered to have happened when the plan was created.

terraform destroy

- **Identical to apply -destroy.** It is a convenience alias for destroy planning mode.
- It destroys in **reverse dependency order**, using the dependency metadata state retains.
- Legitimate use: **transient environments** that stop being useful once the work is done.
- **prevent_destroy** on a lifecycle block makes Terraform refuse. Module eleven covers it.
- It shows a plan and prompts, exactly like apply. **-auto-approve** skips that, and rarely should.

> ON THE EXAM
>
> Destroy is not a separate mechanism. Knowing it is apply in a different mode explains why its flags are the same.

Flags for running in automation

FLAG	WHY A PIPELINE NEEDS IT
`-input=false`	There is no human to answer a prompt. Missing values become errors
`-auto-approve`	Skips confirmation when applying without a saved plan
`-detailed-exitcode`	0 no changes, 1 error, 2 changes present. Lets a pipeline branch on the result
`-lock-timeout=DURATION`	Wait for a held lock instead of failing immediately
`-no-color`	Strips terminal formatting for systems that cannot render it
`-parallelism=n`	Limits concurrent operations while walking the graph. Defaults to 10

ON THE EXAM

-detailed-exitcode is the one people forget. Without it, a plan with changes and a plan with none both exit zero.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
A new module block was added and plan fails	Run terraform init again
Speed up applies by working on one part at a time	-target misused. Split the configuration instead
A degraded instance must be rebuilt identically	-replace, not taint and not destroy
The pipeline must know whether changes exist	-detailed-exitcode
Approve now, apply after review	A saved plan with -out, and re-check before applying
Changed outside Terraform, record it only	Refresh-only mode, not normal mode

ON THE EXAM

Ask which of the three steps is under stress: preparing the directory, previewing the change, or executing it.

A configuration has grown to roughly two thousand resources covering networking, databases and twelve application services. Applies take a long time, so the team has adopted a habit of running apply with -target set to whichever service they are working on.

Over six months, three incidents have been traced to resources that were never updated because nobody targeted them. The team asks how to make targeting safer.

What should they be advised?

A Target the parent module rather than individual resources, so dependencies are included

B Stop routine targeting and split the configuration into smaller ones, sharing values via data sources

C Continue targeting, but run a full untargeted apply weekly to catch stragglers

D Raise -parallelism so full applies complete fast enough to remove the incentive

◆ PAUSE AND DECIDE

What should they be advised?

A Target the module instead

Targeting a module does widen the selection, and it is still targeting. The undetected drift the documentation warns about is unchanged.

B Split the configuration

Correct, and it is the alternative HashiCorp names explicitly: smaller independently applied configurations, connected by data sources.

C Weekly full apply

Reduces the window without closing it. It also means a weekly apply carrying six days of unreviewed change, which is a worse blast radius.

D Raise parallelism

Treats a design problem as a throughput problem. Provider rate limits usually bite first, and the configuration is still one unit of change.

THE SIGNAL

The question asks how to make targeting safer. The documented answer is to stop needing it.

A pipeline runs terraform plan -out=tfplan, then publishes tfplan as a build artefact so reviewers can download it. The artefact store is readable by everyone in the engineering organisation.

The configuration provisions a database whose administrator password is supplied through an input variable marked sensitive. Plan output in the build log correctly shows the value as sensitive.

What is the problem with this pipeline?

A There is no problem. The sensitive marking protects the value in the plan file

B The plan file stores the value in cleartext and is readable by the whole organisation

C Plan files cannot be published as artefacts because apply requires them in place

D The problem is the build log, since plan output always contains variable values

◆ PAUSE AND DECIDE

What is the problem with this pipeline?

A. No problem

Repeats the misconception from module four in a new location. sensitive governs display, and the plan file is storage, not display.

B. Cleartext, and readable

Correct. HashiCorp states that sensitive data is saved in cleartext in the plan file, even when obscured in terminal output, and calls plan files potentially sensitive artefacts.

C. Cannot be published

False. Moving plan files between steps is normal and is what the two-step workflow is for. Where it goes and who can read it is the issue.

D. The build log

The scenario says the log correctly showed the value as sensitive. The one component behaving properly is being blamed.

THE SIGNAL

The same pattern as state: obscured on screen, cleartext in the file. Two files now, one rule.

A change control process requires that plans are generated on Monday, reviewed by a board on Wednesday, and applied on Friday using the saved plan file so that what was approved is exactly what runs.

On Thursday, an unrelated team applies a change to the same configuration to resolve an incident. The Friday apply is run against Monday's saved plan file.

What should the team expect?

A Apply will prompt for confirmation because the plan is no longer current

B Apply may fail or produce an unintended result, because the saved plan reflects Monday's state

C Apply will silently regenerate the plan, since saved plans expire after 24 hours

D Apply will merge Monday's plan with Thursday's changes automatically

◆ PAUSE AND DECIDE

What should the team expect?

A **It will prompt**

Apply never prompts for a saved plan. The approval is treated as having happened when the plan was generated, which is the entire premise here.

B **May fail or misfire**

Correct. HashiCorp warns that changes made in the meantime can make the final effect differ from what an earlier plan indicated, and that you should re-check the final plan.

C **Plans expire**

Invents a lifetime. A plan file has no expiry. It is stale the moment anything changes, and it never announces that.

D **It merges**

Terraform has no merge concept for plans. A plan is a fixed set of actions computed against one moment.

THE SIGNAL

The process is well intentioned and its guarantee is the wrong one. It guarantees what runs, not that what runs is still correct.

A virtual machine managed by Terraform has become unresponsive after a failed in-guest upgrade. Its configuration is correct and unchanged, and a freshly built instance from the same configuration would resolve the problem.

Terraform plans no changes, because from the provider's point of view the instance still matches the configuration.

What should the engineer run?

A terraform apply -replace=ADDRESS for that instance

B terraform destroy followed by terraform apply

C terraform apply -refresh-only, so Terraform notices the instance is degraded

D Edit the configuration to force a difference, apply, then revert it

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should the engineer run?

A apply -replace

Correct. This is exactly the documented use case: a degraded remote object replaced by an identical one, and the replacement appears in the plan for approval.

B destroy then apply

Destroys everything in the configuration, not one instance. A vastly larger blast radius for a single sick machine.

C apply -refresh-only

Refresh-only records reality into state and changes no infrastructure. The provider reports the instance as healthy, so it would find nothing.

D Fake a change

Works, and leaves two misleading commits in history and a window where configuration does not describe intent. -replace exists to avoid exactly this.

THE SIGNAL

Terraform cannot see in-guest failure. -replace is how you tell it something is wrong that the provider API cannot report.

> An engineer adds a module block sourcing a networking module from the private registry, then runs terraform plan. Terraform stops with an error saying the module is not installed and that she should run terraform init.
>
> She is confident nothing else changed, and asks whether running init will affect the twelve resources this configuration already manages.

What is the correct answer?

A. Init may recreate resources, so a maintenance window should be booked first

B. Init only prepares the working directory. It creates and destroys nothing

C. Init will refresh state against the provider, which may show unexpected drift

D. Init should be avoided here; terraform get installs modules without other side effects

◆ PAUSE AND DECIDE

What is the correct answer?

A Book a window

Init touches no infrastructure at all. Caution is reasonable in general and is misdirected at the one command that cannot break anything.

B Prepares only

Correct. Init handles backend, providers and modules. It is safe to run repeatedly, and the error message is a prompt rather than a warning.

C Init refreshes state

Refresh belongs to plan and apply. Init initialises the backend so state can be reached; it does not read remote objects.

D Use terraform get

terraform get does install modules, and it does not handle providers or the backend. It solves a narrower problem for no benefit here.

THE SIGNAL

Init's three responsibilities are backend, providers and modules. None of them touch a running resource.

Module 5 recap

- **Init handles backend, providers and modules.** It creates nothing and is safe to repeat.
- **Plan refreshes, compares, proposes.** Alone it changes nothing at all.
- **A saved plan applies without a prompt,** and can go stale. Re-check before applying.
- **Plan files hold secrets in cleartext,** exactly like state. Never publish them freely.
- **-target is a recovery tool.** Routine use is the wrong answer; split the configuration instead.
- **-replace rebuilds a degraded object** and shows it in the plan. taint is deprecated.

> ON THE EXAM
>
> Three planning modes: normal, destroy, refresh-only. Mutually exclusive, and destroy is just apply in a different mode.

MODULE 06 · DOMAIN 3 - CORE TERRAFORM WORKFLOW

Reading a Plan. fmt, validate, show, output, console

The symbols, the summary, and the five commands that inspect rather than change.

DOMAIN 3 of 8 COURSE WEIGHT 16% MODULE 6 of 27

admin@sthithapragna.com

The plan is the artefact you approve. If you cannot read it precisely, **approval means nothing**.

admin@sthithapragna.com

The plan symbols

SYMBOL	ACTION
+	Create. This resource does not currently exist
-	Destroy. Terraform will remove this resource
~	Update in place. Changed without destroying and recreating
-/+	Replace. Destroyed, then recreated
<=	Read. A data source will be read during apply, not at plan time

ON THE EXAM

The gap between ~ and -/+ is the difference between a config change and an outage. Read that symbol before anything else.

Reading a single resource block

- A header line names the resource and the action, such as **will be updated in-place** or **must be replaced**.
- **forces replacement** appears beside the specific attribute causing it. That attribute is the whole story.
- **(known after apply)** means the value cannot be computed yet, because it comes from the provider.
- A destroy may be annotated **because it is not in configuration**, meaning you deleted the block.
- Changed values are shown as **old arrow new**, so you can see exactly what moved.

> ON THE EXAM
>
> When a plan surprises you, find the line marked 'forces replacement'. It names the single attribute responsible.

The summary, and what Terraform tells you unprompted

- Every plan ends with a count: **so many to add, to change, and to destroy**.
- If nothing differs, Terraform reports that **no actions need to be taken**. That is a successful plan.
- Terraform separately reports **changes made outside of Terraform** that it noticed during the refresh.
- That section is **drift**. It is the difference between state and reality, not between state and configuration.
- Reading the count first, then the drift note, then the blocks, is a habit worth forming.

> ON THE EXAM
>
> A plan with zero changes and a long drift section is not a quiet plan. Somebody has been editing your infrastructure.

terraform fmt

- **Rewrites configuration files** to the canonical format and style. A subset of the documented style conventions.
- **Intentionally opinionated, with no customisation options.** The goal is consistency between codebases, not your favourite style.
- **By default it only processes the current directory.** Subdirectories need -recursive.
- Terraform's own generated configuration conforms to it, so following it keeps everything consistent.
- Works well as a **pre-commit hook**, which is where HashiCorp suggests putting it.

> ON THE EXAM
>
> It is a formatter, not a linter. It never tells you your configuration is wrong, only that it is untidy.

fmt options

OPTION	EFFECT
`-check`	Do not write. Exit non-zero if any file needs formatting
`-diff`	Show what would change, as a diff
`-recursive`	Process subdirectories too. Off by default
`-list=false`	Suppress the list of filenames that were or would be changed
`-write=false`	Do not overwrite files. Implied by -check

ON THE EXAM

-check plus -recursive is the pipeline combination. It fails the build rather than quietly rewriting the repository.

The canonical format is allowed to change

- **The canonical format may change in minor ways between Terraform versions.**
- HashiCorp does **not** consider new formatting rules to be a breaking change.
- So after upgrading Terraform, **run fmt across your modules** as part of adopting the new version.
- New rules are usually expansions of style already documented, so surprises are small.
- This is why a fmt check can start failing on a repository nobody has touched.

> ON THE EXAM
>
> A formatting check that fails after a CLI upgrade is expected behaviour, not a broken pipeline.

terraform validate

- Checks that a configuration is **syntactically valid and internally consistent**.
- That includes **attribute names and value types** for resources and modules, and references that do not resolve.
- **It does not access remote services.** No provider APIs, no remote state, no credentials.
- It works **regardless of any provided variables or existing state**, so no values are needed.
- Primarily useful for **reusable modules**, and safe to run automatically on save or in CI.

> ON THE EXAM
>
> Validate answers 'is this configuration coherent?'. It never answers 'will this work against my cloud account?'.

validate or plan?

VALIDATE ANSWERS

Is this configuration coherent?

- Structure, types, names, references
- No credentials needed
- No state needed, no variables needed
- Fast, offline, safe on every save

PLAN ANSWERS

What would happen if I applied this?

- Real state, real provider APIs
- Credentials required
- Variable values required
- Slower, and the only real answer

THE TIEBREAKER

A green validate means the code is shaped correctly. It says nothing about whether the change is safe or will succeed.

Two details about validate

- **Validate requires an initialised working directory**, with providers and modules installed. It needs their schemas.
- For CI, initialise without touching the backend: **terraform init -backend=false**, then validate.
- **plan and apply validate automatically** before doing any other work.
- So validate is not a required step in the core workflow. It is a fast, cheap check placed earlier.
- **-json** gives machine-readable output for a pipeline to parse.

> ON THE EXAM
>
> Validate needs init but not a backend. That combination is exactly what a module's CI pipeline wants.

terraform show

- With no arguments, prints the **current state** in human-readable form.
- Given a **saved plan file**, prints that plan in the same readable form.
- That second form is how a reviewer inspects a plan generated elsewhere, such as by a pipeline.
- **-json** produces machine-readable output, and is the supported way to build tooling.
- Reading the raw state file yourself is **not** supported. The format is internal and may change.

> ON THE EXAM
>
> show is the supported interface to state and to plan files. It is the answer whenever a question involves inspecting either.

terraform output

- Reads **root module output values** from state. Not from configuration, and not from the provider.
- With no name it prints all of them. With a name it prints one.
- By default a string is printed **with quotes**, which breaks shell scripts that use the value directly.
- **-raw** prints the bare value with no quotes and no formatting. Use it when piping into something else.
- **Outputs marked sensitive are redacted** in normal output. -raw and -json will still reveal them.

> ON THE EXAM
>
> Outputs come from state, so a value cannot appear until it has been applied. -raw exists for scripts; -json for tooling.

console, graph, version

- **terraform console** is an interactive prompt for evaluating expressions against your configuration and state.
- It is the fastest way to answer 'what does this function actually return'. Module nine leans on it.
- **terraform graph** emits the dependency graph in DOT format, for rendering with a graph tool.
- **terraform version** prints the CLI version **and the versions of installed providers**.
- It also tells you when a newer Terraform release is available.

> ON THE EXAM
>
> console reads state, so it can evaluate expressions against real values rather than hypothetical ones.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Check code without credentials or state	terraform validate, after init -backend=false
Fail the build if code is untidy	terraform fmt -check -recursive
A reviewer must inspect a plan built by CI	terraform show against the saved plan file
A script needs an output value with no quotes	terraform output -raw
Why is this resource being recreated?	The attribute annotated 'forces replacement'
Somebody changed things in the console	The 'changes outside of Terraform' section

ON THE EXAM

None of these commands change infrastructure. If a scenario needs something changed, none of them is the answer.

A team's pipeline runs terraform validate on every pull request and treats a pass as confirmation that the change is safe to merge. Plans are only generated after merge, during deployment.

A change passes validate, merges, and then fails at deployment because the configuration references a subnet that does not exist in the target account.

Why did validate not catch this?

A Validate was run before terraform init, so it had no provider schemas

B Validate checks configuration only and never contacts provider APIs or remote state

C Validate skips data sources, which is where the subnet lookup happens

D Validate requires variable values, and none were supplied in the pipeline

◆ PAUSE AND DECIDE

Why did validate not catch this?

A Run before init

Validate does require an initialised directory, and it would have failed loudly rather than passing. The scenario says it passed.

B Never contacts remote services

Correct. Validate verifies structure, types and references using local schemas. Whether a subnet exists in an account is a question only plan can ask.

C Skips data sources

Invents an exemption. Validate checks data blocks like any other, for attribute names and types. It just cannot execute the lookup.

D Needs variable values

Backwards. HashiCorp states validate works regardless of any provided variables or existing state, which is why it suits reusable modules.

THE SIGNAL

Nothing failed. The pipeline asked a structural question and treated the answer as a contextual one.

A repository has passed its formatting check for two years. The pipeline runs terraform fmt -check -recursive. Nobody has changed the Terraform code this week.

After the pipeline image was rebuilt and picked up a newer Terraform CLI, the formatting check now fails on forty files across the repository.

What is the most likely explanation?

A The newer CLI applies additional formatting rules. This is expected and not a breaking change

B The -recursive flag behaves differently in newer versions and is now scanning vendor directories

C The repository was reformatted by a third-party tool during the image rebuild

D fmt -check is non-deterministic and should be replaced with fmt -diff

◆ PAUSE AND DECIDE

What is the most likely explanation?

A New rules in a new CLI

Correct. HashiCorp states the canonical format may change in minor ways between versions and that new rules are not a breaking change.

B -recursive changed

Plausible-sounding and unsupported. Recursive has meant subdirectories since it was introduced, and the count would be far larger.

C A third-party tool

The image rebuild changed the tool, not the repository. Nothing in the pipeline writes back to source.

D fmt is non-deterministic

It is entirely deterministic for a given version, which is the whole point of a canonical format. -diff shows more detail; it changes no outcome.

THE SIGNAL

The planted detail is that nobody changed the code. When inputs are constant and output changes, the tool changed.

> A deployment script captures a load balancer hostname from Terraform and passes it to a health check command. The script uses terraform output lb_hostname and stores the result in a shell variable.
>
> Every health check fails. The logs show the tool attempting to reach a hostname that includes double quotation marks at both ends.

What should the script use instead?

A terraform output -raw lb_hostname

B terraform show lb_hostname

C terraform console, piping the output name into it

D terraform output -json lb_hostname, then use the value directly

◆ PAUSE AND DECIDE

What should the script use instead?

A output -raw

Correct. -raw prints the bare value with no quoting or formatting, which is exactly what a shell variable needs.

B show with a name

show prints whole state or a whole plan file. It does not take an output name and cannot select a single value.

C console piping

Console can evaluate the expression, and it is an interactive tool wrapped in a script. -raw exists so you do not have to.

D output -json

Closer, and it returns a JSON document that still needs parsing. It would leave the quotes unless something extracted the value.

THE SIGNAL

Default output is formatted for a human reading a terminal. -raw is formatted for another program.

> An engineer changes one tag on a managed database and expects an in-place update. The plan instead shows the database with -/+ and the header 'must be replaced'. Elsewhere in the block, one line reads 'forces replacement'.
>
> She is under time pressure and the change is considered routine.

What should she do?

A. Apply. The plan symbol indicates an update with a brief interruption

B. Read the attribute marked 'forces replacement', because the database will be destroyed and recreated

C. Re-run plan with -refresh=false, since stale state is the usual cause of a replacement

D. Apply with -replace on the database, which performs the same change more safely

◆ PAUSE AND DECIDE

What should she do?

A Apply, it is an update

Misreads the symbol. -/+ is destroy then create, not update. For a database that means data loss unless something else protects it.

B Read the forces replacement line

Correct. That annotation names the single attribute responsible, which is both the explanation and, usually, the thing to change.

C Plan with -refresh=false

Skips reading reality, so it would hide information rather than add any. Replacement comes from a configuration difference, not from stale data.

D Apply with -replace

Asks for the destruction explicitly. It is the correct tool for a degraded resource and here it would guarantee the outcome she wants to avoid.

THE SIGNAL

Time pressure plus a routine-looking change is when replacement symbols get skimmed. That combination is the actual hazard.

Module 6 recap

- **Plus create, minus destroy, tilde update in place, minus-slash-plus replace.** The last one is the one to slow down for.
- **forces replacement** names the single attribute responsible. Find it before you argue with the plan.
- **fmt is a formatter, not a linter,** and it is not recursive by default.
- **The canonical format may change between versions.** A check failing after an upgrade is expected.
- **validate never contacts remote services.** Coherent is not the same as correct or permitted.
- **show inspects state and plan files. output -raw is for scripts.** Both read from state.

> ON THE EXAM
>
> Everything in this module is read-only. If a scenario needs infrastructure changed, none of these commands is the answer.

MODULE 07 · DOMAIN 4 - TERRAFORM CONFIGURATION

Resources, Data Sources, References, Addressing

The two block types that touch infrastructure, and the syntax that wires them together.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 7 of 27

admin@sthithapragna.com

A **resource** block says Terraform owns this.
A **data** block says somebody else does,
and I need to look at it.

admin@sthithapragna.com

The resource block

- **resource**, then two labels: the **resource type**, then a **name** you choose.
- The type comes from a provider. Its first word is how Terraform picks the plugin, as in module two.
- The name is **local to the module** and has no meaning to the provider. It appears nowhere in your cloud account.
- Together they form the **resource address**: the type, a dot, then the name.
- Two resources in one module cannot share a type and name. That pair must be unique.

> ON THE EXAM
>
> The name is for you and for the dependency graph. Providers never see it, so renaming a resource means Terraform thinks you deleted one and made another.

Arguments or attributes?

ARGUMENTS

What you write in

- Values you supply in the block
- Required or optional, per the provider
- Known before anything is created
- Changing one may update or replace

ATTRIBUTES

What comes back out

- Values the provider reports
- Include everything you supplied, plus more
- Computed ones are unknown until apply
- Referenced by other resources

THE TIEBREAKER

(known after apply) in a plan means a computed attribute. The provider decides it, so Terraform genuinely cannot say in advance.

The four things Terraform can do to a resource

ACTION	WHEN TERRAFORM CHOOSES IT
Create	It is in configuration and has no binding in state
Update in place	An argument changed, and the provider can change it live
Replace	An argument changed that the provider cannot change live
Destroy	It has a binding in state and is no longer in configuration
No action	Configuration, state and reality all agree

ON THE EXAM

The provider decides which attributes force replacement, not Terraform. That is why the same change is trivial on one resource type and destructive on another.

The data block

- **data**, then the data source type, then a name you choose. Same shape as a resource.
- **The address is different.** It carries a data prefix: data, the type, then the name.
- Forgetting that prefix is one of the most common syntax errors in the language.
- Data sources are provided by the same providers, and are declared in the registry alongside resources.
- **A data source creates nothing and destroys nothing.** It appears in state only as a cached read.

> ON THE EXAM
>
> Resource address: type dot name. Data source address: data dot type dot name. That extra segment is examinable.

resource or data?

USE A RESOURCE WHEN

Terraform owns the object

- You want it created and managed
- Its lifecycle is yours to control
- Destroying the config destroys the object
- It appears in state as a binding

USE A DATA WHEN

Something else owns it

- Created by another team or another config
- Created outside Terraform entirely
- You need its attributes, not control of it
- Destroying the config leaves it untouched

THE TIEBREAKER

Ask what should happen on destroy. If the answer is 'that object must survive', it is a data source.

When data sources are read

- **Terraform tries to read data sources during the plan phase**, so their values are available in the plan.
- It **defers the read until apply** when an argument refers to something it cannot predict.
- That happens when the data block depends on a managed resource **configured to change in the current plan**.
- It also happens when a **custom condition** on the block depends on such a resource. Module twelve covers those.
- Terraform **says so in the plan**: 'will be read during apply', with the reason in brackets.

> ON THE EXAM
>
> A deferred read is contagious. Anything referencing that data source becomes unknown at plan time too.

depends_on on a data source, and why to avoid it

- Adding **depends_on** to a data block forces Terraform to wait for the named object before reading.
- If that dependency has **changes pending**, the read is pushed to apply and its attributes show as known after apply.
- **Every resource consuming that data source becomes partially unknown** in the plan too.
- So the plan becomes far less informative, which defeats the reason plans exist.
- **Prefer an implicit dependency** through an attribute reference. Reserve depends_on for what references cannot express.

> ON THE EXAM
>
> depends_on guarantees ordering. It never guarantees the data source will find what it is looking for.

References create dependencies

- Writing one resource's attribute inside another creates an **implicit dependency**.
- Terraform builds the graph from those references, and derives creation order from the graph.
- **This is the mechanism**. It is why module one said Terraform never needs you to write the order of operations.
- Hardcode an identifier as a literal string and **no dependency exists**. Terraform may build them in any order.
- Destroy runs the graph in reverse, using the dependency metadata state retains, as in module four.

> ON THE EXAM
>
> A reference is not just a way to read a value. It is how you tell Terraform that one thing must exist before another.

The reference syntax

TO REFER TO	YOU WRITE
A resource attribute	aws_instance.web.id
A data source attribute	data.aws_ami.ubuntu.id
An input variable	var.region
A local value	local.common_tags
A child module output	module.network.vpc_id
The current workspace name	terraform.workspace

ON THE EXAM

Six shapes cover almost everything. Only data sources and module outputs need a prefix segment.

Addressing an instance, and a module

- With **count**, instances are addressed by number in square brackets, starting at zero.
- With **for_each**, instances are addressed by their key in square brackets, as a string.
- A **resource address** names the resource as a whole. A **resource instance address** names one instance.
- Inside a module, prefix with **module** and the module name, and repeat for nesting.
- The same syntax is used by **-target**, by the state commands, and by moved and import blocks.

> ON THE EXAM
>
> Learn this syntax once. It reappears in -target, terraform state mv, and the moved, removed and import blocks.

Values that are not resources

- **path.module** is the filesystem path of the module where the expression is written.
- **path.root** is the path of the root module. **path.cwd** is the current working directory.
- Use path.module when a module reads a file it ships with, so it works wherever the module is used.
- **terraform.workspace** gives the current workspace name, which lets configuration vary by environment.
- These are **named values**, not resources. They have no attributes and create no dependencies.

> ON THE EXAM
>
> path.module and path.root differ the moment your code lives in a child module. That difference is the whole point.

Cycles, and what they mean

- If A references B and B references A, Terraform reports a **cycle** and refuses to plan.
- The graph must be acyclic. There is no ordering that satisfies a loop, so Terraform cannot proceed.
- Usually the fix is to notice that **one direction is not really needed**, and remove that reference.
- Sometimes the answer is a **separate association resource** that references both, rather than each other.
- **depends_on can create cycles too**, and is a common cause when it is added carelessly.

> ON THE EXAM
>
> A cycle error is a modelling problem, not a syntax problem. Terraform is telling you the relationship you described is impossible.

Getting a value from somewhere else

A DATA SOURCE WHEN

The platform can tell you

- Query the provider by name or tag
- Always current at plan time
- No coupling to another state file
- Works for anything Terraform did not create

TERRAFORM_REMOTE_STATE WHEN

Another configuration knows

- Reads another configuration's outputs
- Only exposes declared outputs
- Requires access to that backend
- Couples the two configurations together

THE TIEBREAKER

Prefer a data source. Remote state couples two configurations and exposes whatever that state contains to whoever can read it.

A scope note on provisioners

- **Provisioners** run scripts on a local or remote machine as part of create or destroy.
- HashiCorp documents them under the heading **provisioners are a last resort**, and means it.
- They are **invisible to the plan**. Terraform cannot predict or show what a script will do.
- Better answers: **cloud-init or user data** for bootstrapping, and a configuration management tool afterwards.
- **terraform_data** is the built-in replacement for the old null_resource pattern.

> ON THE EXAM
>
> Provisioners are not named in the 004 objectives. Know what they are, know they are discouraged, and do not expect them to be the answer.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
The object must survive when we destroy this config	A data source, not a resource
Resources were created in the wrong order	A hardcoded value where a reference belongs
Plan is full of 'known after apply' unexpectedly	A deferred data source read
Another team owns the network we deploy into	A data source first, remote state second
A module reads a file it ships with	path.module, never path.root
Terraform reports a cycle	Two resources referencing each other. Remove one direction

ON THE EXAM

Ownership decides resource against data. References decide ordering. Those two questions settle most of this module.

A team deploys application servers into a subnet created by the networking team in a separate configuration. To avoid coupling, an engineer looks up the subnet identifier in the console and writes it into the configuration as a literal string.

The deployment works for months. Then a new region is added, and in that region Terraform occasionally attempts to create a security group rule before the security group it belongs to exists, and the apply fails intermittently.

What is the underlying cause?

A Terraform's parallelism is too high and should be reduced with -parallelism=1

B Hardcoded identifiers create no references, so no dependency exists and ordering is undefined

C Security group rules must always be declared before security groups in the file

D The subnet data is stale and the configuration should be re-initialised

◆ PAUSE AND DECIDE

What is the underlying cause?

A **Reduce parallelism**

Would make the failure rarer by serialising work, and the order would still be arbitrary. It hides an undefined ordering rather than defining it.

B **No reference, no dependency**

Correct. Terraform derives ordering from references. A literal string carries the right value and declares no relationship at all.

C **Declaration order matters**

Terraform is declarative. Position in the file has no effect on ordering; only the dependency graph does.

D **Stale data, re-init**

Init handles backend, providers and modules. It has nothing to do with resource ordering, and no data source is involved here.

THE SIGNAL

It worked for months because Terraform happened to choose a workable order. Nothing changed except the luck.

admin@sthithapragna.com

> A configuration manages an application stack. It must deploy into a shared VPC that predates Terraform and is managed by a central platform team, who tag it with a known name.
>
> A junior engineer writes a resource block for the VPC so that the configuration is complete and self-documenting, then runs plan.

What will happen, and what should be done?

A Terraform will adopt the existing VPC automatically, since the tag matches

B Terraform will plan to create a second VPC. Use a data source to look up the existing one

C Terraform will error, because a resource cannot be declared for an object that already exists

D Terraform will plan correctly, but the VPC will be destroyed on the next apply

◆ PAUSE AND DECIDE

What will happen, and what should be done?

A Adopts it automatically

Terraform never adopts existing objects by matching attributes. Module four explained why tags were rejected as an identity mechanism.

B Creates a second one

Correct. Nothing is in state for that resource, so Terraform plans a create. The platform team owns the VPC, so it is a data source.

C Terraform errors

Terraform has no idea the object exists. It compares configuration with state, and state is empty here, so a create is the honest answer.

D Destroyed next apply

Mixes up the failure. The danger with a wrongly declared resource is destroying somebody else's object later, and that requires importing it first.

THE SIGNAL

Apply the destroy test. If this configuration were destroyed, should the shared VPC go with it? Obviously not. So it is a data source.

A configuration creates a cluster, then uses a data source to read the cluster's generated endpoint, which several downstream resources consume. An engineer adds depends_on to the data block, reasoning that the cluster must exist before it can be read.

The next plan shows the data source as 'will be read during apply', and every downstream resource now shows its arguments as known after apply. Reviewers say the plan has become useless.

What should the engineer do?

A. Keep depends_on and accept it, since ordering correctness matters more than plan readability

B. Remove depends_on and reference a cluster attribute in the data block, creating an implicit dependency

C. Add depends_on to the downstream resources as well, so ordering is explicit throughout

D. Run plan with -refresh=false so the data source is read from cache instead

◆ PAUSE AND DECIDE

What should the engineer do?

A Accept the trade

Presents a false choice. An implicit dependency gives the same ordering guarantee without deferring the read, so nothing has to be traded.

B Reference an attribute instead

Correct. A reference establishes ordering and keeps the value predictable, which is HashiCorp's stated preference over depends_on.

C depends_on everywhere

Spreads the cause rather than removing it. More explicit dependencies on changing resources means more of the plan becoming unknown.

D Plan with -refresh=false

Refresh concerns re-reading existing state. It does not make a deferred data source resolve, and it would hide external changes as well.

THE SIGNAL

depends_on with a changing dependency defers the read. Everything downstream inherits that unknown, which is why the plan emptied out.

A shared module ships a startup script alongside its Terraform files and reads it when configuring instances. The module author tests it from the module's own directory and everything works.

A consuming team adds the module to their root configuration and the apply fails, reporting that the script file cannot be found.

What is the most likely cause?

A The module reads the file using path.root, which resolves to the caller's directory

B Shared modules cannot include files other than .tf files

C The consuming team must run terraform get before the file becomes available

D The file must be passed in as an input variable; modules cannot read files at all

◆ PAUSE AND DECIDE

What is the most likely cause?

A path.root instead of path.module

Correct. In the author's own directory the two are identical, so the bug is invisible until somebody calls the module from elsewhere.

B Only .tf files allowed

Modules routinely ship scripts, templates and policy documents. Nothing restricts a module directory to Terraform files.

C Run terraform get

terraform get installs modules, and init already does it. The module was clearly installed, since Terraform reached the point of reading the file.

D Pass the file in

Modules can read files perfectly well. Passing the content in would work and would push a detail onto every consumer for no reason.

THE SIGNAL

Testing a module from its own directory cannot catch this class of bug, because that is the one place both paths agree.

Module 7 recap

- **Resource means Terraform owns it. Data means somebody else does.**
- **Data source addresses carry a data prefix.** Three segments, not two.
- **References create dependencies.** A hardcoded identifier gives no ordering.
- **Data reads defer to apply** when they depend on something changing.
- **depends_on on a data block is usually wrong.** Prefer an attribute reference.
- **path.module for anything a module ships.** path.root breaks elsewhere.

> ON THE EXAM
>
> Arguments go in, attributes come out, and computed attributes are why plans say 'known after apply'.

MODULE 08 · DOMAIN 4 - TERRAFORM CONFIGURATION

Variables, Outputs, Locals, and Precedence

Three ways to name a value, and the five-level order that decides which one wins.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 8 of 27

admin@sthithapragna.com

Variables are a module's **inputs**. Outputs are its **return values**. Locals are its **working notes**.

The variable block

- **variable**, then a name. The name is how you reference it: var, a dot, then the name.
- **default** makes it optional. With no default, the variable is **required**.
- **type** constrains what may be assigned. **description** documents it for whoever uses the module.
- **sensitive** suppresses the value in Terraform's output. **nullable = false** forbids null being passed in.
- **validation** blocks add custom rules. Module twelve covers those in full.

> ON THE EXAM
>
> Declaring a variable and assigning it are separate acts. The block declares; the value arrives from one of five places.

Type constraints, briefly

- **Primitives:** string, number, bool. These cover most variables you will ever write.
- **Collections:** list, set and map, each holding one element type.
- **Structural:** object and tuple, where each attribute or position has its own type. Module nine.
- **any** accepts anything, and is a decision to do no checking rather than a type.
- Omit **type** entirely and Terraform accepts any value, which is the same as writing any.

> ON THE EXAM
>
> Terraform converts between primitives where it safely can, so the string "5" satisfies a number constraint.

Required variables, and automation

- If a required variable has no value, **Terraform prompts you for it interactively**.
- In a pipeline there is nobody to answer, so the run **hangs** until it times out.
- **-input=false** turns the prompt into an error, which fails fast and tells you which variable is missing.
- That is why module five listed it as an automation flag. This is the failure it prevents.
- A **default** is not always the fix. Some values genuinely should have no safe default.

> ON THE EXAM
>
> A hanging pipeline and a missing variable look nothing alike until you know Terraform prompts. -input=false makes them look the same.

Variable precedence, lowest to highest

SOURCE	NOTES
The default in the variable block	The lowest of all. A fallback, nothing more
Environment variables, TF_VAR_name	Higher than a default, and lower than every file
terraform.tfvars	Auto-loaded if present
terraform.tfvars.json	Auto-loaded, and beats the non-JSON file
*.auto.tfvars and .auto.tfvars.json	Auto-loaded, in lexical order of filename
-var and -var-file on the command line	Highest. In the order provided, so the last wins

ON THE EXAM

Later sources override earlier ones. HCP Terraform workspace variables sit at the top level, alongside the command line.

Three traps in that order

- **TF_VAR_ does not override a tfvars file.** It is second weakest. People assume the opposite constantly.
- **auto.tfvars files load in lexical order**, so a file starting with z overrides one starting with a.
- **A leftover .auto.tfvars from testing silently overrides your real values**, and nothing warns you.
- On the command line, **the last option wins**, whether it is -var or -var-file.
- So **-var-file=base.tfvars -var-file=prod.tfvars** layers production over base, which is the intended pattern.

> ON THE EXAM
>
> Auto-loaded means loaded whether you wanted it or not. That is convenient until a stray file is sitting in the directory.

Working an example

REGION IS SET IN	VALUE USED
default only	The default. Nothing else is in play
default, and TF_VAR_region	The environment variable. It beats the default
TF_VAR_region, and terraform.tfvars	terraform.tfvars. Files beat environment variables
terraform.tfvars, and b.auto.tfvars	b.auto.tfvars. Auto-loaded files rank higher
a.auto.tfvars, and b.auto.tfvars	b.auto.tfvars. Lexical order breaks the tie
b.auto.tfvars, and -var on the CLI	The command line. It beats every file

ON THE EXAM

Work upwards through the order and stop at the highest source present. There is never any merging or averaging.

Which files load themselves

- **terraform.tfvars** and **terraform.tfvars.json** are loaded automatically if present.
- Any file ending **.auto.tfvars** or **.auto.tfvars.json** is loaded automatically.
- **Any other name is not.** A file called prod.tfvars is ignored unless you pass -var-file.
- That is deliberate, and it is how per-environment variable files are meant to work.
- **tfvars files are ordinary text.** Committing secrets to one is module one's mistake in a new file.

> ON THE EXAM
>
> Naming a file prod.tfvars and expecting it to load is a common error. Only terraform.tfvars and *.auto.tfvars load themselves.

Where the precedence order does not apply

- **That order is for root module variables only.** Child modules do not participate in it.
- A child module's variables are set **solely by the arguments in its module block**.
- **-var and -var-file cannot reach a child module.** Neither can TF_VAR_ environment variables.
- To make a child module's input configurable, the root module declares its own variable and passes it through.
- That pass-through is the normal pattern, and it is why root modules often look like a list of variables.

> ON THE EXAM
>
> A module block argument is not 'like' setting a variable. It is the only way a child module variable ever gets a value.

The output block

- **output**, then a name, then a **value** expression. That is the minimum.
- In a **child module**, outputs are the only way a caller can read anything from inside.
- In the **root module**, outputs are printed after apply and read with terraform output.
- **sensitive** redacts the value in output. **description** documents it. **depends_on** is available but rarely needed.
- Outputs are stored in **state**, which is why terraform output works without re-planning.

> ON THE EXAM
>
> A module's outputs are its API. Anything not declared as an output is unreachable from outside, by design.

Local values

- Declared in a **locals** block, note the plural, and referenced as local, a dot, then the name.
- **Nobody can set a local from outside.** It is computed inside the module, from variables, resources or literals.
- Use one when the same expression appears several times, so a change happens in one place.
- Also use one to give a complicated expression a **readable name**, which is often the bigger win.
- **Overuse hides things.** A local defined far from where it is used is harder to follow, not easier.

> ON THE EXAM
>
> The block is 'locals' plural; the reference is 'local' singular. That inconsistency catches people writing configuration from memory.

Variable or local?

A VARIABLE WHEN

Somebody else should decide

- The value differs per environment or caller
- It belongs in the module's interface
- It should be documented and typed
- Settable from five different places

A LOCAL WHEN

The module decides

- Computed from other values
- The same in every use of the module
- An implementation detail, not an interface
- Settable from nowhere at all

THE TIEBREAKER

A variable with a default that nobody ever overrides is usually a local wearing the wrong block type.

The three, side by side

	SET FROM OUTSIDE? READ FROM OUTSIDE?
variable	Set from outside: yes. Read from outside: no
output	Set from outside: no. Read from outside: yes
local	Set from outside: no. Read from outside: no

ON THE EXAM

Variables flow in, outputs flow out, locals stay put. That single table answers a surprising number of questions.

sensitive, one more time

- **sensitive = true** is available on both variables and outputs. It redacts the value in Terraform's output.
- Terraform also **tracks it through expressions**, so a value derived from a sensitive one is redacted too.
- **It does nothing about state,** where the value sits in plain text. Module four.
- **It does nothing about plan files** either, where it is stored in cleartext. Module five.
- **terraform output -raw** and **-json** reveal sensitive outputs deliberately, so scripts can use them.

> ON THE EXAM
>
> Third module in a row making the same point. sensitive controls display. Storage is a separate problem, solved in module thirteen.

How files are usually organised

- Terraform reads **every .tf file in the directory** and treats them as one configuration. Filenames carry no meaning.
- Convention, not requirement: **variables.tf**, **outputs.tf**, **main.tf**, **versions.tf**.
- **Subdirectories are not read.** A directory is a module, and its children are separate modules.
- So splitting files changes nothing functionally. It is entirely for humans.
- The one file that is special is **terraform.tfvars**, and the .auto.tfvars pattern.

> ON THE EXAM
>
> Since filenames carry no meaning, a question about which file a block 'must' go in is almost always testing that fact.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
An exported TF_VAR_ value is being ignored	A tfvars file is overriding it. Env vars are nearly lowest
A value changed and nobody edited anything	A stray .auto.tfvars, loaded automatically
We need a value from inside a child module	The module must declare an output. There is no other route
The pipeline hangs with no error	A required variable, and a missing -input=false
Every caller passes the same value	It should probably be a local, not a variable
Two -var-file options were passed	The last one wins, which is the layering pattern

ON THE EXAM

Precedence questions are mechanical. Recall the order, find the highest source in play, and that value wins.

A pipeline sets TF_VAR_environment to staging before running terraform apply against the staging account. The repository contains a terraform.tfvars file, committed a year ago, which sets environment = "production".

Engineers have exported the variable correctly, confirmed it with echo, and the deployment still behaves as production.

Why is the environment variable being ignored?

A TF_VAR_ variables must be exported in the same shell that runs Terraform, which the pipeline does not do

B terraform.tfvars has higher precedence than environment variables and overrides it

C The variable name must match exactly including case, and TF_VAR_ names are case-sensitive

D Environment variables are only read when no variable block declares a default

◆ PAUSE AND DECIDE

Why is the environment variable being ignored?

A Wrong shell

A real class of bug, and ruled out by the scenario, which says the export was confirmed. Read that sentence as the question closing a door.

B tfvars outranks env vars

Correct. Environment variables sit second from the bottom, above only the default. Every tfvars file beats them.

C Case sensitivity

Names are case-sensitive, and a mismatch would mean the variable was simply unset, producing a prompt or an error rather than a wrong value.

D Only read without a default

Invents a rule. Environment variables are always consulted; they just lose to anything above them in the order.

THE SIGNAL

The instinct that environment variables override files comes from other tools. In Terraform they are nearly the weakest source.

> A root configuration calls a networking module that creates a VPC, subnets and a NAT gateway. A new requirement needs the NAT gateway's public IP address, to add it to a partner's allow list.
>
> An engineer writes a reference to the NAT gateway resource inside the module, using module dot network followed by the resource address. Terraform reports that the reference is invalid.

What is required?

A Use terraform_remote_state to read the module's internal resources

B Add an output to the networking module exposing the IP, then reference module.network and that output name

C Move the NAT gateway into the root module, since module internals cannot be referenced

D Reference it through the state file path instead, using terraform state show

◆ PAUSE AND DECIDE

What is required?

A. Use remote state

Remote state reads another configuration's root outputs. This module is in the same configuration, and outputs are still the only interface.

B. Declare an output

Correct. A module's outputs are its entire public interface, so exposing a value means the module author declares it.

C. Move it to root

Would work and dismantles the module to read one value. Modules exist precisely so internals can stay internal.

D. Read it via state show

An inspection command, not an expression. It prints for a human and cannot supply a value to configuration.

THE SIGNAL

Encapsulation is not a side effect here. A module exposes exactly what its author declared, and nothing else is reachable.

> A module declares a variable called name_prefix with a default built from two other variables, joined with a hyphen. It is documented, typed, and has never been overridden by any of the eleven teams using the module.
>
> A reviewer suggests the declaration is the wrong shape.

What should it be instead, and why?

A. Leave it. A variable with a default is the correct way to provide a computed convenience value

B. A local value, since it is computed inside the module and nobody should be setting it

C. An output, so consuming configurations can see the prefix that was used

D. A variable with nullable = false, which prevents callers from overriding it

◆ PAUSE AND DECIDE

What should it be instead, and why?

A Leave it as a variable

A variable default cannot reference other variables at all, so the described declaration would not even be valid. It is also an interface nobody asked for.

B Make it a local

Correct. It is computed inside the module from other values, and no caller should decide it. That is exactly what a local is for.

C Make it an output

An output would let callers read it, which nobody has asked for, and would not stop it being settable. It answers a different question.

D nullable = false

Rejects null being passed. It does not stop a caller passing some other value, so the unwanted knob is still there.

THE SIGNAL

Every variable is a promise to support that input. Eleven teams and zero overrides means the promise was never needed.

> A newly added variable called alert_email has no default, because there is no sensible one. It is supplied through a tfvars file in every environment except one, where somebody forgot.
>
> In that environment the nightly pipeline job now runs for two hours and is killed by the job timeout. The log ends after the provider initialisation lines, with no error.

What is happening, and what is the immediate fix?

A. The provider is rate limiting. Add -parallelism to reduce concurrent requests

B. Terraform is waiting at an interactive prompt for the missing variable. Add -input=false

C. The state lock is held by another run. Add -lock-timeout

D. The variable must be given a default, since Terraform cannot run without one

◆ PAUSE AND DECIDE

What is happening, and what is the immediate fix?

A Rate limiting

Would produce retries and eventually errors in the log. The log simply stops, which is the shape of something waiting, not something failing.

B Waiting at a prompt

Correct. A required variable with no value makes Terraform prompt. -input=false turns that into an immediate, named error.

C Lock contention

Terraform reports lock contention explicitly and says who holds the lock. Silence is the tell that this is not it.

D Must have a default

False, and undesirable. Variables without defaults are normal and correct where no safe default exists. A default here would silently send alerts nowhere.

THE SIGNAL

No error message is itself the diagnostic. Terraform is loud about failures and silent only when it is waiting for a person.

A production configuration has always set instance_count = 6 in terraform.tfvars. During a load test three weeks ago, an engineer created a file named zz-loadtest.auto.tfvars setting instance_count = 1, and never removed it.

Since then production has been running one instance. Nobody changed terraform.tfvars, and every plan has reported no changes.

Why did this happen without warning?

A Files ending .auto.tfvars are loaded automatically and override terraform.tfvars

B The file name begins with z, so it is loaded last and wins over all other sources

C Terraform merges conflicting values and chooses the lower one for numeric variables

D The plan reported no changes because state was never refreshed after the load test

◆ PAUSE AND DECIDE

Why did this happen without warning?

A auto.tfvars is auto-loaded and outranks

Correct. Auto-loaded means loaded whether or not you intended it, and .auto.tfvars sits above terraform.tfvars in the order.

B The z makes it win

Lexical order only breaks ties between several .auto.tfvars files. It does not lift the file above the command line, so 'all other sources' is wrong.

C Terraform merges values

Invents a rule. Terraform never merges scalar values or picks between them; the highest-precedence source simply wins.

D State was not refreshed

The plan is correct. One instance is genuinely desired, given the values in force. Nothing is drifting.

THE SIGNAL

Every plan was honest. It compared configuration against state, and the configuration really did ask for one instance.

Module 8 recap

- **Variables in, outputs out, locals stay put.** That table answers most design questions.
- **TF_VAR_ is second weakest,** above only the default. Every tfvars file beats it.
- **terraform.tfvars and *.auto.tfvars load themselves.** Anything else needs -var-file.
- **Command line is strongest, last option wins.** Which is why -var-file layering works.
- **A module's outputs are its only interface.** Undeclared internals are unreachable.
- **A required variable makes Terraform prompt.** In a pipeline that looks like a hang. Use -input=false.

> ON THE EXAM
>
> Variable defaults must be literal. They cannot reference other variables, which is often the hint that you wanted a local.

MODULE 09 · DOMAIN 4 - TERRAFORM CONFIGURATION

Complex Types, Expressions and Functions

The type system, the expression forms, and the functions worth recognising by name.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 9 of 27

admin@sthithapragna.com

Terraform's type system exists so that **mistakes surface at plan time** rather than halfway through an apply.

The type system

FAMILY	TYPES
Primitive	string, number, bool
Collection	list, set, map. One element type, many elements
Structural	object, tuple. Each attribute or position typed separately
Special	any accepts anything. null means absent

ON THE EXAM

Collections are homogeneous and structural types are not. That is the line between the two families.

list or set?

LIST

Ordered, duplicates allowed

- Elements have positions, indexed from zero
- The same value may appear twice
- Order is preserved and meaningful
- Reordering it is a real change

SET

Unordered, no duplicates

- Elements have no position at all
- Duplicates collapse into one
- Order is not preserved or meaningful
- Reordering it is not a change

THE TIEBREAKER

This is the most consequential distinction in the type system, because count uses positions and for_each uses keys. Module ten.

map or object?

MAP

Any keys, one value type

- Keys are arbitrary strings you choose
- Every value has the same type
- Add keys freely without changing the type
- Good for tags and lookup tables

OBJECT

Fixed keys, each typed

- Attribute names are part of the type
- Each attribute has its own type
- Adding an attribute changes the type
- Good for a structured settings block

THE TIEBREAKER

If callers invent the keys, it is a map. If you fix the keys and they vary in type, it is an object.

Conversion, and null

- Terraform **converts between primitives** where it safely can. The string five satisfies a number constraint.
- It converts between **collection types** too, so a list can become a set, losing order and duplicates.
- **That conversion is lossy and silent.** Converting a list to a set discards ordering with no warning.
- **null** means the value is absent, which is different from an empty string or an empty list.
- Setting an argument to null tells Terraform to **use the provider default**, as if you had omitted it.

> ON THE EXAM
>
> null is not zero, not empty, and not false. It means 'no value here', and providers treat it as omission.

Operators and the conditional expression

- Arithmetic, comparison and logical operators work as you would expect from most languages.
- The **conditional expression** uses a question mark and a colon: condition, then the true value, then the false value.
- Both result values are **converted to the same type**, so returning a string and a number gives you two strings.
- There is **no if statement** in Terraform. The conditional expression is the only branching form.
- Common pattern: choose an instance size or a count based on the environment name.

> ON THE EXAM
>
> Terraform is declarative, so there is no control flow. Conditionals choose between values, they do not choose whether code runs.

for expressions

- A **for expression** transforms one collection into another, in square brackets or braces.
- **Square brackets produce a tuple. Braces produce an object,** and braces need a key and a value.
- An optional **if clause** filters, so you can transform and select at the same time.
- Iterating a map gives you two symbols, the key and the value. Iterating a list gives one.
- This is **not a loop that creates resources**. It builds a value. Resource repetition is count and for_each.

> ON THE EXAM
>
> for expressions build values. count and for_each build resources. Confusing the two is a common wrong answer.

Splat expressions

- A **splat**, written as square bracket star, pulls one attribute from every element of a list.
- It is shorthand for a simple for expression, and far more readable when it fits.
- **It only works over a list.** Applied to a single object it will not do what you expect.
- It pairs naturally with **count**, where instances form a list already.
- With **for_each** the instances form a map, so you want **values()** or a for expression instead.

> ON THE EXAM
>
> Splat suits count. for_each produces a map, and a splat over a map is a common mistake.

String templates and heredocs

- **Interpolation** embeds an expression in a string using a dollar sign and braces.
- **Heredoc** syntax carries multi-line strings. The indented form strips leading whitespace.
- Templates support **directives** with a percent sign and braces, giving conditionals and repetition inside a string.
- To output a literal dollar-brace, **double the dollar sign**. The same trick works for the percent form.
- For anything longer than a line or two, put it in a file and use **templatefile** instead.

> ON THE EXAM
>
> Heredocs are for readability in configuration. Real templates belong in their own file, rendered with templatefile.

Functions

- Terraform ships a **large library of built-in functions**, grouped into documented categories.
- Numeric, string, collection, encoding, filesystem, date and time, hash and crypto, IP network, and type conversion.
- **You cannot write your own function in the Terraform language.** There is no function block in HCL.
- Functions are **pure**. Given the same inputs they return the same output, with no side effects.
- **terraform console** is the right way to learn one. Type the call, see the result, no infrastructure involved.

> ON THE EXAM
>
> Learn the categories rather than memorising signatures. Questions name a task, and you need to know which category solves it.

Provider-defined functions, since Terraform 1.8

- **Providers can now supply functions**, not just resources and data sources. Added in Terraform 1.8.
- Called with a namespaced syntax: **provider, then the local name, then the function name**, separated by double colons.
- Launch examples: **arn_parse and arn_build** in AWS, **direxists** in local, **rfc_3339_parse** in time.
- The provider must be declared in **required_providers**, exactly as in module two.
- They must be **pure and offline**. A provider function makes no API calls and has no side effects.

> ON THE EXAM
>
> The old answer 'Terraform has only built-in functions' is now outdated. You still cannot write one in HCL, but providers can ship them.

Functions worth recognising

FUNCTION	WHAT IT DOES
`lookup(map, key, default)`	Read a map key, falling back to a default if absent
`merge(a, b, ...)`	Combine maps. Later arguments win on conflicts
`toset / tolist / tomap`	Convert between collection types explicitly
`length / keys / values`	Count a collection, or pull out its keys or values
`file / templatefile`	Read a file. templatefile also renders variables into it
`cidrsubnet / cidrhost`	Calculate subnet ranges and host addresses from a prefix

ON THE EXAM

merge and lookup dominate map questions. templatefile is the one people reach for file instead of.

try and can

- **try** evaluates several expressions and returns the first that succeeds without error.
- **can** evaluates one expression and returns true or false, depending on whether it errored.
- Use **try** to supply a fallback for a value that may be absent or the wrong shape.
- Use **can** inside a **validation** condition, where you need a boolean rather than a value. Module twelve.
- **Do not wrap large expressions in try.** It will swallow real errors and hide the cause.

> ON THE EXAM
>
> try returns a value, can returns a boolean. That is the whole difference, and it decides which one a question wants.

Working an example

EXPRESSION	RESULT
`toset(["a", "b", "a"])`	A set of two elements. The duplicate collapses
`length({x = 1, y = 2})`	2. length counts elements of any collection
`merge({a=1, b=2}, {b=9})`	a is 1, b is 9. The later argument wins
`lookup({a=1}, "z", 0)`	0. The key is absent, so the default is returned
`[for s in ["a","b"] : upper(s)]`	A tuple holding A and B
`can(tonumber("abc"))`	false. The conversion errors, so can reports it

> ON THE EXAM
>
> Every one of these can be checked in terraform console in a few seconds. That is the fastest way to make them stick.

Learning this with terraform console

- **terraform console** evaluates expressions interactively against your configuration and state.
- It is the fastest way to settle a question about a function, and it changes nothing.
- It works in an **empty directory** too, if you only want to try functions on literal values.
- **Type an expression, read the result.** Ten minutes here beats memorising signatures.
- It also shows you **types**, so you can see whether something is a list, a set or a tuple.

> ON THE EXAM
>
> This is the one hands-on habit worth building for Domain 4. The type system stops being abstract once you can inspect it.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Adding an item recreated unrelated resources	A list where a set was needed. Positions shifted
The configuration breaks when a key is missing	lookup with a default, or try
We need to parse a provider identifier	A provider-defined function, if the provider ships one
Variables in the template are not being substituted	file was used where templatefile was needed
Choose a size based on the environment	A conditional expression. There is no if statement
Build a list of names from a list of objects	A splat, or a for expression

ON THE EXAM

Most function questions describe a task, not a name. Identify the category first, then the function.

A module creates one storage bucket per team, iterating over a variable declared as a list of strings. Teams are listed alphabetically and there are fourteen of them.

A new team named Atlas is added at the front of the list. The plan proposes to destroy and recreate thirteen existing buckets, in addition to creating one.

What is the cause, and what fixes it?

A The provider requires buckets to be created in order. Add depends_on between them

B Instances are tracked by list position. Use a set of strings with for_each so they are tracked by value

C The list must be sorted with the sort function before use

D Bucket names must be globally unique, so Terraform recreates them when any name changes

◆ PAUSE AND DECIDE

What is the cause, and what fixes it?

A Add depends_on

Ordering is not the problem. Terraform is not confused about sequence; it believes thirteen different buckets are now required.

B Track by value, not position

Correct. A list gives positional identity, so inserting at the front shifts every index. A set with for_each keys instances by value.

C Sort the list first

The list was already alphabetical, and Atlas sorts to the front regardless. Sorting cannot rescue an identity scheme based on position.

D Global uniqueness

True of bucket names and irrelevant here. Only one name was added; the other thirteen are unchanged.

THE SIGNAL

Inserting at the front is the worst case. Appending to the end of a list causes no churn at all, which is why this bug hides for years.

> A module accepts a map of per-environment settings and reads a key called backup_retention_days from it. Three of the four environments define that key. The fourth does not, because backups are handled elsewhere.
>
> The configuration fails for that environment with an error saying the key does not exist.

What is the cleanest fix?

A. Use lookup with a default value for the missing key

B. Change the variable type to any, so the missing key is tolerated

C. Wrap the whole module call in a try expression

D. Add the key to every environment with a value of null

◆ PAUSE AND DECIDE

What is the cleanest fix?

A lookup with a default

Correct. lookup exists for exactly this: read a key, and fall back to a stated default when it is absent. try would also work and is less explicit.

B Type any

Relaxes the type constraint, which was never the problem. The key is genuinely missing, and a looser type does not conjure it.

C try around the module

Exactly the misuse HashiCorp warns about. A large try swallows every error inside it, so a real failure would vanish silently.

D Add the key as null

Workable and spreads the special case into four files. It also asserts a value exists when the point is that it does not.

THE SIGNAL

try and lookup would both work. lookup states the intent in the expression itself, which is why it is the cleaner answer.

> A configuration receives a full AWS resource identifier as a string and needs the account number and the region from it. An engineer has written a chain of split and element calls, with a comment apologising for it.
>
> The team is on Terraform 1.12 and uses a recent version of the AWS provider.

What should be suggested at review?

A Extract the logic into a reusable module so the parsing lives in one place

B Use the provider-defined function provider::aws::arn_parse

C Define a custom function in a locals block and call it where needed

D Use a data source, since parsing requires querying the provider

◆ PAUSE AND DECIDE

What should be suggested at review?

A Extract to a module

Centralises the awkwardness without removing it, and a whole module for a string operation is heavy. Reasonable before 1.8; unnecessary now.

B Use the provider function

Correct. Provider-defined functions arrived in 1.8, and the AWS provider shipped ARN parsing among the first of them.

C Custom function in locals

There is no function block in HCL and a local is a value, not a callable. This half of the old rule still holds.

D Use a data source

Provider functions are pure and offline by design. Parsing a string needs no API call, which is why it is a function and not a data source.

THE SIGNAL

Option C is the answer that used to be closest to right. Providers, not the language, are where Terraform gained extensibility.

A module ships a configuration template containing placeholders for a hostname and a port. It reads the template with the file function and passes the result to a resource.

The deployed configuration contains the literal placeholder text rather than the values. Nothing errors; the file simply arrives unchanged.

What is wrong?

A file reads the file verbatim. templatefile is needed to render values into it

B The template must be renamed with a .tftpl extension before file will render it

C The placeholders must be escaped, because file strips interpolation syntax

D The values are not known at plan time, so they render as placeholders

◆ PAUSE AND DECIDE

What is wrong?

A Use templatefile

Correct. file returns the contents unchanged. templatefile takes a second argument, a map of values, and renders them into the template.

B Rename to .tftpl

That extension is a readable convention that editors recognise. It changes nothing about how a function behaves.

C Escape the placeholders

Backwards. file does not strip anything; it is not looking at the contents at all.

D Unknown at plan time

Would show as (known after apply) in the plan, not as literal placeholder text in a deployed file.

THE SIGNAL

Nothing errors, which is the tell. A wrong function that succeeds is harder to spot than one that fails.

Module 9 recap

- **Collections are homogeneous, structural types are not.** That is the line between the families.
- **list is ordered with duplicates. set is neither.** Positional identity is where resource churn comes from.
- **The conditional expression is the only branching form.** There is no if statement.
- **for expressions build values. count and for_each build resources.**
- **try returns a value, can returns a boolean.** Do not wrap large expressions in try.
- **Provider-defined functions exist since 1.8.** You still cannot write one in HCL.

> ON THE EXAM
>
> null means absent, not empty. Assigning null is the same as omitting the argument entirely.

MODULE 10 · DOMAIN 4 - TERRAFORM CONFIGURATION

count, for_each and Dynamic Blocks

Repeating a resource, repeating a nested block, and the plan-time rule that governs both.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 10 of 27

admin@sthithapragna.com

count numbers your instances. **for_each** names them. Identity is the whole difference.

count

- **count** takes a whole number and creates that many instances of the resource.
- **count.index** gives the current instance's position, starting at zero.
- Instances form a **list**, addressed with a number in square brackets.
- Best suited to instances that are **genuinely interchangeable**, such as three identical workers.
- **Identity is positional.** Remove the first element and every later instance is renumbered.

> ON THE EXAM
>
> Use count when the instances differ only in number. The moment they have meaningful names, positions become a liability.

The conditional resource pattern

- **count = 0** creates nothing. This is how you make a resource conditional.
- The usual form pairs it with a **conditional expression**: one if a condition holds, zero otherwise.
- There is **no other way** to make a resource optional. Terraform has no if statement, as module nine established.
- The cost: the resource still uses **index zero**, so references need square bracket zero even when there is only one.
- **for_each over an empty map** achieves the same thing, and keeps named addressing.

> ON THE EXAM
>
> count = 0 is the idiomatic answer to 'create this only in production'. Expect it as a correct option.

for_each

- **for_each** accepts a **map** or a **set of strings**, and creates one instance per member.
- **It does not accept a list.** Convert one with toset, which is why that function appears so often.
- **each.key** is the map key or set member. **each.value** is the map value.
- With a **set**, key and value are identical, so use whichever reads better.
- Instances form a **map**, addressed with a string in square brackets.

> ON THE EXAM
>
> Each instance is associated with a distinct infrastructure object, tracked by key. Adding or removing a member touches only that member.

Working an example

FOR_EACH IS GIVEN	EACH.KEY AND EACH.VALUE
`toset(["a", "b"])`	key and value are both "a", then both "b"
`{ web = 2, api = 5 }`	key "web" value 2, then key "api" value 5
`["a", "b"]`	Error. A list is not accepted; wrap it in toset
`{}`	No instances. Valid, and the empty-map conditional pattern
`var.names (a list)`	Error, unless converted. This is the usual first mistake

ON THE EXAM

A set makes key and value identical. A map is the only way to carry a payload alongside the key.

The idiom you will actually write

- Real input is usually a **list of objects**, and for_each will not take a list.
- The standard move: a **for expression producing a map**, keyed by a stable field of each object.
- That gives named instances **and** the full object as each.value, so nothing is lost.
- **Choose the key carefully.** It is the instance identity, so changing it destroys and recreates.
- This is module nine's for expression and this module's for_each working together.

> ON THE EXAM
>
> Key on something the caller controls and will not change, such as a name. Never on something derived or generated.

count or for_each?

COUNT WHEN

The instances are interchangeable

- They differ only in number
- Nothing distinguishes one from another
- Conditional creation with zero
- Identity is a position

FOR_EACH WHEN

The instances are distinct

- Each has a meaningful name or key
- They may differ in configuration
- Adding or removing one is safe
- Identity is a key

THE TIEBREAKER

Ask whether removing the middle item should disturb the others. With count it does. With for_each it never does.

How instances are addressed

WRITTEN WITH	ONE INSTANCE IS
`count`	aws_instance.web[0] - a number, from zero
`for_each`	aws_instance.web["api"] - a string key
`count on a module`	module.app[0]
`for_each on a module`	module.app["eu-west-1"]
`Neither`	aws_instance.web - no index at all

ON THE EXAM

These addresses appear in plan output, in -target, in state commands and in moved blocks. Recognising them is worth marks on its own.

The plan-time rule

- **Terraform must know every instance address before it can produce a plan.** That is the whole rule.
- So **for_each keys must be known values**, and a **count number must be known**, at plan time.
- Deriving either from a **computed resource attribute** fails. Those values do not exist yet.
- The error says the value **cannot be determined until apply**, and suggests using -target.
- **The real fix is structural:** key on something you already know, such as a variable, not on a generated identifier.

> ON THE EXAM
>
> The instance key becomes part of the resource address, and addresses build the graph. Terraform cannot plan a graph it cannot name.

Two things for_each will not accept

- **Impure functions.** Keys cannot come from uuid, bcrypt or timestamp.
- Terraform defers evaluating impure functions, so their results are not available when instance addresses are decided.
- **Sensitive values.** A sensitive variable, output or resource attribute cannot be used as a for_each argument.
- That follows from addressing. A key becomes part of a resource address, and addresses are printed everywhere.
- Both restrictions are **documented rules**, not quirks. Expect them as options.

> ON THE EXAM
>
> Sensitive values cannot be instance keys, because an instance key is a name that will appear in plans, state and error messages.

Repeating a module

- **count and for_each work on module blocks too**, since Terraform 0.13.
- Everything the module creates is repeated, and addresses gain the index or key at the module level.
- Common use: **one module instance per region, per environment, or per tenant**.
- **The same plan-time rule applies.** Module keys must be known before the plan.
- Aliased providers still have to be **passed in explicitly** with the providers meta-argument, as in module two.

> ON THE EXAM
>
> Repeating a module multiplies everything inside it. That is powerful, and it makes the blast radius of a key change much larger.

Switching between them

- **count and for_each are mutually exclusive** on one resource. A block may use one, never both.
- **Changing from count to for_each changes every instance address**, from numbers to keys.
- Terraform reads that as the old instances vanishing and new ones appearing: **destroy and recreate, all of them**.
- **moved blocks** tell Terraform an instance was renamed rather than replaced. Module seventeen.
- The alternative is **terraform state mv** per instance, which is the same idea done by hand.

> ON THE EXAM
>
> The migration is safe and it is not automatic. Without moved blocks, a purely cosmetic refactor destroys production.

Dynamic blocks

- A **dynamic block** repeats a **nested block** inside a resource. It does not repeat resources.
- It takes a **for_each**, and a **content** block describing the body of each generated block.
- The label names the block being generated, and an **iterator** argument can rename the temporary variable.
- The classic use: **security group rules**, where the number of nested blocks depends on input.
- With a **set**, key and value are identical, and HashiCorp says the key **should not be used**.

> ON THE EXAM
>
> count and for_each repeat the resource. dynamic repeats a block inside it. Questions rely on that confusion.

What a dynamic block cannot generate

- **It can only generate arguments belonging to the resource type, data source, provider or provisioner.**
- **Meta-argument blocks cannot be generated.** That rules out lifecycle and provisioner blocks.
- The reason: Terraform must **process meta-arguments before it is safe to evaluate expressions**.
- So a dynamic lifecycle block is not merely discouraged. It is **impossible**, and the error says so.
- The for_each value must be a collection with **one element per desired nested block**.

> ON THE EXAM
>
> Meta-arguments are read before expressions are evaluated. Anything Terraform needs early cannot be produced by something evaluated late.

HashiCorp's own warning

- **Overuse of dynamic blocks makes configuration hard to read and maintain.** That is the documentation's phrasing.
- The recommended use: **hiding details to build a clean interface for a reusable module**.
- In a root module written for humans, a literal nested block is usually **clearer and better**.
- Dynamically generated configuration is harder to trace, which makes troubleshooting harder too.
- Same shape as the -target warning in module five. Powerful, documented, and explicitly not for routine use.

> ON THE EXAM
>
> If a scenario uses dynamic blocks to avoid typing three nearly identical rules, the answer is usually to type the three rules.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Cannot be determined until apply	A key or count derived from a computed attribute
Create this only in production	count with a conditional expression, or an empty map
Removing one item recreated the others	count where for_each was needed
A refactor proposes to destroy everything	Instance addresses changed. Use moved blocks
The number of nested rules varies by input	A dynamic block, inside one resource
A dynamic lifecycle block is rejected	Meta-arguments cannot be generated. Not a bug

ON THE EXAM

Ask what is being repeated. A resource means count or for_each. A nested block inside a resource means dynamic.

A configuration creates a set of DNS records, one per application server. An engineer writes for_each over a set built from the servers' generated private IP addresses, so each record is named after its address.

terraform plan fails, reporting that the for_each set includes values derived from resource attributes that cannot be determined until apply.

What is the correct fix?

A Apply with -target on the servers first, then plan the records

B Key the for_each on the server names, which are known, and read the IP inside the block

C Wrap the set in toset, which resolves the values at plan time

D Replace for_each with count, since count does not require known values

◆ PAUSE AND DECIDE

What is the correct fix?

A **Target the servers first**

The documented workaround, and a workaround. It must be repeated on every fresh apply, and module five explained why routine targeting is the wrong habit.

B **Key on the known names**

Correct. Instance addresses come from the names, which are known at plan time, while the unknown IP is used only inside the block body.

C **Wrap in toset**

toset changes the type, not the knowability. An unknown value is still unknown once it is in a set.

D **Use count instead**

count has the same rule. Its number must also be known at plan time, so this trades one plan-time error for another.

THE SIGNAL

Only the key must be known. Everything inside the block body may be computed, which is what makes the fix cheap.

> A module is used in development, staging and production. A bastion host and its associated firewall rules should exist in production only. The module already takes an environment variable as a string.
>
> An engineer asks how to make the bastion conditional without splitting the module in two.

What is the idiomatic approach?

A Wrap the resource in an if block that tests the environment variable

B Set count to a conditional expression giving 1 in production and 0 elsewhere

C Use a dynamic block whose for_each is empty outside production

D Declare the resource and set all its arguments to null outside production

◆ PAUSE AND DECIDE

What is the idiomatic approach?

A An if block

There is no if block in Terraform. It is declarative, and the conditional expression chooses between values rather than deciding whether code runs.

B count with a conditional

Correct, and idiomatic. count = 0 creates nothing, so the resource is present in the configuration and absent from the infrastructure.

C An empty dynamic block

Dynamic blocks repeat nested blocks inside a resource. They cannot decide whether the resource itself exists.

D All arguments null

null means 'use the provider default', so the resource is still created. It would be built with defaults, which is worse than either outcome.

THE SIGNAL

for_each over an empty map does the same job and keeps named addressing. Both are correct; count is the one questions expect.

> A module author wants callers to be able to enable prevent_destroy on the resources the module creates. She writes a dynamic block with the label lifecycle, whose for_each is empty when the feature is off.
>
> Terraform rejects the configuration.

Why, and what should she do instead?

A Dynamic blocks cannot generate meta-argument blocks. lifecycle must be written literally

B The label must be quoted, and lifecycle is a reserved word requiring an iterator

C Dynamic blocks require a non-empty for_each, so an empty collection is invalid

D prevent_destroy is not supported inside modules, only in root configurations

◆ PAUSE AND DECIDE

Why, and what should she do instead?

A Meta-arguments cannot be generated

Correct. Terraform processes meta-arguments before it is safe to evaluate expressions, so lifecycle and provisioner blocks cannot be produced dynamically.

B Quoting and iterator

Invents a rule. Labels are quoted anyway, and no keyword becomes valid by adding an iterator argument.

C Empty for_each invalid

An empty collection is perfectly valid and simply generates no blocks. That is a normal and useful pattern.

D Not supported in modules

prevent_destroy works anywhere a lifecycle block is valid. Module scope has nothing to do with it.

THE SIGNAL

prevent_destroy also requires a literal value, so it cannot be driven by a variable either. Module eleven returns to this.

A team has nine production databases created with count over a list of names. After the incident described in module nine, they decide to switch to for_each over a set of the same names.

The change is purely a refactor. The plan proposes to destroy all nine databases and create nine new ones.

What should they do before applying?

A Nothing. The plan is wrong and Terraform will reconcile the addresses on apply

B Add moved blocks mapping each old index to its new key, then re-plan

C Apply with -refresh=false so the existing instances are not re-read

D Revert to count, since migrating between the two is not supported

◆ PAUSE AND DECIDE

What should they do before applying?

A **The plan is wrong**

The plan is correct. Every instance address changed, so from Terraform's view nine objects are no longer declared and nine new ones are. It will do exactly what it says.

B **Add moved blocks**

Correct. A moved block tells Terraform an instance was renamed rather than replaced, so state is updated and no infrastructure is touched.

C **Apply with -refresh=false**

Refresh controls re-reading remote objects. It has no effect on addressing, and would only make the plan less accurate.

D **Migration unsupported**

It is supported and routine. moved blocks exist for precisely this refactor, and terraform state mv did the job before them.

THE SIGNAL

Terraform compares addresses, not intentions. A refactor that changes addresses is indistinguishable from a deletion unless you say otherwise.

Module 10 recap

- **count numbers instances, for_each names them.** Positions shift, keys do not.
- **for_each takes a map or a set of strings.** Never a list, hence toset.
- **Keys and counts must be known at plan time,** because they become addresses in the graph.
- **No impure functions and no sensitive values as for_each keys.** Both are documented rules.
- **count = 0 is how you make a resource conditional.** There is no if statement.
- **dynamic repeats a nested block, not a resource,** and cannot generate meta-argument blocks.

> ON THE EXAM
>
> Switching between count and for_each rewrites every address. Use moved blocks, or Terraform will destroy and recreate everything.

MODULE 11 · DOMAIN 4 - TERRAFORM CONFIGURATION

Dependencies and Lifecycle Rules

depends_on, create_before_destroy, and the three other rules that change how Terraform behaves.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 11 of 27

admin@sthithapragna.com

Lifecycle settings do not change what Terraform tells the provider. They change **how Terraform builds and walks the graph**.

Implicit dependencies, restated

- Referencing one resource's attribute inside another **creates the dependency automatically**.
- This is how nearly all ordering should be expressed. It is precise, self-documenting, and impossible to forget.
- The dependency is **exactly as narrow as the reference**, so the graph stays accurate.
- Module seven's failure: a hardcoded identifier gives the right value and **no dependency at all**.
- Everything else in this module exists for cases references cannot express.

> ON THE EXAM
>
> Reach for depends_on only after establishing that no reference can express the relationship. That is the order of operations.

depends_on

- **depends_on** declares a dependency Terraform cannot infer, because no value passes between the two.
- The classic case: a resource needs **an IAM policy or a permission** that must exist first, but references nothing from it.
- It takes a **list of resource or module addresses**, not arbitrary expressions.
- It is available on **resources, modules and data sources**, and behaves differently on the last of those.
- **It does not force replacement.** A changed dependency is ordered before, not propagated into.

> ON THE EXAM
>
> depends_on adds an edge to the graph. It never passes a value, and it never causes the dependent resource to change.

Implicit or explicit?

IMPLICIT, BY REFERENCE

Almost always this

- Created by using a value
- Narrow and accurate
- Self-documenting in the code
- Keeps plan output informative

EXPLICIT, WITH DEPENDS_ON

Only when nothing is passed

- Declared as a list of addresses
- Coarse: the whole object, not an attribute
- Needs a comment explaining why
- Can make downstream values unknown

THE TIEBREAKER

On a data source, depends_on with a changing dependency defers the read to apply. Module seven showed what that costs.

The lifecycle block

- A nested **lifecycle** block inside a resource. It is a meta-argument, available on every resource type.
- Four behaviour rules: **create_before_destroy, prevent_destroy, ignore_changes, replace_triggered_by**.
- Plus **precondition** and **postcondition**, which are custom conditions and belong to module twelve.
- **It cannot be generated by a dynamic block.** Module ten explained exactly why.
- Rules can be combined, and combinations interact in ways worth knowing.

> ON THE EXAM
>
> The lifecycle block changes Terraform's behaviour, not the resource's configuration. Nothing in it reaches the provider API.

Literal values only

- **Lifecycle settings shape how Terraform builds and traverses the graph.**
- **So only literal values are allowed.** Processing happens too early to evaluate expressions.
- You **cannot** set prevent_destroy from a variable, or drive create_before_destroy from an input.
- Third appearance of one principle: **what the engine needs early cannot come from what runs late**.
- The others: **plan-time keys**, and **dynamic blocks not generating meta-arguments**.

> ON THE EXAM
>
> A module cannot offer 'protect this resource' as a variable. Callers who need that must write the lifecycle block themselves.

create_before_destroy

- By default, a replacement **destroys the old object first**, then creates the new one. That means an interruption.
- **create_before_destroy reverses the order**: build the replacement, then remove the original.
- The reason to use it: **avoiding downtime** on load balancers, certificates, addresses and instances.
- **It is opt-in,** and deliberately so. Terraform cannot know whether reversing the order is safe for a given resource type.
- **Destroy provisioners do not run** when create_before_destroy is set.

> ON THE EXAM
>
> This is objective 4f by name. Expect it as the answer whenever a scenario says a replacement must not cause an outage.

Why it is opt-in: the name collision

- **Many remote object types have unique name requirements**, and two cannot exist at once.
- So creating the replacement first **fails**, because the original still holds the name.
- Some resource types offer a **name prefix** option that appends a random suffix, which resolves it.
- **Terraform cannot activate those features for you.** You must know the constraints of the resource type.
- Other constraints exist too: quotas, port bindings, and anything else that admits only one holder.

> ON THE EXAM
>
> create_before_destroy plus a fixed unique name is a failed apply, not a safer one. Check the resource type first.

It propagates to dependencies

- **create_before_destroy propagates to all of a resource's dependencies.**
- Set it on A, and if A depends on B, it is **enabled on B implicitly**.
- **You cannot override it back to false on B.** That would imply a cycle.
- The logic is unavoidable: A cannot be created before its dependency exists.
- So it can change replacement behaviour **for things you did not name**.

> ON THE EXAM
>
> The propagation is not a courtesy. Without it the ordering would be impossible, which is why Terraform refuses to let you disable it.

prevent_destroy

- **prevent_destroy** makes Terraform **error rather than proceed** with any plan that would destroy the resource.
- The intent: a guard on databases, state buckets and anything else where deletion is unrecoverable.
- **It blocks replacement too**, because replacement destroys the existing object.
- **create_before_destroy does not bypass it.** That changes ordering, not whether a destroy happens.
- It must be a **literal true**. It cannot be driven by a variable.

> ON THE EXAM
>
> Any plan containing a destroy of that resource fails, including a replacement forced by an unrelated attribute change.

What prevent_destroy misses

- **Remove the resource block and the protection goes with it.** The apply destroys it.
- The rule lives **in the configuration**. Deleting the configuration deletes the rule.
- Documented behaviour, not a bug, and the most misunderstood thing about it.
- To keep the object and stop managing it, **remove it from state**. Module eighteen.
- Real protection lives in the **provider**: deletion flags, and permissions.

> ON THE EXAM
>
> prevent_destroy guards against a careless change. It cannot guard against deleting the block, because the guard was in the block.

ignore_changes

- Takes a **list of attribute names** that Terraform should stop comparing against configuration.
- The use case: an **external system legitimately modifies the resource**, such as autoscaling or a policy engine.
- Elements of maps and lists can be named with index notation, so you can ignore one tag rather than all of them.
- **Only attributes defined by the resource type.** It cannot be applied to itself or to any other meta-argument.
- **Ignoring an attribute does not prevent replacement** when a different argument still forces one.

> ON THE EXAM
>
> This is the one lifecycle rule that hides information from you. Every use of it should carry a comment saying why.

ignore_changes = all

- The keyword **all** ignores every attribute, rather than a named list.
- **Terraform can then create and destroy the object, and will never propose an update to it.**
- So your configuration **stops describing** the running resource, and nobody is told.
- Occasionally correct: an object handed to another system entirely after creation.
- Usually a mistake, and a durable one, because the plan will stay clean while reality drifts away.

> ON THE EXAM
>
> You keep the create and destroy stages and give up the update stage. That is a big trade to make in one word.

replace_triggered_by

- Added in **Terraform 1.2**. Replaces this resource when something else changes.
- Takes a list of expressions referencing **managed resources, instances, or instance attributes**.
- The use case: a resource that must be rebuilt when its **configuration source** changes, with no argument linking them.
- **depends_on cannot do this.** That orders operations; it never propagates a replacement.
- Inside count or for_each, you can use **count.index or each.key** to reference the matching instance.

> ON THE EXAM
>
> depends_on says 'after'. replace_triggered_by says 'again'. Those are different requests, and questions pair them.

How the rules interact

COMBINATION	WHAT HAPPENS
prevent_destroy with create_before_destroy	Still blocked. Ordering changed; a destroy still occurs
create_before_destroy on a dependency	Enabled implicitly, and cannot be set back to false
ignore_changes with a forced replacement	Replacement still happens, if another argument forces it
ignore_changes with replace_triggered_by	The trigger still fires. Ignoring is not immunity
create_before_destroy with destroy provisioners	The destroy provisioners do not run at all

ON THE EXAM

These rules are not independent switches. Combinations appear in questions precisely because people assume they are.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
A replacement must not cause an outage	create_before_destroy
The apply failed because the name is taken	create_before_destroy on a uniquely-named object
An autoscaler keeps fighting Terraform	ignore_changes on that specific attribute
Plans are clean but the resource has drifted	ignore_changes = all, hiding the difference
Rebuild this whenever that changes	replace_triggered_by, not depends_on
The database was destroyed despite protection	The resource block was deleted, taking the rule with it

ON THE EXAM

Ask what is being requested: an order, a guard, a blind spot, or a rebuild. Each maps to exactly one rule.

A production database carries prevent_destroy = true, added two years ago after an incident. During a repository cleanup, an engineer removes what she believes is an unused module call, which included that database's resource block.

The plan shows the database will be destroyed. She is surprised, having expected prevent_destroy to block it.

What is the correct explanation?

A prevent_destroy only applies to terraform destroy, not to a normal apply

B Removing the resource block removes the rule with it, so the destroy proceeds

C prevent_destroy expired, because lifecycle rules are only honoured for one year

D The plan is a preview only. Apply would fail with a prevent_destroy error

◆ PAUSE AND DECIDE

What is the correct explanation?

A Only blocks destroy command

False. prevent_destroy blocks any plan containing a destroy of that resource, including a replacement forced by an attribute change.

B The rule went with the block

Correct, and documented. HashiCorp states Terraform destroys the infrastructure if you remove the configuration, even when prevent_destroy was enabled.

C It expired

Invents a mechanism. Lifecycle rules have no lifetime; they exist exactly as long as the block containing them.

D Apply would fail

The dangerous belief. The plan is accurate, and the apply would carry it out. There is no second safety net behind the plan.

THE SIGNAL

The guard was inside the thing that was deleted. To keep the object while stopping management, remove it from state instead.

A load balancer is replaced whenever its subnet configuration changes, causing about ninety seconds of downtime. An engineer adds create_before_destroy = true to eliminate it.

The next apply fails partway through, reporting that a resource with the requested name already exists. The load balancer has a fixed name set from a variable.

What is the cause?

A create_before_destroy requires the resource to also set prevent_destroy = false

B The replacement cannot be created while the original still holds the unique name

C create_before_destroy is unsupported for load balancers in most providers

D The apply needs -parallelism=1 so the destroy completes before the create begins

◆ PAUSE AND DECIDE

What is the cause?

A Needs prevent_destroy = false

Invents a dependency between two unrelated rules. Absent prevent_destroy is simply absent; it does not need declaring as false.

B The name is still held

Correct. This is exactly why HashiCorp made the setting opt-in: many object types have unique name requirements and cannot coexist.

C Unsupported for load balancers

It is a Terraform-side ordering rule available on every resource type. The constraint is the naming, not the resource category.

D Set parallelism to 1

Reverts to destroy-then-create sequencing, which is the downtime she was removing. It undoes the change rather than fixing it.

THE SIGNAL

The fix is a name prefix with a random suffix, where the resource type offers one. Terraform cannot enable that for you.

> Eighteen months ago, an engineer added ignore_changes = all to a compute cluster resource, to stop Terraform reverting changes an operations team was making by hand. It worked, and plans have been clean ever since.
>
> A new engineer runs a plan expecting to change the instance type. The plan reports no changes at all.

What should the team understand and do?

A State is stale. Run terraform apply -refresh-only to reconcile it

B ignore_changes = all means Terraform will never propose an update. Narrow it to the attributes actually managed elsewhere

C The instance type is a computed attribute and cannot be changed by Terraform

D Add replace_triggered_by so the change is applied as a replacement instead

◆ PAUSE AND DECIDE

What should the team understand and do?

A Refresh-only

Refresh-only records reality into state. It changes nothing about the rule that suppresses every proposed update.

B Narrow the ignore list

Correct. all keeps create and destroy and removes update entirely. Naming the specific contested attributes restores management of everything else.

C Computed attribute

Instance type is an ordinary configurable argument. Nothing about it is provider-computed.

D replace_triggered_by

Forces replacement when something else changes. It neither restores update behaviour nor addresses why the update was suppressed.

THE SIGNAL

The original fix was correct in intent and far too wide. Eighteen months of clean plans is the symptom, not the reassurance.

> An application instance reads its configuration from an object in a storage bucket at boot time only. When the object's contents change, the instance keeps running the old configuration until it is rebuilt.
>
> The team wants the instance rebuilt automatically whenever that object changes. No argument on the instance references the object.

Which mechanism does this?

A depends_on referencing the storage object

B replace_triggered_by referencing the storage object

C create_before_destroy on the instance

D ignore_changes on the instance's user data

◆ PAUSE AND DECIDE

Which mechanism does this?

A depends_on

Orders operations only. The object would be updated before the instance is considered, and the instance would not change at all.

B replace_triggered_by

Correct. It replaces this resource when a referenced resource or attribute changes, which is precisely the requirement.

C create_before_destroy

Changes the order of a replacement that is already happening. It never causes one.

D ignore_changes

The opposite direction entirely. It suppresses reactions to change rather than creating one.

THE SIGNAL

depends_on says after. replace_triggered_by says again. Scenarios pair them precisely because they look similar in a list.

Module 11 recap

- **Prefer implicit dependencies.** depends_on is for relationships no reference can express.
- **Lifecycle settings take literal values only,** because they shape the graph before expressions are evaluated.
- **create_before_destroy avoids downtime** and is opt-in because of unique name constraints.
- **It propagates to dependencies** and cannot be disabled on them.
- **prevent_destroy dies with its block.** Delete the resource block and the infrastructure goes.
- **depends_on says after. replace_triggered_by says again.**

> ON THE EXAM
>
> ignore_changes = all keeps create and destroy and removes update. Clean plans then mean nothing at all.

MODULE 12 · DOMAIN 4 - TERRAFORM CONFIGURATION

Custom Conditions and Check Blocks

Four ways to assert something is true, and the one that deliberately does not stop you.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 12 of 27

admin@sthithapragna.com

Type constraints check the **shape** of a value. Custom conditions check whether it is **sensible**.

The four mechanisms

MECHANISM	WHEN IT RUNS, AND WHAT FAILURE DOES
Variable validation	When variable values are set. Blocks the operation
Precondition	Before the enclosing block is provisioned. Blocks
Postcondition	After the enclosing block is provisioned. Blocks
Check block	Last step of plan or apply. Warns, and continues

ON THE EXAM

Three halt the operation. The check block is the deliberate exception, and that is the distinction questions turn on.

Variable validation

- A **validation** block inside a variable block, holding a **condition** and an **error_message**.
- Available since **Terraform 0.13**. A variable may carry **several** validation blocks.
- The condition must return a **boolean**. This is where **can()** earns its place, as module nine noted.
- It is the **first line of defence**, catching a bad input before any provider is contacted.
- It **cannot see resource attributes**, because no resources exist when variables are evaluated.

> ON THE EXAM
>
> Validation runs earliest and therefore knows least. Anything requiring real infrastructure has to be a precondition instead.

Since 1.9: validation can look outside the variable

- **Before Terraform 1.9, a validation condition could reference only the variable itself.**
- **Since 1.9, conditions may reference other input variables, data sources and local values.**
- So rules like **if this flag is true, that other variable must not be empty** now live in the variable block.
- The old workaround was a **precondition on a placeholder resource**, often a null_resource.
- The exam tests **1.12**, so cross-object validation is available and the workaround is outdated.

> ON THE EXAM
>
> Older study material states flatly that validation cannot reference other variables. That was true, and it stopped being true in 1.9.

Writing the error message

- The message is what a user sees when the condition fails. It is the point of the whole exercise.
- It should be a **full sentence** describing the problem and, ideally, how to fix it.
- **Since Terraform 1.2, error messages are evaluated as expressions**, so you can interpolate values.
- That means the message can quote **what was actually supplied**, not just what was required.
- A message reading 'invalid value' wastes the mechanism entirely.

> ON THE EXAM
>
> The condition catches the mistake. The message is what determines whether the person can fix it without asking you.

Preconditions

- A **precondition** block inside **lifecycle**, holding a condition and an error message. Terraform 1.2 or later.
- Evaluated **before** the enclosing block is provisioned. If it fails, Terraform halts further evaluation.
- It documents an **assumption**: this resource only makes sense if something else is true.
- Unlike variable validation, it **can reference other resources and data sources**.
- The classic use: checking a looked-up image, subnet or certificate is suitable before building on it.

> ON THE EXAM
>
> A precondition fails before anything is created, so nothing is left half-built. That is why it is preferable to a postcondition where either would work.

Postconditions

- A **postcondition** block, also inside lifecycle. Evaluated **after** the object is provisioned or read.
- It can use **self** to refer to the enclosing object's own attributes, which a precondition cannot.
- It documents a **guarantee**: whatever this resource produced, it satisfies these properties.
- **A failure prevents changes to resources that depend on it**, so the damage stops spreading.
- Useful on a **data source** too, asserting that what was read is fit to use.

> ON THE EXAM
>
> self is the practical tell. If the condition must inspect the object itself, only a postcondition can see it.

Precondition or postcondition?

PRECONDITION

An assumption, checked first

- Runs before provisioning
- Cannot use self
- Nothing is created if it fails
- Checks inputs and other objects

POSTCONDITION

A guarantee, checked after

- Runs after provisioning or reading
- Can use self
- The object exists before it fails
- Checks the result

THE TIEBREAKER

Where both would work, prefer the precondition. Failing before creation is cheaper than failing after it.

Where each one is allowed

- **validation**: inside a variable block only.
- **precondition and postcondition**: in a resource or data source lifecycle block.
- **Output values accept a precondition** too, which is how a module checks what it is about to hand back.
- **Module blocks do not accept them.** There is no lifecycle block on a module call.
- **check**: a top-level block in the configuration, not nested in anything.

> ON THE EXAM
>
> An output precondition is the natural place for a module to assert its own contract before returning a value.

check blocks

- A top-level **check** block containing one or more **assert** blocks, each with a condition and a message.
- Introduced in **Terraform 1.5**, and executed as the **last step** of a plan or apply.
- It may contain a **scoped data source**, available only inside that check, for querying something live.
- That is how you assert about **the running system** rather than about configuration.
- The typical example: fetch a health endpoint and assert it returns success.

> ON THE EXAM
>
> A check runs after everything else, so it can see the finished result of the whole operation rather than one resource.

A check warns. It does not fail.

- **When an assertion fails, Terraform reports a warning and continues the current operation.**
- The run still succeeds. Nothing is rolled back and nothing is blocked.
- That is the point: **operational visibility without stopping a deployment**.
- So a check is the **wrong tool** for anything that genuinely must not proceed.
- HCP Terraform builds **continuous validation** on check blocks, re-running them on a schedule. Module twenty-three.

> ON THE EXAM
>
> If a scenario says the deployment must be stopped, a check block is the wrong answer no matter how well the condition fits.

What ends up in state

- **Terraform records the results of precondition and postcondition checks to state.**
- **It does not record the contents of the checks**, only whether they passed.
- So state tells you a condition held. It does not tell you what the condition was.
- Combined with module four, that is another reason to read state through supported commands rather than directly.
- Check block results surface as warnings in output, and HCP Terraform tracks them over time.

> ON THE EXAM
>
> The result is stored, the rule is not. A condition removed from configuration leaves no trace of what it once asserted.

Conditions are not policy as code

- Custom conditions live **inside the configuration they check**, written by whoever wrote it.
- So an author who does not want a rule can simply **not write it**, or delete it.
- **Sentinel and OPA policies sit outside**, are owned by a different team, and cannot be edited by the author.
- That separation is the whole point of governance. Module twenty-three covers it.
- Use conditions for **correctness**. Use policy for **enforcement across teams**.

> ON THE EXAM
>
> A condition is a module author's assertion about their own code. A policy is an organisation's rule about everybody's code.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Reject a bad input before anything is created	Variable validation
This rule involves two different variables	Variable validation, since 1.9
Check the image is suitable before building on it	A precondition
Verify what the resource actually produced	A postcondition, using self
Alert us, but do not block the deployment	A check block
This must never be allowed to proceed	Not a check block. Use a precondition

ON THE EXAM

Two questions settle it: at what moment must this be known, and should failure stop the operation?

A platform team wants to be told when any TLS certificate managed by their configuration is within thirty days of expiry. They run plans nightly in a pipeline.

The requirement is explicit: the warning must appear in the run output, and it must never cause an unrelated deployment to fail.

Which mechanism fits?

A. A postcondition on the certificate resource

B. A check block with an assert on the expiry date

C. A variable validation comparing the expiry to the current date

D. A precondition on every resource that uses the certificate

◆ PAUSE AND DECIDE

Which mechanism fits?

A A postcondition

Would fail the operation, and worse, would block every dependent resource. An expiry warning would become a deployment outage.

B A check block

Correct. A failed assertion reports a warning and the operation continues, which is exactly the stated requirement.

C Variable validation

Runs when variables are set, before any resource exists, so it cannot see a certificate's expiry date. It would also block.

D Preconditions everywhere

Blocks, and multiplies the problem across every consumer. The certificate would take the whole configuration down with it.

THE SIGNAL

The deciding sentence is 'must never cause an unrelated deployment to fail'. Only one of the four mechanisms behaves that way.

> A module takes a boolean enable_logging and a string log_bucket_name. If logging is enabled, the bucket name must not be empty. The team is on Terraform 1.12.
>
> A senior engineer says variable validation cannot reference another variable, and proposes a null_resource carrying a precondition to enforce the rule.

How should the reviewer respond?

A Approve. The null_resource precondition is the documented pattern for cross-variable rules

B Since Terraform 1.9, validation conditions may reference other variables. Put the rule in the variable block

C Use a check block instead, since it can see both variables

D Make log_bucket_name a required variable with no default, which removes the need for a rule

◆ PAUSE AND DECIDE

How should the reviewer respond?

A Approve the workaround

It was the documented pattern, before 1.9. On 1.12 it adds a placeholder resource to state for no benefit.

B Use cross-object validation

Correct. Since 1.9, validation conditions may reference other input variables, data sources and local values.

C A check block

Warns rather than blocks, so an invalid combination would deploy anyway. It also runs last, long after the damage.

D Make it required

Forces every caller to supply a bucket name even with logging disabled, which changes the module's interface to avoid writing one rule.

THE SIGNAL

The senior engineer is not wrong, only out of date. Option A was the correct answer for several years, which is what makes it tempting.

> A module looks up a machine image with a data source, then launches instances from it. The module only works with images built for the ARM architecture. Callers occasionally supply a filter that matches an x86 image, and the failure appears much later as an unhelpful provider error.
>
> The author wants a clear message before anything is created.

Where should the condition go?

A. A validation block on the image filter variable

B. A precondition on the instance resource, checking the data source's architecture attribute

C. A postcondition on the instance resource using self

D. A check block asserting the architecture after apply

◆ PAUSE AND DECIDE

Where should the condition go?

A Validation on the filter

Validation runs before anything is read, so it can inspect the filter string but never the image the filter matched. The filter is not the problem.

B Precondition on the instance

Correct. By then the data source has been read, so the architecture is knowable, and failing here creates nothing.

C Postcondition with self

Runs after the instance is provisioned. The stated requirement is a clear message before anything is created.

D A check block

Runs last and only warns, so the wrong instances would be built and the operation would succeed.

THE SIGNAL

A postcondition on the data source would also work, and is arguably better: it fails at the point the wrong image was read.

> A security team requires that no storage bucket created by a shared module can ever be publicly readable. They add a check block with an assert on the bucket's access setting, and a clearly worded error message.
>
> Three weeks later, a bucket is deployed with public access. The pipeline run that created it is marked successful, and the warning is in the log.

What went wrong, and what should replace it?

A. The assert condition was written incorrectly and evaluated true

B. A check block only warns. Use a precondition or postcondition, which block the operation

C. Check blocks only run during plan, not apply, so the apply was unaffected

D. The check block needed a scoped data source to read the live bucket setting

◆ PAUSE AND DECIDE

What went wrong, and what should replace it?

A Condition was wrong

The warning is in the log, which means the assertion evaluated false and fired correctly. The mechanism worked; the choice of mechanism did not.

B Checks warn, conditions block

Correct. A failed assertion reports a warning and the operation continues. Enforcement requires a mechanism that halts.

C Only run during plan

False. A check block executes as the last step of plan and of apply. Both were unaffected because warnings never block.

D Needed a scoped data source

Would change what the check can see, not what a failure does. A better-informed warning is still a warning.

THE SIGNAL

The deliberate inverse of scenario 1. Same mechanism, opposite requirement, opposite answer. Read what failure must do.

Module 12 recap

- **Four mechanisms, four moments.** Validation, precondition, postcondition, check.
- **Only the check block warns.** The other three halt the operation.
- **Since 1.9, validation can reference other variables, data sources and locals.**
- **Only a postcondition has self.** If the condition inspects the object itself, that is the answer.
- **Prefer the earliest mechanism that can see what it needs to see.**
- **Module blocks accept none of them.** Outputs accept a precondition.

> ON THE EXAM
>
> State records that a condition passed, not what it asserted. The rule lives only in your configuration.

MODULE 13 · DOMAIN 4 - TERRAFORM CONFIGURATION

Sensitive Data, Ephemeral Values, Write-Only Arguments

The problem raised in module four, and the mechanisms that finally solve it.

DOMAIN 4 of 8 COURSE WEIGHT 24% MODULE 13 of 27

admin@sthithapragna.com

sensitive hides a value. **Ephemeral** means the value is never written down at all.

Where secrets ended up, before all this

- **State.** Every attribute a provider returns, in plain text. Module four.
- **Saved plan files.** Full configuration and all planned values, in cleartext. Module five.
- **Outputs**, which are stored in state, and revealed by output -raw or -json. Module eight.
- HashiCorp's response was blunt: **treat your state as sensitive data**, and they still advise it.
- **sensitive = true never changed any of this.** It suppresses display, and nothing more.

> ON THE EXAM
>
> Three modules made this point separately. It is one problem with one shape, and it has one root: the value was persisted.

The mitigations that came first

- **Encrypt state at rest**, and use a backend that supports it. Necessary, and not sufficient.
- **Restrict who can read the backend.** Anyone with read access sees plain text, encryption or not.
- **Never commit state, tfvars files or plan files.** All three are ordinary text.
- **Supply secrets at run time**, through environment variables or a secrets manager, rather than in files.
- **Rotate anything that has been exposed.** Removing a value from a file does not un-expose it.

> ON THE EXAM
>
> These remain correct and remain necessary. They reduce who can see the secret. They never stopped it being written down.

Vault, and why a data source is not the answer

- The objective names **secrets management with Vault**, and the Vault provider is the obvious tool.
- **But reading a secret with a data source puts it in state.** A data source result is stored like any other.
- So the classic pattern moved the secret out of your repository and **straight into your state file**.
- It was still an improvement, because the secret was centrally managed and rotatable.
- **An ephemeral resource is the fix**, and Vault ships them. Same source, no persistence.

> ON THE EXAM
>
> This is the trap in the traditional answer. Fetching from a secrets manager solves distribution. It never solved persistence.

Ephemerality

- An **ephemeral** value exists only for the duration of the current operation.
- **It is never written to the plan artefact or the state file.** That is the definition, not a side effect.
- Introduced in **Terraform 1.10** for input variables, output values, and a new kind of resource.
- Extended in **Terraform 1.11** so ephemeral values could reach ordinary managed resources.
- The exam tests **1.12**, so both halves are in scope.

> ON THE EXAM
>
> Two releases, two halves. 1.10 created ephemeral values. 1.11 gave them somewhere useful to go.

Ephemeral variables and outputs

- A variable or output marked **ephemeral = true** is not persisted anywhere.
- **An ephemeral output is not stored in state**, so terraform output cannot read it afterwards.
- That is the point. It can be consumed within the run, and it leaves no record.
- **In 1.10, ephemeral values could only flow into other ephemeral contexts.**
- So a token could authenticate a provider, and could not be passed to a normal resource argument.

> ON THE EXAM
>
> sensitive and ephemeral are different arguments answering different questions. A value can be both.

Ephemeral resources

- A third resource mode, declared with an **ephemeral** block, alongside managed resources and data sources.
- Terraform **opens** it, uses it, optionally **renews** it, then explicitly **closes** it before the phase ends.
- **They are executed during every plan and every apply.** A new value each time.
- They are **nodes in the dependency graph**, so anything depending on one is provisioned after it.
- Examples: a generated random password, a fetched secret version, a short-lived token.

> ON THE EXAM
>
> Three resource modes now: managed resources create things, data sources read things, ephemeral resources borrow things.

Open, renew, close

- Terraform **opens** an ephemeral resource when its value is first needed in a phase.
- It **renews** it if the provider says the value is leased and would otherwise expire mid-run.
- It **closes** it explicitly before the phase ends, so a temporary credential is revoked rather than abandoned.
- That is why these suit **dynamically issued credentials**, not just static secrets.
- A plan and an apply are **separate phases**, so the cycle happens twice across a normal workflow.

> ON THE EXAM
>
> The close step is the part that distinguishes this from just 'not storing it'. The credential is actively given back.

Data source or ephemeral resource?

DATA SOURCE

Reads, and remembers

- Read during plan, or deferred to apply
- Result stored in state
- Other resources reference its attributes
- Right for configuration you look up

EPHEMERAL RESOURCE

Borrows, and forgets

- Opened, used, closed within the operation
- Nothing stored, ever
- Feeds providers and write-only arguments
- Right for secrets and credentials

THE TIEBREAKER

Both read from somewhere else. The difference is entirely about what happens afterwards.

Write-only arguments

- **Terraform 1.11.** Managed resource attributes that are **not persisted to the plan or state artefacts**.
- Named by convention with a **_wo** suffix, such as password_wo on a database instance.
- **The provider is the terminal point.** It uses the value to call the API, then Terraform discards it.
- **Unlike other ephemeral constructs, they accept ephemeral and non-ephemeral values.**
- **Not usable with set attributes, set nested attributes or set nested blocks.**

> ON THE EXAM
>
> This is the bridge. Ephemeral values could not reach a managed resource until a write-only argument gave them a door.

The version argument, and why it exists

- Terraform does not store a write-only value, so **it cannot detect when that value changed**.
- So write-only arguments are **sent to the provider on every operation**, regardless.
- Providers therefore pair them with a **version argument**, such as password_wo_version.
- **The version IS stored in state.** Incrementing it is how you tell Terraform to accept a new value.
- With several write-only arguments on one resource, **keep their versions in sync** so they update together.

> ON THE EXAM
>
> Ephemeral resources run every time, so without the version a password would appear to change on every apply. The version is the brake.

End-to-end ephemerality

01

Generate

An ephemeral resource produces a random password. Nothing is stored.

02

Store

Write it to a secrets manager using a write-only argument.

03

Fetch

Read it back with an ephemeral resource during apply.

04

Use

Pass it to the database's write-only password argument.

ON THE EXAM

The secret is generated, stored, retrieved and used without ever appearing in the plan file or state. That is the documented pattern.

Finding out what supports it

- **Providers indicate in the Terraform Registry whether an argument is write-only.**
- **Support is per resource, per provider.** Not every sensitive argument has a write-only counterpart.
- Ephemeral resources are likewise **published by providers**, and listed in the registry.
- So adopting this is partly a **provider version** question, which is module three all over again.
- You also need **Terraform 1.11 or later** for write-only, and 1.10 for ephemeral resources.

> ON THE EXAM
>
> Two version constraints, on two different pieces of software. The Terraform version and the provider version must both be new enough.

Does this end up in state?

THE VALUE	IN STATE?
A resource attribute returned by the provider	Yes, in plain text
A variable marked sensitive	Yes. sensitive is display only
A data source result, from Vault or anywhere	Yes
An output marked sensitive	Yes, and redacted when printed
An ephemeral resource's value	No, never
A write-only argument's value	No. But its version argument is stored

ON THE EXAM

Five of these six were true before 1.10, and the last two are the whole of what changed.

What this does not do

- **It does not clean your existing state.** A password already recorded there stays exactly where it is.
- **It does not retrospectively protect anything.** Adoption changes future runs only.
- So adopting it should be paired with **rotating the secrets that were already exposed**.
- **It does not replace access control or encryption at rest.** State still holds plenty worth protecting.
- HashiCorp still advises **treating state as sensitive data**, after all of this.

> ON THE EXAM
>
> A secret in state is exposed the moment it is written. Nothing added later un-writes it, which is the same logic as a Git commit.

Which mechanism, for which requirement

THE REQUIREMENT	THE MECHANISM
Do not print this in the terminal	sensitive = true
Do not store this in state at all	ephemeral, or a write-only argument
Fetch a secret without persisting it	An ephemeral resource
Pass a secret to a normal resource	A write-only argument on that resource
Change the stored secret deliberately	Increment the write-only version argument
Protect what is already in state	Rotate it. Nothing else works

ON THE EXAM

sensitive and ephemeral are not alternatives. One controls display, the other controls persistence, and a value can be both.

> A team stores database credentials in Vault and reads them into Terraform with the Vault provider's data sources, believing this keeps the passwords out of their configuration.
>
> A security review finds the passwords in plain text in the remote state file. The team is on Terraform 1.12 with a current Vault provider.

What should they change?

A Mark the data source's output as sensitive, which prevents it being written to state

B Replace the data source with an ephemeral resource, which is never persisted

C Enable encryption at rest on the backend, which resolves the finding

D Move the credentials to TF_VAR_ environment variables instead of Vault

◆ PAUSE AND DECIDE

What should they change?

A Mark it sensitive

The misconception this course has flagged in four modules. sensitive suppresses display; it has never affected storage.

B Use an ephemeral resource

Correct. Same secret, same source, and the value is never written to the plan artefact or state. Available on 1.12 with a current provider.

C Encrypt at rest

Worth doing anyway, and it does not resolve this. The reviewer had legitimate read access, and encryption does nothing about that.

D Environment variables

Moves the secret out of Vault into a pipeline configuration, losing central management and rotation. The value would still land in state.

THE SIGNAL

Fetching from a secrets manager solved distribution and never solved persistence. Ephemeral resources are what closed that gap.

> An engineer configures a database with an ephemeral random_password resource feeding the password_wo argument. She sets password_wo_version to 1 and applies successfully.
>
> Security asks for the password to be rotated. She reruns apply, expecting the ephemeral resource to generate a new password. The plan reports no changes.

What is happening?

A The ephemeral resource caches its value in state after the first apply

B The write-only version argument is unchanged, so Terraform does not accept a new value

C Write-only arguments can only be set at creation and never updated

D The plan needs -replace on the database to pick up the new password

◆ PAUSE AND DECIDE

What is happening?

A It caches in state

Ephemeral resources persist nothing, by definition, and they execute on every plan and apply. A new password genuinely was generated.

B The version is unchanged

Correct. Terraform cannot see the value, so the version argument stored in state is the only signal it has. Increment it to rotate.

C Create-only

False. Updating is exactly what the version mechanism exists to support.

D Use -replace

Would destroy and recreate the database to change one argument. The version increment updates it in place.

THE SIGNAL

The ephemeral resource did generate a new password. Terraform simply had no reason to send it, because nothing it can see had changed.

A platform team migrates twelve configurations to use ephemeral resources and write-only arguments. The work is completed and reviewed, and every new run keeps secrets out of state.

They close the security finding, noting that credentials are no longer present in Terraform state.

What is wrong with closing the finding?

A Nothing. Once write-only arguments are in use, previous values are purged on the next apply

B Existing state still contains the old credentials. They must be rotated, and old state versions considered

C Write-only arguments only take effect after terraform init -upgrade is run

D The finding should stay open until every provider supports write-only arguments

◆ PAUSE AND DECIDE

What is wrong with closing the finding?

A. Old values are purged

Nothing purges them. Adoption changes what future runs write; it never rewrites what past runs already wrote.

B. Rotate, and check history

Correct. The credentials were exposed the moment they were written, and a versioned backend keeps earlier state versions containing them.

C. Needs init -upgrade

Confuses provider installation with data handling. -upgrade re-resolves versions and touches no stored values.

D. Wait for all providers

Would keep the finding open indefinitely for no benefit. Partial adoption genuinely reduces exposure going forward.

THE SIGNAL

Exactly module one's Git logic, one layer down. Removing a secret from future writes does not un-write the past ones.

> A configuration authenticates a third-party provider using a long-lived API token, currently supplied through an environment variable and shared across the team.
>
> The team wants a short-lived token issued per run, from their secrets platform, with nothing stored in Terraform's artefacts. They are on Terraform 1.12.

What approach fits?

A. A data source that fetches the token, referenced from the provider block

B. An ephemeral resource fetching the token, referenced from the provider block

C. A write-only argument on the provider block

D. A sensitive variable holding the token, supplied by the pipeline

◆ PAUSE AND DECIDE

What approach fits?

A. A data source

Fetches the token and stores it in state, which is scenario 1's problem again. It also cannot issue a per-run credential.

B. An ephemeral resource

Correct. Provider configuration is an ephemeral context, so an ephemeral resource can populate it, and the token is opened, used and closed within the run.

C. Write-only on the provider

Write-only arguments are a feature of managed resources. A provider block does not need them, because it is already ephemeral.

D. A sensitive variable

Keeps the long-lived shared token and only hides it in output. It addresses neither the lifetime nor the storage requirement.

THE SIGNAL

Provider configuration was ephemeral from the start, in 1.10, before write-only arguments existed. Authentication was the first use case.

> A module author is reviewing three requirements. First, an admin password must not appear in state. Second, a cost centre code should not be printed in plan output, though storing it is fine. Third, the module must work for consumers on Terraform 1.9.
>
> She asks which mechanisms to use for the first two, and what the third constraint implies.

What is the correct assessment?

A. Use ephemeral for both, and 1.9 is sufficient for both features

B. Write-only for the password, sensitive for the cost centre, and 1.9 consumers cannot use the write-only argument

C. Use sensitive for both, since sensitive covers storage and display

D. Use write-only for both, since it supersedes sensitive entirely

◆ PAUSE AND DECIDE

What is the correct assessment?

A Ephemeral for both

Overreaches on the second requirement and gets the version wrong. Ephemeral resources need 1.10; the module targets 1.9.

B Write-only, sensitive, and a version problem

Correct. Different requirements, different mechanisms, and write-only arguments need 1.11 or later, which 1.9 consumers do not have.

C sensitive for both

The running misconception. sensitive controls display only, so the password would still be written to state in plain text.

D Write-only for both

They are not alternatives. The cost centre must be stored, and a write-only argument would discard it, breaking the resource.

THE SIGNAL

Not every secret needs the strongest tool. A value that must be stored but not displayed is exactly what sensitive is for.

Module 13 recap

- **sensitive hides. Ephemeral never writes it down.** Different questions, and a value can be both.
- **1.10 gave ephemeral variables, outputs and resources.** They could only feed other ephemeral contexts.
- **1.11 added write-only arguments**, the door into ordinary managed resources.
- **Ephemeral resources run on every plan and apply**, which is why the version argument exists.
- **The version is stored in state.** Increment it to rotate a write-only value.
- **None of it cleans existing state.** Rotate what was already exposed.

> ON THE EXAM
>
> A Vault data source still lands in state. An ephemeral resource does not. That single swap closes most of the gap.

MODULE 14 · DOMAIN 5 - TERRAFORM MODULES

Modules: Sourcing, Scope, Composition

Where a module comes from, what crosses its boundary, and when not to write one.

DOMAIN 5 of 8 COURSE WEIGHT 10% MODULE 14 of 27

admin@sthithapragna.com

A module is **a directory containing Terraform files**. Nothing more. You have been writing one all along.

Root and child

- The **root module** is the directory where you run Terraform. Every configuration has exactly one.
- A **child module** is any module called by another, through a **module** block.
- Child modules can call further children. The relationships form a tree with the root at the top.
- **Terraform reads only the current directory.** Subdirectories are separate modules, not part of this one.
- So a module is **not** a special file or a keyword. It is a directory, viewed from outside.

> ON THE EXAM
>
> Module eight's point again: Terraform reads every .tf file in one directory and no further. That is what makes a directory a module.

The module block

- **module**, then a name you choose, which becomes part of the address for everything inside.
- **source** is the only required argument. It tells Terraform where to find the code.
- **version** applies to registry sources only. Module fifteen covers it.
- **Every other argument sets one of the module's input variables.** There is no separate inputs block.
- **Changing source requires terraform init again.** Module installation is one of init's three jobs.

> ON THE EXAM
>
> count, for_each, depends_on and providers are also available on a module block. Everything else is an input variable.

Where a module can come from

SOURCE TYPE	NOTES
Local paths	Must begin ./ or ../ . Not downloaded, read in place
Terraform Registry	NAMESPACE/NAME/PROVIDER. Supports the version argument
Private registry	The same, with a hostname in front
GitHub and Bitbucket	Recognised shorthand, no prefix needed
Generic Git and Mercurial	git:: or hg:: prefix, with a ref argument
HTTP URLs, S3 and GCS buckets	Archive files fetched over HTTP or from object storage

ON THE EXAM

Only registry sources support the version argument. Every other source pins by ref, by URL, or not at all.

Local paths

- **A local path must begin with ./ or ../**, which is how Terraform tells it apart from a registry address.
- Write a bare directory name and Terraform will try the **public registry** instead, and fail confusingly.
- **Local paths are not installed.** The files are already on disk, so Terraform just reads them where they are.
- So **no version argument**, and no copy in the modules cache. Whatever is on disk is what runs.
- Edit a local module and the change is live on the next plan, which makes development fast.

> ON THE EXAM
>
> The leading dot-slash is not decoration. Without it the string is a registry address, and that is a whole different lookup.

An absolute path is not a local path

- **Terraform does not treat an absolute filesystem path as a local path.** Not a leading slash, not a drive letter.
- It treats it like a **remote module** and copies it into the local module cache.
- So edits stop taking effect immediately, and **init must be re-run** to pick them up.
- HashiCorp recommends against absolute paths, because they **couple your configuration to one machine's layout**.
- Relative paths are portable. Absolute ones work on your laptop and nowhere else.

> ON THE EXAM
>
> Two things that look like local paths, and only one behaves like one. The relative prefix is what triggers the local behaviour.

Registry and Git sources

- **Public registry:** three parts, namespace, name and target provider, separated by slashes.
- **Private registry:** the same three parts with a hostname in front, making four.
- **GitHub and Bitbucket are detected automatically** from the address. No prefix required.
- **Generic Git needs the git:: prefix**, and should always carry a **ref** argument.
- **Without ref, you get the default branch, whatever it happens to contain today.**

> ON THE EXAM
>
> A Git source with no ref is the module equivalent of a provider with no version constraint. It works until somebody merges something.

Sub-directories, with a double slash

- When a source is a repository or archive, the module may live in a **sub-directory** of it.
- A **double slash** marks where the package ends and the path within it begins.
- **The sub-directory portion must come before any arguments**, such as ref. Order matters.
- Terraform still downloads the whole package, and then reads only that sub-directory.
- HashiCorp recommends putting each reusable module in the **root of its own repository** where possible.

> ON THE EXAM
>
> This is how a monorepo of modules is consumed. One repository, many modules, each addressed after the double slash.

Reading a source address

SOURCE =	WHAT TERRAFORM DOES
`./modules/vpc`	Reads the directory in place. No install
`modules/vpc`	Looks on the public registry, and fails
`/opt/shared/vpc`	Treated as remote. Copied to the module cache
`terraform-aws-modules/vpc/aws`	Public registry. Accepts a version argument
`app.terraform.io/acme/vpc/aws`	Private registry. Also accepts version
`git::https://git.acme.io/tf.git//vpc?ref=v2.1`	Git, sub-directory vpc, pinned to tag v2.1

ON THE EXAM

The last row shows the full ordering: package, then double slash and path, then the ref argument.

HashiCorp's guidance on choosing

- **Local file paths** for closely related modules, used mainly to factor out repeated code in one repository.
- **A native module registry** for modules intended to be shared by multiple calling configurations.
- Other sources exist so you can distribute modules using **infrastructure you already have**.
- Local paths across repository boundaries are fragile, because they depend on where somebody cloned things.
- The deciding question: **is this shared beyond this repository?**

> ON THE EXAM
>
> Factoring out repetition within one repository, or distributing to other teams. Those are different problems with different answers.

Interacting with inputs and outputs

- A caller sets inputs as **arguments in the module block**, matching the child's variable names.
- A caller reads outputs as **module, the module name, then the output name**.
- **Outputs chain.** One module's output becomes another module's input, which is how flat composition works.
- **A missing required input is an error at plan time**, naming the module and the variable.
- A module's **variable descriptions and output descriptions** are its documentation. The registry publishes them.

> ON THE EXAM
>
> Chaining outputs into inputs creates the dependency between two modules, exactly as a resource reference does.

Scope: what crosses the boundary

- **Nothing is shared implicitly.** A module cannot see the caller's variables, locals or resources.
- **In: module block arguments**, which set the child's input variables. That is the only route.
- **Out: declared outputs.** Undeclared internals are unreachable, as module eight established.
- **Provider configurations are the exception.** Default ones are inherited automatically.
- **Aliased provider configurations are not inherited** and must be passed with the providers meta-argument.

> ON THE EXAM
>
> One exception to 'nothing is shared', and it is providers. That asymmetry is examinable and easy to forget.

Composition

- **Flat composition:** a root module calls several children and wires them together.
- **Deep nesting** makes values travel through layers of pass-through variables.
- Each layer must **declare, accept and forward** every value.
- Flatter trees are easier to read and produce shorter resource addresses.
- **Deep nesting also makes plan output harder to read**, as addresses grow.

> ON THE EXAM
>
> The cost of nesting is not performance. It is that every layer becomes a place where a value must be declared and forwarded.

The standard module structure

- **main.tf, variables.tf, outputs.tf** at the root of the module. Convention, not requirement.
- A **README** describing what it does, and a **LICENSE** if it is published.
- An **examples** directory showing how to call it. These are separate root modules, not part of the module.
- A **modules** subdirectory for nested modules the main one uses.
- **The registry reads this structure** to generate documentation automatically.

> ON THE EXAM
>
> Filenames still carry no meaning to Terraform. They carry meaning to the registry, to tooling, and to the next person.

When not to write a module

- **A thin wrapper around a single resource** adds a layer and removes nothing.
- If every variable is passed straight through, the module is **a rename, not an abstraction**.
- It also **constrains callers**, who can now only use the arguments you chose to expose.
- A module earns its place when it **encapsulates a decision**: several resources wired together correctly.
- The test: does calling this module save the caller from knowing something they would otherwise need to know?

> ON THE EXAM
>
> Modules are for encapsulating decisions, not for hiding resource types. A wrapper with no opinion is overhead with a version number.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Edits to the module are not taking effect	An absolute path, cached like a remote module
Terraform looked for it on the registry	A local path missing its ./ prefix
Shared across several repositories	A registry, not a relative path
Different results on different machines	A Git source with no ref argument
The module lives in a monorepo folder	A double slash before the ref argument
Every variable is passed straight through	A thin wrapper. Probably should not be a module

ON THE EXAM

Most sourcing questions are decided by one character: the leading dot, the double slash, or the missing ref.

> A developer is building a new networking module. Her root configuration references it with a source of /home/dev/work/modules/networking, an absolute path on her machine.
>
> She edits the module, reruns plan, and sees no change. Running init again picks the edit up. She expected local modules to update immediately.

What is happening?

A. Local modules are cached in state and require -refresh to update

B. An absolute path is not treated as a local path. Terraform copies it into the module cache

C. The module needs a version argument before edits are detected

D. Plan does not read modules. Only apply reinstalls them

◆ PAUSE AND DECIDE

What is happening?

A Cached in state

Modules are installed into the .terraform directory, not state. State records infrastructure bindings, as module four established.

B Absolute is not local

Correct. HashiCorp documents that an absolute path is treated similarly to a remote module and copied into the local module cache.

C Needs a version

The version argument applies to registry sources only, and would be rejected here. It has nothing to do with detecting edits.

D Only apply reinstalls

Neither plan nor apply installs modules. Installation is one of init's three jobs, which is exactly why init fixed it.

THE SIGNAL

Change the source to a relative path and the fast edit-and-plan loop returns. One character of prefix decides the behaviour.

> Nine teams across four repositories all deploy a similar service. Each repository has copied a set of module directories and references them with relative paths such as ../../shared/service.
>
> Fixes are applied inconsistently, and two repositories have diverged. The platform team wants one authoritative copy with controlled adoption.

What should they do?

A. Move the modules to one repository and reference them by absolute path from each consumer

B. Publish the modules to a private registry and reference them with a version constraint

C. Keep relative paths but add a script that copies the canonical module into each repository

D. Combine all four repositories into a monorepo so relative paths resolve

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should they do?

A Absolute paths

Requires every machine and every runner to have the same filesystem layout, which is precisely what HashiCorp warns against.

B A private registry

Correct. This is the documented purpose of a registry: modules shared by multiple calling configurations, with versioning and controlled adoption.

C A copying script

Automates the divergence rather than removing it. Every repository still holds its own copy, now with a script to maintain as well.

D Merge into a monorepo

Restructures four teams' repositories to solve a distribution problem. It also removes their ability to adopt changes independently.

THE SIGNAL

The phrase 'controlled adoption' is the tell. Only a versioned source lets a team choose when to take a change.

> A configuration sources a module from a generic Git repository with a git:: prefix and no ref argument. It has worked for a year.
>
> A pipeline run in a fresh working directory now produces a plan proposing changes nobody made. The same configuration on an engineer's laptop, initialised months ago, plans no changes.

What is the cause, and the fix?

A The dependency lock file was not committed, so module versions diverged

B With no ref, each init takes the default branch as it is then. Pin with a ref to a tag

C The laptop's state is stale and should be refreshed

D Git sources require terraform init -upgrade on every run to stay consistent

◆ PAUSE AND DECIDE

What is the cause, and the fix?

A Missing lock file

The lock file tracks providers only, as module three established. It has never recorded module versions, so committing it would change nothing here.

B No ref, so the branch moved

Correct. Without a ref, init takes the default branch as it stands at that moment. The two environments installed at different times.

C Stale state on the laptop

Both environments share the same remote state. What differs is the module code each has installed, which state knows nothing about.

D Always use -upgrade

Would guarantee everyone tracks the moving branch, making the problem universal rather than intermittent.

THE SIGNAL

Module three's finding, arriving as an incident. Providers are locked and modules never were, so an unpinned module is genuinely unpinned.

> A team has written a module containing exactly one storage bucket resource. It declares eleven input variables, each passed straight through to the corresponding resource argument, and one output returning the bucket name.
>
> A reviewer is asked whether this should be published to the private registry.

What is the strongest response?

A. Yes. Every resource should be wrapped in a module for consistency

B. Probably not. It adds a layer and a release cycle while encapsulating no decision

C. Yes, but only after adding count so it can be made conditional

D. No. Modules containing a single resource are rejected by the registry

◆ PAUSE AND DECIDE

What is the strongest response?

A Wrap everything

Consistency for its own sake. It also restricts callers to the eleven arguments chosen, where the resource offers many more.

B It encapsulates nothing

Correct. Eleven pass-through variables is a rename, not an abstraction. A module should save the caller from knowing something.

C Add count first

Improves a module that should not exist. Conditional creation works equally well on the resource directly.

D The registry rejects it

Invents a rule. Single-resource modules are perfectly valid and sometimes correct, when they encode defaults or policy.

THE SIGNAL

A single-resource module is fine when it encodes an opinion: enforced encryption, naming, logging. Pure pass-through encodes nothing.

Module 14 recap

- **A module is a directory of Terraform files.** The root module is one, and every configuration has it.
- **Local paths must begin ./ or ../**, and are read in place rather than installed.
- **An absolute path is not a local path.** It is cached like a remote module.
- **Local paths for factoring out repetition; a registry for sharing across configurations.**
- **Nothing crosses the boundary except inputs and outputs**, apart from default provider configurations.
- **A Git source with no ref is unpinned**, and the lock file will not save you.

> ON THE EXAM
>
> A module should encapsulate a decision. Eleven pass-through variables around one resource encapsulates nothing.

MODULE 15 · DOMAIN 5 - TERRAFORM MODULES

Module Versioning and the Registry

The version argument, publishing requirements, and why module pinning matters more than provider pinning.

DOMAIN 5 of 8 COURSE WEIGHT 10% MODULE 15 of 27

admin@sthithapragna.com

Providers are **constrained and then locked**. Modules are **constrained only**. The constraint is all you get.

The version argument

- **version** goes in the module block, alongside source. It uses the same operators as providers.
- **It works only with registry sources**, public or private. Nothing else accepts it.
- Put version on a Git or local source and Terraform **errors**. Those pin by ref, or not at all.
- Recall the operators from module three: equals, not equals, the comparisons, and the pessimistic form.
- Constraints from every module are combined, and Terraform selects **the newest release satisfying all of them**.

> ON THE EXAM
>
> Registry sources support version. Git sources support ref. Local paths support neither, because there is nothing to select from.

Which pin does which source take?

SOURCE	HOW YOU PIN IT
Public registry	version argument. Full operator set
Private registry	version argument. Identical behaviour
Generic Git	ref argument, naming a tag, branch or commit
GitHub or Bitbucket shorthand	ref argument, same as generic Git
HTTP, S3 or GCS archive	The URL itself. Point it at a versioned artefact
Local path	Nothing. Whatever is on disk is what runs

ON THE EXAM

Only the first two accept version. Everything else pins by naming one specific thing, or does not pin at all.

The consequence of no lock file

- **Terraform never records which module version it selected.** There is no module entry in the lock file.
- So on **every init**, it re-resolves and takes the newest release that satisfies your constraint.
- Two machines initialising on different days can therefore run **different module code**.
- HashiCorp's own remedy: **use an exact version constraint** if you need the same version every time.
- **A pessimistic constraint is not a pin.** It still allows the rightmost component to move.

> ON THE EXAM
>
> This is module three's finding, restated as advice. Providers are reproducible by default; modules are reproducible only if you pin them.

How strict should a module pin be?

EXACT VERSION

Reproducible, and manual

- Every environment runs the same code
- Upgrades are a deliberate commit
- Diffs show exactly what changed
- You must remember to move forward

PESSIMISTIC CONSTRAINT

Convenient, and drifting

- Patches arrive automatically
- No commit needed to take a fix
- Two environments may differ
- A plan can change with no code change

THE TIEBREAKER

The trade is different from providers, because there is no lock file underneath to make the loose option safe.

init, and moving a module version

- **terraform init installs modules** into the .terraform directory. One of its three jobs, from module five.
- **Changing a source or a version requires init again.** Plan will stop and tell you so.
- **terraform init -upgrade** re-resolves modules as well as providers against current constraints.
- **terraform get** installs modules only, without touching the backend or providers.
- Plain init **does not** reinstall a module that is already present and still satisfies the constraint.

> ON THE EXAM
>
> -upgrade means both things at once: newest matching providers, and newest matching modules. It is not provider-only.

Publishing to the public registry

- **GitHub, and the repository must be public.** This applies to the public registry only.
- **Named terraform, then the provider, then the name.** Three parts, separated by hyphens.
- **A repository description**, which becomes the module's one-sentence summary.
- **The standard module structure**, so the registry can inspect it and generate documentation.
- **Release tags in x.y.z form**, optionally prefixed with a v. At least one is needed to publish.

> ON THE EXAM
>
> Five requirements, and the naming one catches people. The name segment may contain further hyphens; the first two parts may not.

Releasing a new version

- **Push a properly formed Git tag.** That is the entire release process for a public module.
- The registry notices the tag and publishes that version. No separate upload step exists.
- **Tags that do not parse as a semantic version are ignored**, silently.
- So a release named release-2024-q3 will never appear, and nothing will tell you why.
- Versions are immutable once published. Fixing a mistake means **publishing a new version**.

> ON THE EXAM
>
> Publishing initially is a one-off setup. After that, a version is a tag, which is why tag discipline becomes release discipline.

What the registry gives you back

- **Generated documentation** from your variables, outputs and README. Descriptions become the docs.
- **A browsable version history**, and a usage example you can copy.
- **Inputs, outputs, dependencies and provider requirements**, all parsed from the source.
- **Submodules and examples are parsed too**, which is why the standard structure is a requirement.
- All of it is available through the **UI and the API**.

> ON THE EXAM
>
> The structure requirement is not bureaucracy. The registry reads those directories to produce everything on this slide.

The private registry

- HCP Terraform and Terraform Enterprise both provide a **private module registry** for an organisation.
- **The public GitHub requirement does not apply.** That one is specific to the public registry.
- Publishing from version control with tag-based releases still requires **x.y.z release tags**.
- The address gains a hostname, so a private module reference has **four parts instead of three**.
- Modules can be marked **no-code ready**, for provisioning without writing configuration. Module twenty-three.

> ON THE EXAM
>
> Private registry modules use the version argument exactly as public ones do. Only the address and the hosting differ.

The advantage people miss

- **The private registry downloads and stores the module source code itself.**
- It serves downloads and controls access using **HCP Terraform API tokens**.
- **So consumers do not need access to the module's source repository**, even running from the CLI.
- With a Git source, every consumer needs credentials for that repository. Every pipeline too.
- That difference is often the real reason to move from Git sources to a private registry.

> ON THE EXAM
>
> Versioning is the obvious benefit. Decoupling module consumption from source repository permissions is the quieter one.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Terraform rejected the version argument	The source is not a registry. Use ref instead
The same config gave different results	An unpinned module, re-resolved on init
Every environment must run identical code	An exact version constraint. There is no lock file
The module never appeared in the registry	Naming, or a tag that is not a semantic version
Consumers need repo credentials to init	A Git source. A private registry removes that
A fix was published but nobody picked it up	An exact pin, working as intended

ON THE EXAM

Half these questions are decided by remembering there is no module lock file. The other half by the five publishing requirements.

> A regulated team must be able to demonstrate that the code deployed to production is byte-for-byte the code that was reviewed. They pin every provider with a pessimistic constraint and commit the dependency lock file.
>
> They use registry modules with the same pessimistic constraints, reasoning that the approach worked for providers.

What is wrong with this, and what should change?

A. Nothing. The lock file records module selections alongside providers

B. Modules are not locked. Use exact version constraints for the modules

C. Switch the modules to Git sources, which are locked by commit hash

D. Run terraform init -upgrade before every apply so all environments match

◆ PAUSE AND DECIDE

What is wrong with this, and what should change?

A The lock file covers modules

It does not, and never has. HashiCorp states the lock file tracks only provider dependencies.

B Pin modules exactly

Correct. With no lock file underneath, an exact constraint is the only mechanism that guarantees the same module version every time.

C Use Git sources

A ref can name a commit, so this can work. It also loses the version argument and the registry, to solve something an exact pin solves already.

D -upgrade everywhere

The opposite of what they need. It re-resolves to the newest matching release every time, maximising the drift they are trying to eliminate.

THE SIGNAL

The reasoning that failed them is the reasonable one: what worked for providers should work for modules. The lock file is the difference.

> An engineer prepares a module for the public registry. The repository is public on GitHub, follows the standard module structure, and has a description. She tags the release as release-2026-03 and clicks publish.
>
> The registry accepts the repository but shows no available versions.

What is the problem?

A. The repository must be named with a terraform prefix, provider and name

B. Release tags must be a semantic version, optionally prefixed with v. Other tags are ignored

C. A module needs at least three released versions before any are shown

D. The publish step requires a signed tag for verification

◆ PAUSE AND DECIDE

What is the problem?

A Naming

A real requirement, and ruled out here: the registry accepted the repository, which it would not have done had the name been wrong.

B The tag is not a version

Correct. The registry identifies versions from tags in x.y.z form, optionally prefixed with v. Anything else is ignored silently.

C Three versions needed

Invents a rule. One release tag is enough, and at least one is required to publish initially.

D Signed tags

Provider releases involve signing. Module publishing has no such requirement.

THE SIGNAL

The registry accepted the repository and found no versions. That split tells you the naming passed and the tags did not.

> A platform team distributes modules from a private Git repository. Every consuming team needs a deploy key, and every pipeline needs credentials for that repository. Onboarding a new team takes two days, mostly spent on access requests.
>
> They already use HCP Terraform for their runs.

What would most reduce the onboarding burden?

A. Make the module repository readable by everyone in the organisation

B. Publish the modules to the HCP Terraform private registry

C. Vendor the modules into each consuming repository with a sync job

D. Move to local path sources so no credentials are needed

◆ PAUSE AND DECIDE

What would most reduce the onboarding burden?

A. Open the repository

Reduces the requests and grants write-adjacent visibility to everyone, and still requires each pipeline to hold VCS credentials.

B. Use the private registry

Correct. The registry stores the source and serves downloads via API tokens, so consumers need no access to the module repository at all.

C. Vendor with a sync job

Module fourteen's divergence problem, deliberately reintroduced. Every consumer holds a copy that can drift.

D. Local path sources

Local paths cannot cross repository boundaries in any reliable way, and they support no version argument at all.

THE SIGNAL

The registry decouples module consumption from source repository permissions. That is often the real reason teams adopt it.

> A configuration sources a module from a generic Git repository with a ref argument pinning it to a tag. An engineer adds a version argument set to a pessimistic constraint, wanting the same behaviour registry modules have.
>
> terraform init fails.

Why, and what achieves the intent?

A The version argument works only with registry sources. Change the ref, or move to a registry

B The version and ref arguments conflict. Remove ref and keep version

C Git sources need a version_constraint argument rather than version

D The constraint syntax differs for modules; only exact versions are allowed

◆ PAUSE AND DECIDE

Why, and what achieves the intent?

A Registry sources only

Correct. A registry publishes a list of versions to choose among; a Git URL does not. Achieving range-based selection means publishing to a registry.

B They conflict

Implies version would work alone on a Git source. It would not. The source type is what makes it invalid, not the combination.

C version_constraint

Invents an argument. There is no such thing on a module block.

D Only exact versions

Backwards. Registry modules accept the full operator set; the problem is that this source is not a registry.

THE SIGNAL

A constraint needs a set of candidates to choose from. A registry publishes one; a Git ref names a single commit and nothing else.

Module 15 recap

- **version works with registry sources only.** Git pins by ref; local paths pin not at all.
- **Modules are never recorded in the lock file**, so a constraint is all the pinning you get.
- **Use an exact version** where reproducibility matters. That is HashiCorp's own advice.
- **Five publishing requirements:** public GitHub, three-part name, description, standard structure, x.y.z tags.
- **A release is a tag.** Tags that are not semantic versions are ignored silently.
- **The private registry stores the source**, so consumers need no repository access.

> ON THE EXAM
>
> terraform init -upgrade re-resolves modules as well as providers. It is not a provider-only flag.

MODULE 16 · DOMAIN 6 - TERRAFORM STATE MANAGEMENT

Backends and State Locking

Where state lives, how two people are stopped from applying at once, and one pattern nearly every guide still teaches wrong.

DOMAIN 6 of 8 COURSE WEIGHT 12% MODULE 16 of 27

admin@sthithapragna.com

The backend is resolved **before Terraform evaluates anything else**. Every limitation on it follows from that.

The default backend

- **With no backend block, Terraform uses the local backend.** There is no such thing as no backend.
- It writes **terraform.tfstate** in the working directory, with a backup of the previous version beside it.
- It is a real backend with real behaviour, including **locking, using a lock file on disk**.
- What it cannot do is **be shared**. That is module four's syncing argument, and it is the reason teams move.
- It remains correct for experiments, for learning, and for anything genuinely single-user.

> ON THE EXAM
>
> Local state is still state. A question offering 'no backend' as an option is describing the local backend by another name.

The backend block

- A nested **backend** block inside the top-level terraform block, labelled with the backend type.
- Its arguments are **specific to the type**, describing where and how state is stored.
- **A configuration can provide only one backend block.** One configuration, one place state lives.
- The backend must be **available in the Terraform version you are running**. Backends are built in, not plugins.
- **Changing it requires terraform init again**, which is init's first responsibility from module five.

> ON THE EXAM
>
> Backends are built into Terraform, unlike providers. There is no registry of backends and no version argument for one.

Backend types worth recognising

BACKEND	NOTES
`local`	The default. State on disk, locked with a file
`s3`	Object storage plus native locking via use_lockfile
`azurerm`	Azure Blob Storage. Locking via blob leases
`gcs`	Google Cloud Storage. Locking supported
`kubernetes`	State in a Kubernetes secret
`http`	A generic REST endpoint you implement yourself

ON THE EXAM

You do not need to memorise every backend. You do need to know locking is per-backend, and that local is the default.

No variables in a backend block

- **A backend block cannot refer to named values.** Not input variables, not locals, not data source attributes.
- The reason is the opening slide: **the backend is resolved before those things exist**.
- **Fourth appearance of one principle** in this course: what the engine needs early cannot come from what runs late.
- The others were **plan-time keys**, **dynamic blocks and meta-arguments**, and **literal-only lifecycle settings**.
- The supported answer is **partial configuration**, which is the next slide.

> ON THE EXAM
>
> Attempting it produces a clear error saying variables may not be used here. Recognise the cause rather than the message.

Partial configuration

- **Omit some arguments from the backend block** and supply them at initialisation instead.
- **-backend-config=FILE** reads a file of settings, conventionally with an .hcl extension.
- **-backend-config="key=value"** sets one argument. Repeat it for several.
- This is how **one configuration serves several environments**, each with its own backend settings file.
- Terraform **merges** the values from all sources during init.

> ON THE EXAM
>
> An empty backend block plus a per-environment settings file is the standard pattern. The block declares the type; the file supplies the details.

Where backend credentials leak to

- **Terraform writes the backend configuration in plain text, in two places.**
- **.terraform/terraform.tfstate** holds the backend configuration for the working directory.
- **Every saved plan file captures it too**, so the plan applies against the right infrastructure.
- So HashiCorp recommends supplying credentials through **environment variables**, not the block and not -backend-config.
- Better still: leave credential arguments **completely unset** and use the target system's own conventions.

> ON THE EXAM
>
> Module five said plan files are sensitive artefacts. This is a second reason why, and it is one people rarely think about.

A saved plan carries its backend with it

- **Applying a saved plan uses the backend configuration stored in that file**, not your current settings.
- That is deliberate, and it is what stops a plan being applied against the wrong infrastructure.
- **If that configuration held time-limited credentials, they may have expired** by the time you apply.
- Another reason the two-step workflow suits automation, where plan and apply happen close together.
- Module five's staleness point again, from a different direction.

> ON THE EXAM
>
> The plan file is more self-contained than people expect. It carries configuration, values, and where the state lives.

State locking

- **Terraform locks state for any operation that could write to it.** Automatically, with no flag required.
- The purpose: stop two applies running at once and **corrupting state** or losing one person's changes.
- **Not every backend supports locking.** It is a per-backend capability, and a real selection criterion.
- If a lock is held, Terraform reports it and says **who holds it and since when**.
- Locking is not the same as access control. It prevents collisions, not unauthorised changes.

> ON THE EXAM
>
> Locking happens by default where supported. A question describing a lock being 'enabled' is usually describing choosing a backend that supports it.

The locking flags, and force-unlock

- **-lock-timeout=DURATION** waits for a held lock instead of failing immediately. Sensible in a pipeline.
- **-lock=false** disables locking entirely. Almost never the right answer.
- **terraform force-unlock** removes a lock that was left behind, for example by a crashed run.
- **It requires the lock ID**, which is deliberate friction. You cannot force-unlock by accident.
- **Never force-unlock a lock held by a running operation.** That is how you get two writers and a corrupted state.

> ON THE EXAM
>
> force-unlock is a recovery tool for an abandoned lock. If somebody's apply is still running, the lock is doing its job.

The S3 backend, and the pattern guides still teach

- **Almost every guide says: an S3 bucket for state, plus a DynamoDB table for locking.**
- **DynamoDB-based locking is deprecated**, and the dynamodb_table argument is marked as such.
- **The current mechanism is S3 native locking: use_lockfile = true.**
- It writes a **.tflock** object beside the state file in the same bucket. No second service.
- Both can be configured together during migration, so you are not forced to switch in one step.

> ON THE EXAM
>
> This is the most reliably out-of-date thing in Terraform study material. A runbook naming DynamoDB is not wrong, it is old.

The rest of a sensible S3 backend

- **bucket, key and region** are the core arguments. The key is the path to the state object.
- **Enable versioning on the bucket.** Module four called this the real protection against losing state.
- **Enable encryption**, and restrict who can read the bucket. State is sensitive data, as ever.
- **Do not put credentials in the block.** Use the platform's own credential chain or environment variables.
- Give each configuration a **distinct key**, so two configurations never share one state object.

> ON THE EXAM
>
> Versioning, encryption and access control are all still required. Native locking replaced one component, not the whole design.

The cloud block

- **A cloud block configures the HCP Terraform integration**, and replaces the older remote backend.
- **A configuration with a cloud block cannot also have a backend block.** They are alternatives.
- With HCP Terraform you **do not configure a backend at all**, because it manages state in its workspaces.
- **The cloud block does not support interpolation either**, and supports partial configuration in the same way.
- It also enables **remote operations**, where plan and apply run on HCP Terraform. Module twenty-one.

> ON THE EXAM
>
> backend and cloud are mutually exclusive. A configuration has one or the other, never both.

-migrate-state or -reconfigure?

-MIGRATE-STATE

Bring the state with you

- Reinitialise, and copy existing state across
- Terraform prompts before copying
- The normal answer when moving backends
- Your resources stay managed

-RECONFIGURE

Start fresh here

- Reinitialise, discarding existing configuration
- No state is copied at all
- For when the new backend already has the state
- Or when the old one was wrong from the start

THE TIEBREAKER

Choosing -reconfigure when you meant -migrate-state leaves the new backend empty, and Terraform will plan to create everything.

Before you migrate anything

- **HashiCorp strongly recommends manually backing up state first.**
- That advice is in the documentation. Migration touches the one unrecreatable thing.
- Terraform will **offer to migrate**, and prompts before acting.
- **-input=false suppresses that prompt**, so automation migrates silently.
- Which argues for doing backend migrations **by hand**, deliberately.

> ON THE EXAM
>
> Backing up state before migrating is documented advice, not caution for its own sake. State is the one artefact with no source.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Variables may not be used here	A backend block. Use partial configuration
One config, several environments	An empty backend block plus per-environment .hcl files
The runbook says create a DynamoDB table	Deprecated. use_lockfile on the S3 backend
Credentials appeared in a build artefact	Backend config is captured in .terraform and plan files
The new backend is empty after moving	-reconfigure was used where -migrate-state was meant
A lock is held by a crashed run	force-unlock, with the lock ID

ON THE EXAM

Most of this domain is decided by remembering that the backend is resolved first, before anything else exists.

> A platform team is standing up state storage for a new account. Their internal runbook, written three years ago, specifies an S3 bucket with versioning and encryption, plus a DynamoDB table for state locking.
>
> An engineer follows it and it works. During review, a colleague says part of it is out of date.

Which part, and what replaces it?

A Bucket versioning is deprecated in favour of state snapshots

B DynamoDB locking is deprecated. Use use_lockfile for native S3 locking

C The S3 backend is deprecated entirely in favour of the cloud block

D Nothing is out of date. The colleague is describing HCP Terraform, not the S3 backend

◆ PAUSE AND DECIDE

Which part, and what replaces it?

A Versioning deprecated

The opposite. Versioning is still recommended, and module four called it the real protection against losing state.

B DynamoDB, replaced by use_lockfile

Correct. The dynamodb_table argument is deprecated and slated for removal. Native locking writes a .tflock object beside the state.

C S3 backend deprecated

Not deprecated and widely used. HCP Terraform is an alternative, not a replacement, and choosing it is a different decision.

D Nothing is out of date

The runbook still works, which is why this survives. Working and current are different things, and the argument is marked deprecated.

THE SIGNAL

The runbook is not wrong. It is old, and it still functions, which is exactly why nobody has revisited it.

A team maintains one configuration deployed to development, staging and production, each with its own state bucket in its own account. An engineer adds a variable for the bucket name and references it from the backend block.

terraform init fails, reporting that variables may not be used here.

What is the correct approach?

A Move the backend block into a separate file so it is parsed after variables

B Use partial configuration: an empty backend block, with values supplied at init

C Define the bucket name as a local value, which backends do accept

D Maintain three copies of the configuration, one per environment

◆ PAUSE AND DECIDE

What is the correct approach?

A **A separate file**

Filenames carry no meaning, as module eight established, and Terraform reads all .tf files as one configuration. Ordering is not affected.

B **Partial configuration**

Correct, and it is the documented mechanism. The block names the type; a per-environment .hcl file supplies bucket, key and region at init.

C **Use a local**

Locals are named values too, and the restriction covers input variables, locals and data source attributes alike.

D **Three copies**

Solves it by duplicating everything, reintroducing exactly the divergence problem module fourteen described.

THE SIGNAL

The restriction is about evaluation order, not syntax, so nothing about where or how you write it will change the outcome.

A pipeline runs terraform init with -backend-config arguments that include an access key and secret for the state bucket. It then runs plan with -out and publishes the plan file for review.

A security scan flags credentials in the build artefacts. The engineer insists nothing writes credentials to disk.

What is happening?

A The scanner is matching the credentials from the pipeline log, not the artefacts

B Terraform writes backend configuration in plain text into .terraform and into every plan file

C -backend-config values are encrypted, so this must be a false positive

D The state file contains the credentials, and the plan file references state

◆ PAUSE AND DECIDE

What is happening?

A It is the log

Plausible and does not fit. The scan flagged the artefacts specifically, and the credentials genuinely are in them.

B Written to both, in plain text

Correct. HashiCorp documents this exactly, and warns that -backend-config or hardcoded values can leak credentials this way.

C They are encrypted

Nothing encrypts them. The documentation says plain text, in two separate files.

D Via the state file

State holds resource attributes, not backend credentials. The plan file captures the backend config directly, which is the real route.

THE SIGNAL

Module five said plan files are sensitive artefacts because of resource values. This is a second, separate reason.

> A team moves state from one S3 bucket to another in a different account. They update the backend block and run terraform init. Terraform asks a question they do not read carefully, and they re-run it with -reconfigure to get past the prompt.
>
> The next plan proposes to create every resource in the configuration. Nothing has been destroyed.

What happened, and what should they do?

A The resources were destroyed by the migration. Restore from the old bucket

B -reconfigure discarded the old configuration without copying state. Re-init with -migrate-state, or copy the state across

C The new bucket needs versioning enabled before Terraform can read state

D Run terraform apply. Terraform will detect the existing resources and adopt them

◆ PAUSE AND DECIDE

What happened, and what should they do?

A They were destroyed

The scenario says nothing was destroyed, and migration never destroys infrastructure. Only an apply changes anything.

B -reconfigure copied nothing

Correct. It discards the existing backend configuration and starts fresh, so the new bucket is empty and Terraform sees nothing as managed.

C Versioning first

Versioning protects against losing state. It is not a precondition for reading it, and the new bucket simply has no state in it.

D Apply will adopt them

The most dangerous option here. Terraform never adopts existing objects, as module four and module seven both established. It would create duplicates.

THE SIGNAL

The infrastructure is fine and untouched. What was lost is Terraform's knowledge of it, which is module four's point about losing state.

Module 16 recap

- **No backend block means the local backend.** Local state is still state.
- **One backend block per configuration, and no named values in it.** Use partial configuration.
- **Backend config is written in plain text** to .terraform and to every plan file. Keep credentials out.
- **Locking is automatic where the backend supports it.** force-unlock needs the lock ID, and is for abandoned locks.
- **S3 uses native locking now.** use_lockfile writes a .tflock object; dynamodb_table is deprecated.
- **-migrate-state brings state with you. -reconfigure does not.**

> ON THE EXAM
>
> cloud and backend are mutually exclusive. With HCP Terraform you configure no backend at all.

MODULE 17 · DOMAIN 6 - TERRAFORM STATE MANAGEMENT

Drift, Refresh-Only, and the Refactoring Blocks

When reality moves, when you move things, and the blocks that put state changes into code.

DOMAIN 6 of 8 COURSE WEIGHT 12% MODULE 17 of 27

admin@sthithapragna.com

The refactoring blocks move state manipulation into your configuration, where a plan describes it and review catches it.

What drift is

- **Drift is the difference between state and reality.** Somebody changed something outside Terraform.
- **It is not the difference between configuration and state.** That is an ordinary plan.
- Terraform notices drift during the **refresh** that precedes a normal plan.
- It reports it separately, under **changes made outside of Terraform**, as module six described.
- Causes are usually mundane: an incident fix, a console click, or another tool with write access.

> ON THE EXAM
>
> Module four said a plan has three inputs. Drift is the gap between two of them, and the plan is the gap between the other two.

Two comparisons, two meanings

CONFIGURATION AGAINST STATE

The plan

- What you asked for, versus what exists
- Resolved by applying
- Shown as creates, updates, destroys
- This is the normal workflow

STATE AGAINST REALITY

The drift

- What Terraform believed, versus what is true
- Resolved by refreshing, or by reverting
- Shown under 'changes outside of Terraform'
- This is what refresh-only mode isolates

THE TIEBREAKER

A plan showing no changes with a long drift section is not a quiet day. It means reality moved and nothing has decided what to do.

Refresh-only mode

- **terraform plan -refresh-only** compares state against reality and proposes **no infrastructure changes**.
- **terraform apply -refresh-only** writes those findings into state, after you approve them.
- It is **unaffected by code changes**, so it purely detects manual modification.
- **It does not fix anything in your infrastructure.** It updates Terraform's belief to match what is true.
- It replaced the deprecated **terraform refresh** command, which did this with no preview and no prompt.

> ON THE EXAM
>
> Refresh-only accepts reality. A normal apply overrules it. Those are opposite responses to the same finding.

Three responses to drift

- **Revert it.** Run a normal apply. Terraform changes reality back to match your configuration.
- **Adopt it.** Run apply with -refresh-only. State accepts what happened, and your configuration is now out of date.
- **Codify it.** Edit the configuration to describe the new reality, then apply normally. The plan should be empty.
- **Adopting alone leaves configuration and reality disagreeing**, so the next normal apply reverts it anyway.
- So adopt and codify usually go together. Accept the change, then write it down.

> ON THE EXAM
>
> Only a human can decide which of these is right. Everything else about drift handling can be automated.

Detecting drift on a schedule

- A scheduled **plan -refresh-only** is a clean drift detector. It cannot propose infrastructure changes.
- Pair it with **-detailed-exitcode**: zero means no drift, two means drift was found.
- That gives a pipeline something to branch on, from module five's automation flags.
- The only step needing a human is **deciding which of the three responses applies**.
- HCP Terraform automates the detection side as **drift detection** and **health assessments**. Module twenty-three.

> ON THE EXAM
>
> Refresh-only plus detailed exit code is the whole detector. It is safe to schedule because it cannot change infrastructure.

The refactoring blocks

- **moved**, from Terraform 1.1. An object kept its identity but changed address.
- **import**, from Terraform 1.5. An existing object should come under management. Module nineteen.
- **removed**, from Terraform 1.7. An object should leave state, with or without being destroyed.
- All three are **declarative equivalents of state CLI commands**, and all three appear in the plan.
- That means they go through **code review**, which is the whole reason they exist.

> ON THE EXAM
>
> Three blocks, three verbs: rename, adopt, forget. Each has a CLI counterpart that does the same thing without a preview.

The three blocks, and their CLI equivalents

BLOCK	THE COMMAND IT REPLACES
moved (1.1)	terraform state mv
import (1.5)	terraform import
removed, destroy = false (1.7)	terraform state rm
removed, destroy = true (1.7)	Deleting the resource block and applying
No block for it	terraform state list, show, pull, push

ON THE EXAM

The last row matters: inspection has no declarative equivalent, which is why the state commands are still needed. Module eighteen.

The moved block

- **moved** with a **from** and a **to** address. Terraform updates state instead of destroying and recreating.
- Covers renaming a resource, moving one into a module, and **changing between count and for_each**.
- The plan shows a **move** rather than a create and a destroy, and no infrastructure is touched.
- **Without it, a rename is a destroy and a create.** Terraform compares addresses, not intentions.
- Module ten's migration problem, solved: nine databases renamed rather than rebuilt.

> ON THE EXAM
>
> This is the answer whenever a purely cosmetic refactor produces a destructive plan. The addresses changed, so tell Terraform they moved.

How long to keep a moved block

- **Delete it too early and you strand anyone who has not applied yet.** Their state still has the old address.
- Keep it until **every environment and every workspace** has applied at least once.
- HashiCorp's position is that it is **fine to keep moved blocks indefinitely**.
- **Especially in a shared module**, where you cannot know when every consumer has caught up.
- A moved block whose from address is already gone is simply a no-op.

> ON THE EXAM
>
> In a published module, keeping the moved block is the safe default. You have no visibility of who is still on the old version.

The removed block

- **removed** with a **from** address. It instructs Terraform to remove that resource from state.
- **You must delete the resource block first.** A removed block replaces it, rather than sitting alongside it.
- It works on a **module address** too, removing everything that module created.
- The plan says the object **will no longer be managed by Terraform**, and the summary counts stay at zero.
- Applying it changes **state**, even though it reports no infrastructure actions.

> ON THE EXAM
>
> A plan reporting zero to add, zero to change and zero to destroy is still a plan that changes state. Read the sentences, not just the summary.

The argument that decides everything

- **A removed block requires a nested lifecycle block.** Omitting it is a validation error.
- **destroy = true removes it from state and destroys the real infrastructure.**
- **destroy = false removes it from state and leaves the object alone.**
- That is the difference between a safe refactor and a real deletion, in one boolean.
- The documented use for false: **handing management to another tool or team**.

> ON THE EXAM
>
> Read the destroy argument before approving any plan containing a removed block. Nothing else in the block changes the outcome.

moved or removed?

MOVED

Same object, new address

- Terraform keeps managing it
- State is updated, nothing is touched
- For renames and restructuring
- Keep the block for a long time

REMOVED

Terraform stops managing it

- The resource block must be deleted
- State loses the binding
- For handing over, or splitting apart
- Delete the block once it has propagated

THE TIEBREAKER

moved keeps a binding and changes its name. removed deletes the binding. Only one of them can also delete infrastructure.

A word on prevent_destroy

- Module eleven established that **prevent_destroy dies with its resource block**.
- **A removed block with destroy = false is the correct way** to stop managing something without destroying it.
- It is declarative, it appears in the plan, and it survives review.
- Simply deleting a protected resource block, by contrast, produces a destroy plan.
- So the answer to 'keep the object, stop managing it' is a **removed block**, not a lifecycle rule.

> ON THE EXAM
>
> Module eleven posed this problem and deferred the answer. This is it, and it did not exist before Terraform 1.7.

Blocks or state commands?

THE BLOCKS

In configuration

- Appear in the plan before applying
- Reviewed like any other change
- Repeatable across environments
- Available from 1.1, 1.5 and 1.7

THE COMMANDS

At the terminal

- Mutate state immediately
- No plan, no preview, no review
- Run once, on one person's machine
- Still necessary in some situations

THE TIEBREAKER

Prefer the blocks. The commands remain the answer when you need to inspect state, or fix something the blocks cannot express.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Somebody changed it in the console	Drift. Then decide: revert, adopt or codify
Record what happened, change nothing	apply -refresh-only
Detect manual changes nightly	plan -refresh-only with -detailed-exitcode
A rename produced a destroy plan	A moved block
Another team should own this now	A removed block with destroy = false
Zero to add, change and destroy, but state changed	A removed block. The summary counts infrastructure

ON THE EXAM

Ask whether reality moved or you moved something. Drift is the first case; the refactoring blocks are the second.

> During an incident, an engineer increased a database's allocated storage through the cloud console. The change was correct and the incident is resolved. The Terraform configuration still specifies the old size.
>
> The team wants to keep the larger storage and have Terraform manage it correctly going forward.

What should they do?

A. Run terraform apply, which will reconcile state with reality

B. Run apply -refresh-only, then update the configuration to the new size

C. Run apply -refresh-only only. State now matches reality, so the work is done

D. Use a removed block so Terraform stops managing the database

◆ PAUSE AND DECIDE

What should they do?

A Just apply

Reverts the change. A normal apply makes reality match configuration, and the configuration still says the old size.

B Refresh-only, then codify

Correct. Adopt the change into state, then write it into configuration so the next normal apply proposes nothing.

C Refresh-only alone

State matches reality and configuration does not. The next normal apply would shrink the database back.

D A removed block

Stops Terraform managing the database entirely, which throws away management to avoid editing one attribute.

THE SIGNAL

Adopt and codify are two steps. Doing only the first leaves a change that the next routine apply will quietly undo.

> A team is handing a legacy storage bucket over to a different department, who will manage it with their own tooling. The bucket must keep existing with all its contents.
>
> An engineer deletes the resource block and writes a removed block with the from address, then runs plan. Terraform reports a validation error.

What is missing, and what must it say?

A. A to address, since removed blocks work like moved blocks

B. A nested lifecycle block with destroy = false

C. A prevent_destroy lifecycle rule on the original resource

D. The resource block must stay, with the removed block alongside it

◆ PAUSE AND DECIDE

What is missing, and what must it say?

A A to address

removed has no destination. The object is leaving management, not moving. Only moved takes a to address.

B lifecycle with destroy = false

Correct. The nested lifecycle block is required, and destroy = false is what forgets the object without deleting it.

C prevent_destroy

Module eleven: it dies with the resource block, which has already been deleted. It also cannot survive to protect anything here.

D Keep the resource block

Backwards. A removed block replaces the resource block; leaving both would be contradictory.

THE SIGNAL

Terraform requiring the lifecycle block is a safety feature. It refuses to let you leave the destroy decision implicit.

> Following module ten's advice, a team converts nine production databases from count over a list to for_each over a set. The names are unchanged. The plan proposes to destroy all nine and create nine replacements.
>
> They need the refactor to touch no infrastructure at all.

What do they add?

A. A removed block per database, with destroy = false, then re-import them

B. A moved block per instance, mapping each old index to its new key

C. prevent_destroy on each database, so the destroy is blocked

D. An ignore_changes rule so the address change is not detected

◆ PAUSE AND DECIDE

What do they add?

A Remove then re-import

Reaches the same state by a longer, riskier route, with a window where nine production databases are unmanaged.

B moved blocks

Correct, and exactly what moved exists for. Each block maps one old index to its new key, and no infrastructure is touched.

C prevent_destroy

Blocks the plan rather than fixing it. Terraform would refuse to apply, and the refactor would stay stuck.

D ignore_changes

Ignores attribute changes, not addresses. Instance identity is not an attribute, so there is nothing for it to ignore.

THE SIGNAL

moved blocks are per instance here, not per resource, because the change is to how instances are keyed.

> A pipeline runs terraform plan -refresh-only nightly against twelve configurations, with -detailed-exitcode, and raises a ticket when the exit code is 2. It has run for six months.
>
> A new engineer asks whether this could accidentally change production, since it runs unattended every night.

What is the correct answer?

A. Yes. Refresh-only writes findings into state, which is a change

B. No. It is a plan, so it changes nothing, and refresh-only cannot propose infrastructure changes anyway

C. Yes, if drift is found, since Terraform reverts it automatically

D. No, but only because -detailed-exitcode suppresses the apply step

◆ PAUSE AND DECIDE

What is the correct answer?

A — It writes to state

Confuses plan with apply. A refresh-only APPLY writes findings to state; a refresh-only PLAN only reports them.

B — No, on both counts

Correct, and doubly safe. Plan never changes anything, and refresh-only mode cannot propose infrastructure changes even when applied.

C — It reverts automatically

Terraform never applies anything unprompted. Reverting drift requires somebody to run a normal apply and approve it.

D — The exit code flag

That flag only changes the process exit code. It has no effect on what the command does.

THE SIGNAL

Two independent reasons this is safe. It is a plan, and it is refresh-only. Either alone would be sufficient.

> A shared module renames an internal resource and ships a moved block. Six months later, believing everyone has upgraded, the maintainer deletes the moved block and releases a new version.
>
> A team that had been pinned to a much older version upgrades directly to the newest, and their plan proposes to destroy and recreate the resource.

What went wrong?

A The consumer skipped versions, so they must upgrade one release at a time

B The moved block was deleted before every consumer had applied it. It should have been kept

C moved blocks only work within the same major version of a module

D The consumer's state is corrupt and must be rebuilt with import

◆ PAUSE AND DECIDE

What went wrong?

A Skipped versions

There is no requirement to upgrade incrementally, and a well-maintained module should not need it. It also only works if the block still exists.

B Deleted too early

Correct. The consumer's state still holds the old address, and nothing remains to explain the rename. HashiCorp says keeping them indefinitely is fine.

C Major version limit

Invents a rule. A moved block is evaluated against whatever state exists, with no relationship to version numbering.

D Corrupt state

The state is perfectly valid. It simply records the old address, which is exactly what the moved block existed to translate.

THE SIGNAL

In a published module you cannot know who is still behind. That uncertainty is the argument for keeping moved blocks permanently.

Module 17 recap

- **Drift is state against reality.** The plan is configuration against state. Different comparisons.
- **Three responses: revert, adopt, codify.** Adopting alone leaves the next apply to undo it.
- **plan -refresh-only with -detailed-exitcode** is a safe scheduled drift detector.
- **moved renames, removed forgets, import adopts.** All three appear in the plan.
- **A removed block requires a lifecycle block**, and destroy = false is what keeps the object.
- **Keep moved blocks for a long time**, and indefinitely in a shared module.

> ON THE EXAM
>
> The blocks exist so state changes go through a plan and a review, instead of one person's terminal.

MODULE 18 · DOMAIN 6 - TERRAFORM STATE MANAGEMENT

State Commands, and CLI Workspaces

Editing state deliberately, and the word that means two completely different things.

DOMAIN 6 of 8 COURSE WEIGHT 12% MODULE 18 of 27

admin@sthithapragna.com

A CLI workspace is **another state file for the same configuration**. It is not an environment, and it is not an HCP Terraform workspace.

The state commands

- **terraform state** is a family of subcommands for reading and editing state directly.
- They are the **supported interface** to state. Module four said never to read or edit the file yourself.
- **They act immediately.** No plan, no preview, no approval, no record in a pull request.
- Terraform writes a **backup** before any command that modifies state. Keep it until you are sure.
- Module seventeen's blocks are preferable where they apply. These remain necessary for **inspection**.

> ON THE EXAM
>
> Inspection has no declarative equivalent. That is the enduring reason these commands exist alongside the refactoring blocks.

The subcommands

COMMAND	WHAT IT DOES
`state list`	List the resource addresses tracked in state
`state show ADDRESS`	Print one resource's recorded attributes
`state mv SRC DEST`	Rename or move a tracked object. The moved block does this declaratively
`state rm ADDRESS`	Forget an object without destroying it. removed does this declaratively
`state pull`	Output the raw state, for inspection or scripted backup
`state push`	Overwrite state from a file. The most dangerous command here

ON THE EXAM

list and show are read-only and completely safe. Everything below them changes state immediately.

Inspecting state

- **terraform state list** shows every address, and accepts an address filter to narrow it.
- **terraform state show** prints one resource's attributes as recorded, including computed ones.
- This is how you find the exact address for a **-target**, a **moved** block or an **import** block.
- **terraform show** without arguments prints the whole state, and with -json gives machine-readable output.
- **Neither changes anything.** Both are safe to run at any time, including in production.

> ON THE EXAM
>
> Getting an address right matters more than it looks. list and show are how you check one before using it.

state mv and state rm

- **state mv** changes the address an object is tracked under. The object itself is untouched.
- **state rm** makes Terraform forget an object. The object continues to exist.
- **Neither destroys anything**, which surprises people. They edit belief, not reality.
- Both now have declarative equivalents: **moved** and **removed with destroy = false**.
- After a state rm, the configuration must be removed too, or the next plan proposes to create it again.

> ON THE EXAM
>
> state rm followed by leaving the resource block in place is how people accidentally create a duplicate.

pull, push and replace-provider

- **state pull** fetches and outputs the current state. Useful for a scripted backup before risky work.
- **state push** overwrites the remote state with a local file. It bypasses everything Terraform normally checks.
- Terraform checks **lineage and serial** and will refuse an obviously wrong push, which is why **-force** exists.
- **Never use -force on a push** unless you fully understand the state you are overwriting.
- **state replace-provider** rewrites which provider a resource belongs to, for provider renames or forks.

> ON THE EXAM
>
> Lineage and serial from module four exist for this. They are how Terraform notices you are pushing an unrelated state.

Rules for editing state by hand

- **Take a backup first**, with state pull or by copying the file. Terraform's automatic backup is not a plan.
- **Check the address with state list** before using it. A typo in an address is not always an error.
- **Nobody else should be running Terraform** while you do this. Locking protects applies, not your editing.
- **Prefer the declarative block** where one exists, so the change is reviewed and repeatable.
- Afterwards, **run a plan and confirm it is empty**. That is the only real verification.

> ON THE EXAM
>
> An empty plan after state surgery is the proof that you did it correctly. Anything else means you are not finished.

CLI workspaces

- **Separate instances of state data inside the same working directory.** That is HashiCorp's definition.
- **One configuration, one backend, one set of credentials, several states.**
- Every initialised directory starts with a workspace named **default**, which **cannot be deleted**.
- **Only some backends support multiple workspaces.** It is a per-backend capability, like locking.
- A plan in one workspace **does not see resources in another**. They exist, but you must switch to manage them.

> ON THE EXAM
>
> One backend, several states. Nothing about credentials, permissions or configuration differs between workspaces.

The workspace commands

COMMAND	WHAT IT DOES
`workspace list`	Show all workspaces, marking the current one
`workspace show`	Print the current workspace name
`workspace new NAME`	Create a workspace and switch to it
`workspace select NAME`	Switch to an existing workspace
`workspace delete NAME`	Delete a workspace. Not the default, and not the current one

ON THE EXAM

workspace show before apply is a cheap habit. The current workspace is stored locally and is easy to forget.

Where workspace state actually lives

- **Local backend:** in a directory called **terraform.tfstate.d**, one subdirectory per workspace.
- **Remote backend:** in the same backend, with the workspace name appended to the state path.
- **One backend holds all workspaces.** You do not need a bucket per workspace, and should not create one.
- Workspace names must be valid in a **URL path segment without escaping**, because of how they are stored.
- The **current workspace name is stored locally**, in the ignored .terraform directory, so colleagues can differ.

> ON THE EXAM
>
> The current workspace is a local, per-machine setting. Two people in the same repository can be in different workspaces.

Creating a workspace changes nothing by itself

- **A new workspace gives you an empty state, and the identical configuration.**
- So applying in it builds **a second copy of exactly the same infrastructure**, unless something varies.
- **terraform.workspace** is the named value holding the current workspace name.
- You must **reference it** to get different behaviour: in names, in sizes, in a lookup of per-workspace settings.
- **Forgetting this is the classic mistake.** The workspaces are isolated and the infrastructure is identical.

> ON THE EXAM
>
> Workspaces isolate state. They do not vary configuration. Only terraform.workspace, or a variable file, does that.

Making a workspace behave differently

PATTERN	HOW IT WORKS
Name things after the workspace	Interpolate terraform.workspace into resource names
Look up per-workspace settings	A map keyed by workspace name, read with lookup
Scale by workspace	A conditional: one size in one workspace, another elsewhere
Per-workspace variable files	-var-file passed at the command line. Not auto-loaded
Do nothing	An identical second copy of everything

ON THE EXAM

Terraform will not load a tfvars file named after the workspace. Only terraform.tfvars and *.auto.tfvars auto-load.

HashiCorp's warning

- **Workspaces are not appropriate for deployments needing separate credentials and access controls.**
- The reason is structural: one backend, one credential set, one configuration.
- So **production and development share a blast radius**, and a wrong select is an outage.
- The recommended alternative: **separate configurations** matching architectural boundaries.
- Where they do fit: **short-lived, structurally identical copies** of one stack.

> ON THE EXAM
>
> Enormous amounts of community material recommend workspaces for dev, staging and production. HashiCorp's documentation says otherwise.

CLI workspace or HCP Terraform workspace?

CLI WORKSPACE

A state file selector

- Separate state inside one working directory
- One configuration, one backend
- Shares credentials with every other workspace
- Switched with terraform workspace select

HCP TERRAFORM WORKSPACE

A managed environment

- Its own configuration and working directory
- Its own variables and credentials
- Its own run history, state versions and permissions
- The unit of organisation in HCP Terraform

THE TIEBREAKER

HashiCorp states the two are distinctly different. An HCP workspace is closer to a whole root module than to a CLI workspace.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Separate credentials or accounts per environment	Not CLI workspaces. Separate configurations
I made a workspace and got identical infrastructure	terraform.workspace was never referenced
Rename a resource without recreating it	A moved block, or state mv as the older route
Stop managing it, keep the object	removed with destroy = false, or state rm
Its own variables and run history	An HCP Terraform workspace, not a CLI one
Which resources does Terraform track here?	terraform state list

ON THE EXAM

When a question says 'workspace', decide which kind it means before anything else. The answers differ completely.

> A team proposes managing development, staging and production with three CLI workspaces on one configuration. Each environment lives in a separate cloud account with its own credentials, and only two people may apply to production.
>
> They cite several blog posts describing workspaces as the standard way to handle environments.

What should a reviewer advise?

A. Approve. Workspaces are the documented approach for multiple environments

B. Advise separate configurations. Workspaces share one backend and one credential set

C. Approve, provided each workspace uses a different state bucket

D. Advise HCP Terraform, since CLI workspaces cannot hold more than two states

◆ PAUSE AND DECIDE

What should a reviewer advise?

A. Approve, it is documented

The blog posts say so; HashiCorp says the opposite. Their documentation calls workspaces inappropriate for deployments needing separate credentials and access controls.

B. Separate configurations

Correct. Separate accounts, separate credentials and a permission boundary on production are exactly the conditions the warning names.

C. A bucket per workspace

Misunderstands the mechanism. One backend holds all workspaces, keyed by name. You cannot give each its own bucket without separate configurations anyway.

D. Two-state limit

Invents a limit. There is no cap on workspaces, and HCP Terraform is a reasonable answer here for entirely different reasons.

THE SIGNAL

Three details name the warning exactly: separate accounts, separate credentials, and a permission boundary on production.

> An engineer runs terraform workspace new staging, then terraform apply. The apply succeeds. She then discovers she has a second, complete copy of the production infrastructure, with identical resource names where the platform allowed it.
>
> She expected the staging workspace to produce smaller, differently named resources.

Why did this happen?

A The workspace inherited production's state and reapplied it

B A workspace isolates state but does not vary configuration. Nothing referenced terraform.workspace

C She needed to run terraform init after creating the workspace

D Workspaces require a matching .tfvars file named after the workspace

◆ PAUSE AND DECIDE

Why did this happen?

A **It inherited state**

The opposite. A new workspace starts with empty state, which is precisely why the apply created everything from scratch.

B **Nothing referenced the workspace**

Correct. The configuration was unchanged, so applying it against an empty state produced a full second copy.

C **Run init**

Workspaces do not need reinitialising. Init handles backend, providers and modules, none of which changed.

D **A matching tfvars file**

Invents a rule. Terraform auto-loads terraform.tfvars and .auto.tfvars only, as module eight established. Workspace names mean nothing to file loading.

THE SIGNAL

The apply behaved perfectly. An empty state plus an unchanged configuration means create everything, and that is what happened.

> An engineer needs to rename a resource in a configuration used by six teams across nine workspaces. Renaming it in configuration alone would destroy and recreate the object.
>
> A colleague suggests running terraform state mv in each workspace. Another suggests a moved block.

Which is better here, and why?

A state mv, because it takes effect immediately without waiting for an apply

B A moved block, because it is applied once per workspace automatically and goes through review

C Neither. Renaming always requires destroy and recreate

D state mv, because moved blocks do not work across multiple workspaces

◆ PAUSE AND DECIDE

Which is better here, and why?

A. state mv, immediate

Immediacy is the drawback, not the benefit. It means nine manual runs, no review, and no record of what was done.

B. A moved block

Correct. It ships with the configuration, so every workspace picks it up on its next apply, and the change is reviewed once.

C. Always destructive

False since Terraform 1.1. moved blocks exist specifically so a rename is not destructive.

D. moved does not span workspaces

Invents a limit. A moved block is part of the configuration, and every workspace uses the same configuration.

THE SIGNAL

Nine workspaces is the deciding detail. It turns a manual command into nine chances to make a mistake.

> A team is planning a migration to HCP Terraform. Their current setup uses four CLI workspaces on one configuration. A manager reads that HCP Terraform also has workspaces and concludes the migration will be a straight one-to-one mapping with no other changes.
>
> An engineer is asked whether that is accurate.

What is the correct characterisation?

A. Accurate. The two are the same concept with different names

B. Not quite. An HCP workspace has its own configuration, variables, credentials and permissions

C. Inaccurate. HCP Terraform does not support multiple workspaces per configuration

D. Accurate, provided the CLI workspaces are renamed to match HCP naming rules

◆ PAUSE AND DECIDE

What is the correct characterisation?

A **Same concept**

HashiCorp's documentation calls them distinctly different. A CLI workspace selects a state file; an HCP workspace is a managed environment.

B **Its own everything**

Correct. An HCP workspace functions as its own working directory, with separate variables, credentials, run history and permissions.

C **HCP has no multiple workspaces**

Backwards. Workspaces are the central unit of organisation in HCP Terraform. There is documented guidance on migrating CLI workspaces to it.

D **Just rename them**

Naming is a detail. The manager's error is conceptual, and renaming does nothing about separate variables or permissions.

THE SIGNAL

The migration is genuinely possible and documented. What is wrong is 'with no other changes', because HCP workspaces carry far more.

Module 18 recap

- **state list and show are safe.** mv, rm, push and replace-provider change state immediately.
- **state rm does not destroy anything**, and leaving the resource block behind creates a duplicate.
- **Verify state surgery with an empty plan.** That is the only real proof.
- **A CLI workspace is another state file** for one configuration, one backend, one credential set.
- **HashiCorp says workspaces are not appropriate** where separate credentials or access controls are needed.
- **An HCP workspace is a different thing entirely**, with its own configuration, variables and permissions.

> ON THE EXAM
>
> Creating a workspace changes nothing on its own. Reference terraform.workspace, or you get an identical second copy.

MODULE 19 · DOMAIN 7 - MAINTAIN INFRASTRUCTURE

Importing Existing Infrastructure

Bringing objects Terraform did not create under management, and the half of the job import does not do.

DOMAIN 7 of 8 COURSE WEIGHT 8% MODULE 19 of 27

admin@sthithapragna.com

Import creates a **binding in state**. It does not write your configuration. That half is still your job.

Why import exists

- Infrastructure that **predates Terraform**, or was created by hand during an incident.
- Objects created by **another tool** that you now want managed in one place.
- Recovering after somebody **removed a resource from state** and still needs it managed.
- Module four's last resort: rebuilding after **state was lost entirely**.
- **Not** for copying a resource between configurations. That is a state move, or a removed and an import.

> ON THE EXAM
>
> Import is adoption. If Terraform created the object and still tracks it, import is not the tool you want.

The terraform import command

- **terraform import ADDRESS ID**. The address is where it should live; the ID is the provider's identifier.
- It **writes state immediately**, with no plan and no preview.
- **One resource per invocation.** Forty resources means forty commands.
- **It does not generate configuration.** The resource block must already exist, or the next plan destroys the object.
- **It is not deprecated.** It suits one-off adoptions and quick repairs.

> ON THE EXAM
>
> This is the older route and it still works. Its weaknesses are the same as every state command: immediate, unreviewed, one at a time.

The import block

- **import** with a **to** address and an **id**. Available from Terraform 1.5.
- **It appears in the plan**, so you preview the import before state is modified.
- HashiCorp's wording: configuration-driven import is **predictable, works with CI/CD, and lets you preview**.
- **Many at once**, and with **for_each** from Terraform 1.7, a whole map in one apply.
- Once applied, the block is a no-op and can be removed, like a moved block.

> ON THE EXAM
>
> The plan reports the count as 'to import', separately from adds, changes and destroys. Recognise that line.

Command or block?

TERRAFORM IMPORT

Immediate and manual

- Writes state at once, no preview
- One resource per run
- Not version controlled
- Fine for a one-off or a repair

IMPORT BLOCK

Declarative and reviewed

- Appears in the plan first
- Many resources, and for_each
- Lives in configuration, goes through review
- Repeatable across environments

THE TIEBREAKER

Neither is deprecated, and both produce identical state entries. The difference is process, not result.

The import workflow

01

Identify

Find the object, and the provider-specific ID format from its docs.

02

Configure

Write a resource block, or generate a draft one.

03

Import

Add an import block and apply, or run terraform import.

04

Reconcile

Plan, and adjust configuration until it proposes nothing.

ON THE EXAM

The last step is where the real work is. An import is only finished when the plan is empty.

Generating configuration

- Write only the import block, then run **terraform plan -generate-config-out=FILE**.
- Terraform writes HCL containing its **best guess at the appropriate value** for each argument.
- **The file must be new.** Terraform will not overwrite an existing one.
- If your configuration has **no other resources for that provider**, add a provider block first, then run init again.
- **Experimental since 1.5.** HashiCorp says formatting and behaviour may change in later minor versions.

> ON THE EXAM
>
> Still marked experimental, and still the fastest way to start. Expect the output to change between versions.

What generated configuration looks like

- **Verbose.** Provider defaults, empty maps and explicit nulls that nobody would commit by hand.
- **Hardcoded.** Values that should be variables or references to other resources are literals.
- **Treat it as a draft.** Trim it, format it, re-plan, and only then commit.
- It is still far faster than reading a provider's documentation attribute by attribute.
- **All generated code needs review before committing.** That is the consistent advice everywhere.

> ON THE EXAM
>
> The output is a starting point, not a final artefact. Committing it unread is how a repository fills with provider defaults.

Where generation will not help

- **-generate-config-out cannot be used with an import block that uses for_each.**
- **Nor with resources that use count or for_each.** Terraform reports this as an error, not a warning.
- So the bulk import feature and the generation feature **do not combine**.
- For those, write the resource block yourself, then import with for_each.
- A workable route: generate for **one** instance, generalise it by hand, then import the rest.

> ON THE EXAM
>
> Two features that sound complementary and are mutually exclusive. This is a clean, checkable fact.

One object, one address

- **Each remote object should be imported to exactly one resource address.** Module four's rule.
- Import the same object twice and the mapping is ambiguous. Terraform may then act unexpectedly against it.
- The likely outcome: **two configurations both believe they own it**, and one destroys what the other manages.
- **Some services are represented by several resource types**, so full management may need several imports.
- Those are different objects, not the same object twice. Check the provider documentation.

> ON THE EXAM
>
> Creation guarantees the one-to-one rule automatically. Import is the one operation where you can break it.

Verifying an import

- **Run a plan. It should propose nothing.** That is the only real confirmation.
- **Changes mean your configuration does not match reality**, and applying would change the object.
- **A proposed destroy means the resource block is missing**, so Terraform sees a binding with nothing declared.
- **A proposed create means the address is wrong**, so the import landed somewhere your configuration is not looking.
- Iterate on the configuration, not on the state. State is already correct.

> ON THE EXAM
>
> Three plan outcomes, three different mistakes. Read which one you got before changing anything.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
After importing, the plan wants to destroy it	No resource block was written
After importing, the plan wants to change it	The configuration does not match reality yet
Adopt forty resources through a pipeline	import blocks, reviewed. Not forty commands
Generate the configuration for these	plan -generate-config-out, to a new file
Generation failed on a for_each import	Documented limitation. They do not combine
Two configurations manage the same object	It was imported twice. Break one binding

ON THE EXAM

Read what the post-import plan proposes. Destroy, change and create each point at a different mistake.

> An engineer adopts a manually created storage bucket. She looks up its identifier and runs terraform import for the address she intends to use. The command reports a successful import.
>
> She runs terraform plan, expecting no changes, and the plan proposes to destroy the bucket.

What happened?

A The import failed silently and left a partial state entry

B No resource block exists at that address, so Terraform sees a binding for something not declared

C The bucket must be tagged before Terraform will recognise it as managed

D terraform import always requires a second run to complete the adoption

◆ PAUSE AND DECIDE

What happened?

A Silent failure

The command reported success, and it succeeded. State now holds a correct binding, which is exactly why the plan can see something to destroy.

B No resource block

Correct. Import writes state and never writes configuration. A binding with nothing declared is, to Terraform, a resource you deleted.

C Tagging

Module four: tags were tried as an identity mechanism and abandoned. Terraform identifies objects by what state records.

D Two runs

Invents a rule. One invocation completes the import. What is missing is the other half of the job, which is yours.

THE SIGNAL

Module six's plan annotation says it exactly: destroyed 'because it is not in configuration'. That is what this plan is reporting.

> A team must adopt about forty existing resources across three accounts. The work must go through code review, run in their pipeline, and be repeatable in a second region later.
>
> One engineer proposes a runbook of forty terraform import commands. Another proposes import blocks.

Which fits the requirements, and why?

A The commands, since they take effect immediately and need no apply

B import blocks, because they are previewed in a plan, reviewed, and repeatable

C The commands, because import blocks cannot be used in automation

D Either. The two are functionally identical, so the choice is a matter of taste

◆ PAUSE AND DECIDE

Which fits the requirements, and why?

A Commands, immediate

Immediacy is the drawback. Forty unreviewed state mutations from a terminal is the opposite of what the requirements describe.

B import blocks

Correct. HashiCorp describes configuration-driven import as predictable, working with CI/CD, and previewable - which is the requirement list restated.

C Blocks cannot automate

Backwards. Working with CI/CD pipelines is one of the stated advantages of the block over the command.

D Functionally identical

The resulting state is identical; the process is not. The requirements here are entirely about process.

THE SIGNAL

The requirements name the block's three documented advantages almost word for word: reviewed, pipeline-friendly, repeatable.

> An engineer writes an import block using for_each over a map of twelve existing databases, intending to adopt them all in one apply. She then runs terraform plan with -generate-config-out to produce the resource configuration.
>
> Terraform reports an error before generating anything.

What is the cause, and what should she do?

A Generation cannot be used with for_each. Generate for one, generalise it, then import the rest

B The output file already exists and must be deleted first

C for_each on import blocks requires Terraform 1.9 or later

D Generation requires the resources to already exist in state

◆ PAUSE AND DECIDE

What is the cause, and what should she do?

A They do not combine

Correct. Terraform states the flag cannot be used with import blocks that use for_each, nor with resources using for_each or count.

B File exists

A real requirement - the file must be new - and it produces a different, clearly worded error about the file rather than about for_each.

C Needs 1.9

for_each on import blocks arrived in 1.7. Version is not the constraint here, and the exam tests 1.12 regardless.

D Must be in state already

Backwards. Generation exists precisely for resources not yet in state; once imported, no generation is needed.

THE SIGNAL

Bulk import and configuration generation are the two headline import features, and they are mutually exclusive.

> Two teams adopt Terraform independently. Both need a shared DNS zone, and each imports it into their own configuration so they can manage records.
>
> Months later, one team decommissions its configuration and runs destroy. The zone disappears, and the other team's next plan proposes to create it.

What was the underlying error?

A The second team should have run terraform refresh to detect the deletion earlier

B One remote object was imported into two addresses, so both configurations believed they owned it

C Shared resources must always use prevent_destroy on both sides

D The zone should have been imported into a CLI workspace shared by both teams

◆ PAUSE AND DECIDE

What was the underlying error?

A Refresh earlier

Detects the deletion sooner and prevents nothing. The zone is already gone by the time any refresh could notice.

B Imported twice

Correct. Each remote object should be imported to exactly one address. Two owners means one of them will eventually destroy it.

C prevent_destroy on both

Would have blocked this particular destroy, and M11 showed it dies when the resource block is deleted. It treats a symptom of dual ownership.

D A shared workspace

M18: a CLI workspace is a state file inside one working directory. Two teams with separate configurations cannot share one.

THE SIGNAL

The correct pattern is one owner and a data source for everyone else. Ownership, as module seven framed it, is the whole question.

Module 19 recap

- **Import creates a state binding. It never writes configuration.**
- **A post-import destroy plan means no resource block exists** at that address.
- **import blocks (1.5) are previewed, reviewed and repeatable.** The command is immediate and unreviewed.
- **-generate-config-out writes a draft**, to a new file, and is still experimental.
- **Generation does not work with for_each**, on the import block or on the resource.
- **One object, one address.** Import is the only operation that can break that rule.

> ON THE EXAM
>
> An import is finished when the plan is empty. Until then, fix the configuration, never the state.

MODULE 20 · DOMAIN 7 - MAINTAIN INFRASTRUCTURE

Logging, Debugging and Troubleshooting

Turning on verbose logging, choosing which half of Terraform to listen to, and reading the error you actually got.

DOMAIN 7 of 8 COURSE WEIGHT 8% MODULE 20 of 27

admin@sthithapragna.com

Module two split Terraform into **core and plugins**. Debugging is mostly working out which half is talking.

The log levels

TF_LOG VALUE	WHAT YOU GET
TRACE	The most verbose. Internal operations and provider API calls
DEBUG	Detailed operations, without the full noise of TRACE
INFO	General progress messages about what Terraform is doing
WARN	Non-critical issues that do not stop execution
ERROR	Only failures that prevent Terraform continuing
JSON	TRACE level or higher, in a parseable JSON encoding

ON THE EXAM

Unset means no logging at all. TRACE is the most verbose, which is the ordering question people get backwards.

TF_LOG and TF_LOG_PATH

- **TF_LOG** sets the level. It is an environment variable, not a command-line flag.
- **TF_LOG_PATH** forces the log to be **appended** to a specific file.
- **Even when TF_LOG_PATH is set, TF_LOG must also be set, or nothing is logged at all.**
- That is the gotcha. Setting only the path produces an empty file and a confusing hour.
- The file is appended to, so a long debugging session accumulates several runs in one file.

> ON THE EXAM
>
> Setting a destination does not enable logging. TF_LOG is the switch; TF_LOG_PATH only redirects the output.

Listening to one half

- **TF_LOG_CORE** enables logging for Terraform itself. **TF_LOG_PROVIDER** for the provider plugins.
- Both take the **same level values** as TF_LOG, and activate only a subset of the logs.
- Available since **Terraform 0.15**.
- A provider API problem: **TF_LOG_PROVIDER=TRACE**. A graph or state problem: **TF_LOG_CORE=TRACE**.
- TRACE across both halves is enormous. Narrowing to one is usually the difference between readable and unreadable.

> ON THE EXAM
>
> This is module two's split made operational. Decide which half you are debugging, then listen to only that half.

Two warnings about verbose logs

- **TRACE logs include provider API calls**, so request and response bodies can contain sensitive values.
- So a TRACE log **should be treated like state or a plan file**: potentially sensitive, and not attached to tickets casually.
- **The JSON encoding is not a stable interface.** HashiCorp says it may change at any time, without warning.
- So do not build tooling that parses it. It exists to support HashiCorp's own tooling.
- Turn logging off again afterwards. An exported variable outlives the problem it was set for.

> ON THE EXAM
>
> Third artefact in this course that quietly holds secrets, after state and plan files. The reasoning is identical each time.

Other TF_ environment variables

VARIABLE	WHAT IT DOES
TF_VAR_name	Sets an input variable. Module eight, and nearly the weakest source
TF_INPUT	Set to false to disable interactive prompts, like -input=false
TF_CLI_ARGS	Adds arguments to every command. TF_CLI_ARGS_init targets one
TF_WORKSPACE	Selects a CLI workspace without running workspace select
TF_DATA_DIR	Moves the .terraform directory somewhere else
TF_IN_AUTOMATION	Softens output that suggests interactive next steps

ON THE EXAM

TF_CLI_ARGS_init is how a pipeline applies -backend-config without editing every command. Module sixteen's pattern.

Four kinds of error

- **Language errors.** Invalid syntax, wrong types, unresolvable references. Caught before anything runs.
- **State errors.** State disagreeing with reality, or with itself. Locks, drift, missing bindings.
- **Core errors.** Terraform's own logic: cycles, unknown values at plan time, graph problems.
- **Provider errors.** The platform refused. Permissions, quotas, validation, rate limits.
- HashiCorp's troubleshooting guidance groups errors this way, and each kind has a different first move.

> ON THE EXAM
>
> Naming the kind first saves time. A permissions failure and a cycle look equally like 'Terraform is broken' until you classify them.

Which tool for which error

THE KIND OF ERROR	REACH FOR
Language	terraform validate, and terraform fmt. No credentials needed
Expression or type confusion	terraform console, to evaluate it directly
State	terraform state list and show, then plan -refresh-only
Core	TF_LOG_CORE=TRACE, and terraform graph for cycles
Provider	TF_LOG_PROVIDER=TRACE, then the provider's own documentation
A crash	The crash log, plus terraform version, in a bug report

ON THE EXAM

Most of these are the read-only commands from modules six and eighteen. Reach for those before turning on TRACE.

When Terraform crashes

- A genuine panic writes a **crash log** alongside the stack trace on screen.
- **A crash is a bug**, in Terraform or in a provider. It is not something you configure around.
- Report it with the **crash log**, your **terraform version** output, and a minimal reproduction.
- **terraform version** prints the CLI version and every installed provider version. Module six.
- **Check the crash log before sharing it**, for the same reason as a TRACE log.

> ON THE EXAM
>
> Most errors are not crashes. A crash means a stack trace and a panic, not a red error message.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
The log file is empty	TF_LOG_PATH was set without TF_LOG
The provider returned something unexpected	TF_LOG_PROVIDER=TRACE
TRACE output is unreadably large	Narrow to one half with CORE or PROVIDER
Check the code without credentials	terraform validate. Module six
A stack trace and a panic	A crash log, and a bug report
Logs were attached to a public ticket	TRACE logs can contain secrets

ON THE EXAM

Logging questions are mechanical. Classify the error first, then pick the narrowest tool that can see it.

> An engineer is asked to capture verbose logs for a support ticket. She exports TF_LOG_PATH pointing at a file in her home directory and runs terraform apply.
>
> The apply completes. The file exists and is completely empty. She repeats it twice with the same result.

What is wrong?

A TF_LOG must also be set. TF_LOG_PATH alone enables nothing

B The path must be relative to the working directory

C Logs are only written when a command fails

D TF_LOG_PATH was deprecated in favour of the -log-path flag

◆ PAUSE AND DECIDE

What is wrong?

A TF_LOG is the switch

Correct, and documented almost word for word: even when TF_LOG_PATH is set, TF_LOG must be set for any logging to be enabled.

B Path must be relative

Absolute paths work fine. The file was created at the path she gave, which proves Terraform read the variable.

C Only on failure

Logging is level-based, not outcome-based. A successful run at TRACE produces a great deal of output.

D Deprecated for a flag

There is no such flag. Terraform's logging is configured entirely through environment variables.

THE SIGNAL

The empty file is the clue. Terraform read the path variable and created the file, then had no logging enabled to write into it.

> An apply fails intermittently on one resource type, with a message from the cloud provider that the team cannot interpret. They set TF_LOG to TRACE and capture a log, which runs to several hundred megabytes and is dominated by Terraform's internal graph output.
>
> They need to see the API request and response for the failing call.

What is the better approach?

A TF_LOG=DEBUG, which shows API calls with less noise

B TF_LOG_PROVIDER=TRACE, which logs only the provider plugins

C TF_LOG_CORE=TRACE, since the provider is invoked by core

D Run apply with -parallelism=1 so the log contains one operation at a time

◆ PAUSE AND DECIDE

What is the better approach?

A DEBUG instead

Reduces volume by lowering detail, which risks losing the request and response bodies they specifically need.

B TF_LOG_PROVIDER=TRACE

Correct. It keeps full detail on the half they care about and removes core's graph output entirely.

C TF_LOG_CORE=TRACE

Exactly the output they are already drowning in. Core is the half that produced the noise, not the API calls.

D Parallelism 1

Makes the log easier to follow and does nothing about its size or which half is talking. It also slows every apply.

THE SIGNAL

Narrow by component before narrowing by level. Lowering the level throws away detail; choosing a half throws away the wrong subject.

> A pull request fails in a pipeline that has no cloud credentials by design. The failure is an error about an argument name that does not exist on a resource type.
>
> An engineer proposes giving the pipeline read-only credentials so it can run a plan and catch these earlier.

What should be advised?

A Approve. Only a plan can detect an invalid argument name

B This is a language error. terraform validate catches it with no credentials

C Set TF_LOG=TRACE in the pipeline to reveal the argument name

D The error is a provider bug and should be reported upstream

◆ PAUSE AND DECIDE

What should be advised?

A Only plan can catch it

False. Module six: validate checks attribute names and value types using local provider schemas, with no remote access.

B validate, no credentials

Correct. An invalid argument name is a language error, caught by validate after init -backend=false. Credentials are unnecessary.

C TRACE logging

Verbose logs describe what Terraform did. The error message already names the invalid argument; more logging adds nothing.

D A provider bug

The provider correctly reports that the argument does not exist. Nothing is malfunctioning except the configuration.

THE SIGNAL

Classifying the error first avoids granting credentials to solve a problem that never needed them.

> To help a vendor investigate, an engineer captures a full TRACE log of an apply that creates a managed database, and attaches it to a ticket in a system readable across the whole company.
>
> A colleague asks whether that was safe.

What is the correct assessment?

A Safe. Terraform redacts sensitive values in logs as it does in plan output

B Not safe. TRACE logs include API request and response bodies, which can contain secrets

C Safe, provided the variables were marked sensitive

D Not safe, but only because the log includes the backend configuration

◆ PAUSE AND DECIDE

What is the correct assessment?

A Terraform redacts logs

Redaction applies to Terraform's own rendered output. TRACE logs record the underlying API traffic, which is a different layer.

B TRACE holds API bodies

Correct. A managed database creation involves a generated password crossing that boundary, and TRACE records the traffic.

C sensitive protects it

The fifth appearance of this misconception. sensitive controls display in Terraform's output; it has never controlled what is written elsewhere.

D Only the backend config

Backend credentials are a real concern (M16), and naming it as the only concern understates a log that also holds resource-level secrets.

THE SIGNAL

State, plan files, and now TRACE logs. Three artefacts, one rule: obscured on screen has never meant absent from the file.

Module 20 recap

- **TRACE, DEBUG, INFO, WARN, ERROR**, plus JSON. TRACE is the most verbose; unset means nothing.
- **TF_LOG_PATH does nothing without TF_LOG.** The path redirects; the level switches on.
- **TF_LOG_CORE and TF_LOG_PROVIDER** narrow by half. Narrow by component before lowering the level.
- **TRACE logs can contain secrets.** Treat them like state and plan files.
- **Classify the error first:** language, state, core or provider. Each has a different first move.
- **validate needs no credentials.** Most language errors never require a plan.

> ON THE EXAM
>
> The JSON log encoding is explicitly not a stable interface. Do not parse it.

MODULE 21 · DOMAIN 8 - HCP TERRAFORM

HCP Terraform: Workflows and Remote Operations

Three ways to start a run, three places it can execute, and what you lose by choosing the wrong one.

DOMAIN 8 of 8 COURSE WEIGHT 12% MODULE 21 of 27

admin@sthithapragna.com

HCP Terraform is not just a place to keep state. It is a place where **runs execute**, and most of its features depend on that.

What HCP Terraform adds

- **Managed state**, versioned and encrypted, with locking, and no backend for you to operate.
- **Remote execution**, in a consistent environment rather than on whichever machine ran the command.
- **Secure variable storage**, including secrets, held by the service rather than in files or a pipeline.
- **Run history**: who ran what, when, with which plan, and who approved it.
- **Governance features** layered on the run: policy enforcement, notifications, run tasks. Module twenty-three.

> ON THE EXAM
>
> Every item after the first depends on the run happening remotely. State management is the only one that survives local execution.

The cloud block

- **A cloud block inside the terraform block**, naming an organisation and a workspace. Terraform 1.1 onwards.
- **It replaces the older remote backend**, and cannot coexist with a backend block. Module sixteen.
- **No interpolation**, exactly like a backend block. Partial configuration works the same way.
- Workspaces can be selected by **name**, or by **tags** so one configuration maps to several.
- **terraform init** connects the working directory to the service, as it would any backend.

> ON THE EXAM
>
> cloud and backend are mutually exclusive. With HCP Terraform you configure no backend at all, because the service manages state.

Authenticating

- **terraform login** opens a browser, you authorise, and a token is stored in your CLI configuration.
- The token lands in a credentials file under your home directory, not in the working directory.
- For automation, set the token in an **environment variable** named after the host instead.
- **The token is a credential.** It goes in a secret store, never in a repository.
- **terraform logout** removes it again.

> ON THE EXAM
>
> Interactive login for a person, an environment variable for a pipeline. The same split as every other credential in this course.

The three workflows

WORKFLOW	HOW A RUN STARTS
UI and VCS-driven	A push to the connected repository, or a button. HashiCorp calls this the primary mode
CLI-driven	terraform plan or apply from your terminal, executing remotely
API-driven	Your own tooling uploads a configuration and starts a run

ON THE EXAM

All three produce the same kind of run in the same environment. What differs is what triggers it.

VCS-driven or CLI-driven?

VCS-DRIVEN

The repository is the source of truth

- Runs trigger on push automatically
- Speculative plans on pull requests
- Nobody applies from a laptop
- Configuration comes from a branch

CLI-DRIVEN

The engineer starts the run

- Familiar terraform plan and apply
- Output streams to the terminal
- Fits an existing CI/CD pipeline
- Good first step when migrating

THE TIEBREAKER

The deciding question is what should trigger a run: a commit, or a person. Both execute in the same place.

The three execution modes

MODE	WHERE THE RUN HAPPENS
Remote	On HCP Terraform's infrastructure. The default for a new workspace
Local	On your workstation. The workspace stores state only
Agent	On a lightweight agent you run, inside private or on-premises infrastructure

ON THE EXAM

Workflow is how a run starts. Execution mode is where it runs. Two independent settings, frequently confused.

What Local execution mode costs you

- **The workspace acts only as a remote backend for state.** That is HashiCorp's own description.
- **Many features rely on remote execution and are not available**: policy enforcement, cost estimation, notifications.
- **Workspace variables and variable sets are not evaluated at all** in local execution mode.
- So values must come from your own environment, which is exactly what the service was meant to centralise.
- It remains legitimate for **debugging**, or where a run genuinely must happen on a particular machine.

> ON THE EXAM
>
> Choosing Local silently disables most of what HCP Terraform is for. Nothing errors; the features simply stop applying.

What a remote run actually does

- Terraform runs on HCP Terraform's infrastructure, and the **progress streams to your terminal**.
- HashiCorp's aim is an experience **equivalent to local operations**, including the confirmation prompt.
- **The service always plans first, then applies that plan's output.** The division is enforced, not optional.
- Variables and credentials come from the **workspace**, so your laptop needs no cloud credentials at all.
- **Saved plans work remotely** too, for workspaces without a linked repository, from CLI v1.6.

> ON THE EXAM
>
> No cloud credentials on the workstation is the quiet security benefit. The engineer authenticates to HCP Terraform, and nothing else.

Agents

- **A lightweight agent you run inside your own network**, which polls HCP Terraform for work.
- For infrastructure the service **cannot reach**: private networks, on-premises systems, isolated environments.
- **The connection is outbound**, so no inbound access to your network is required.
- You keep policy enforcement, run history and variable management, unlike local execution mode.
- It is a paid feature rather than part of the free offering. Module twenty-three covers editions.

> ON THE EXAM
>
> Agent mode is the answer for unreachable infrastructure. Local mode is not, because it gives up the governance features too.

Dynamic credentials

- **Short-lived credentials issued per run**, rather than long-lived keys stored as workspace variables.
- HCP Terraform authenticates to your cloud, which trusts it, and the credential expires with the run.
- **Nothing long-lived is stored**, so there is no key to leak, rotate or forget.
- This is module two's advice and module thirteen's ephemerality, provided as a managed feature.
- It requires remote execution, since the credential is issued to the run environment.

> ON THE EXAM
>
> Storing a long-lived access key as a workspace variable works and is the weaker option. Dynamic credentials remove the secret entirely.

A workspace created by init

- **Name a workspace that does not exist and init creates it**, telling you it has done so.
- **No variables are created by default**, unless your organisation has a global variable set.
- **HCP Terraform will use *.auto.tfvars files if present**, but workspace-specific values usually still need setting.
- **Execution mode defaults to Remote**, and the workspace is not connected to any repository.
- HashiCorp's warning: an automatically created workspace **might not be immediately ready to use**.

> ON THE EXAM
>
> A successful init and a working workspace are different things. The first apply often fails on missing variables.

Migrating to HCP Terraform

- **Replace the backend block with a cloud block**, then run terraform init.
- Terraform offers to **migrate existing state** into the workspace, as with any backend change.
- **Back up your state first.** Module sixteen's advice applies unchanged.
- Set workspace variables **before the first run**, since the configuration no longer reads your local environment.
- The **CLI-driven workflow is the gentlest first step**, because the commands you type do not change.

> ON THE EXAM
>
> The migration is a backend migration with extra setup. What changes afterwards is where credentials and variables live.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
Policies stopped being enforced	Execution mode was set to Local
The repository should trigger runs	The VCS-driven workflow
Infrastructure we cannot reach from the internet	An agent, not local execution mode
Engineers should not hold cloud credentials	Remote execution, and dynamic credentials
The first apply failed on missing variables	An auto-created workspace has none
Custom tooling starts our deployments	The API-driven workflow

ON THE EXAM

Separate the two questions: what starts the run, and where it executes. Most confusion here is one being read as the other.

> An organisation runs Sentinel policies on every workspace to enforce tagging and instance size limits. One team found remote runs slow to debug and changed their workspace's execution mode to Local.
>
> Three months later an audit finds that team's workspace has deployed resources violating two policies. No policy failures were recorded, and no runs errored.

What happened?

A The policies were misconfigured and silently passed everything

B Local execution mode disables remote operations, and policy enforcement depends on them

C Sentinel policies apply only to VCS-driven workspaces

D The team used -auto-approve, which bypasses policy checks

◆ PAUSE AND DECIDE

What happened?

A Misconfigured policies

They worked everywhere else in the organisation. One workspace differing points at a workspace setting, not at the policies.

B Local disables remote operations

Correct. HashiCorp lists policy enforcement, cost estimation and notifications as features requiring remote execution.

C VCS-driven only

Policies apply across workflows. What matters is the execution mode, not what triggered the run.

D -auto-approve bypasses

It skips the interactive confirmation, not the policy stage. A policy failure would still stop the run.

THE SIGNAL

Nothing errored, which is the tell. The policies were not failing; they were never being evaluated.

> A team manages Kubernetes clusters that sit in a private data centre with no inbound internet access and no route from HCP Terraform's runners. They want policy enforcement, run history and centrally managed variables.
>
> One engineer suggests setting execution mode to Local so runs happen on a jump host inside the network.

What should be used instead, and why?

A. Local execution mode is correct. It is the only way to reach private infrastructure

B. An HCP Terraform agent, which runs inside the network and keeps the governance features

C. Open inbound firewall rules so HCP Terraform's runners can reach the clusters

D. The API-driven workflow, which does not require network access

◆ PAUSE AND DECIDE

What should be used instead, and why?

A Local mode

Would reach the clusters and would give up the three things they explicitly asked for. Scenario 1 is this decision playing out.

B An agent

Correct. The agent runs inside the network with an outbound connection, and runs remain HCP Terraform runs with policies, history and variables intact.

C Open inbound rules

Solves a networking constraint by removing it, on a private data centre, which is the opposite of what agents exist to avoid.

D API-driven

Confuses workflow with execution mode. An API-driven run still executes somewhere, and by default that is HCP Terraform's infrastructure.

THE SIGNAL

Local and Agent both move execution off HCP Terraform's runners. Only the agent keeps the run under HCP Terraform's control.

> An engineer adds a cloud block naming a workspace that does not exist yet and runs terraform init. Terraform reports that it created the workspace, and init succeeds.
>
> She immediately runs terraform apply. The run starts remotely and fails, reporting that a required variable has no value. The same configuration applies without issue on her machine.

What is the explanation?

A The workspace is still being provisioned and needs a few minutes

B A new workspace has no variables. Her local environment supplied them before, and remote runs do not see it

C The cloud block does not support variables and she must switch to a backend block

D terraform init must be run a second time to synchronise variables

◆ PAUSE AND DECIDE

What is the explanation?

A Still provisioning

HashiCorp does warn a new workspace 'might not be immediately ready', meaning settings need configuring - not that time alone fixes it.

B No variables, and no local environment

Correct. No variables are created by default, and a remote run executes on HCP Terraform's infrastructure with no sight of her shell.

C cloud does not support variables

Workspace variables are a core feature. The cloud block itself takes no interpolation, which is a different restriction.

D Run init twice

Invents a synchronisation step. Init connects the directory to the workspace; it never uploads environment variables.

THE SIGNAL

The planted detail is that it works locally. That is what proves the value was coming from her environment all along.

> A regulated team wants their Git repository to be the single source of truth. Every change must be reviewed as a pull request with a plan attached, and nobody should be able to apply from a workstation.
>
> They currently use the CLI-driven workflow with remote execution.

What should they change?

A Nothing. Remote execution already prevents applying from a workstation

B Move to the VCS-driven workflow, so runs trigger from the repository and pull requests get speculative plans

C Switch execution mode to Agent, which enforces repository-based runs

D Move to the API-driven workflow, which is the only one supporting approvals

◆ PAUSE AND DECIDE

What should they change?

A Nothing needed

Remote execution changes where a run happens, not who can start one. An engineer can still type terraform apply and trigger a real run.

B VCS-driven workflow

Correct. Runs trigger from the repository, pull requests receive speculative plans, and the branch becomes the source of truth.

C Agent mode

Execution mode again, answering a question about triggering. An agent changes where a run executes and nothing about what starts it.

D API-driven only

Invents an exclusivity. Approvals are available across workflows, and the API-driven route would mean building tooling for no reason.

THE SIGNAL

Option A is the sharp one. Remote execution moves the run off the laptop; it does not stop the laptop starting it.

Module 21 recap

- **Three workflows:** UI and VCS-driven, CLI-driven, API-driven. What starts a run.
- **Three execution modes:** Remote, Local, Agent. Where it runs.
- **Local mode makes the workspace a state store only**, disabling policies and variables.
- **Agent mode keeps the governance features** while running inside your network.
- **The cloud block replaces the backend block** and takes no interpolation.
- **A new workspace has no variables**, and remote runs cannot see your local environment.

> ON THE EXAM
>
> Remote execution means your workstation holds no cloud credentials. Dynamic credentials go further and store no key at all.

MODULE 22 · DOMAIN 8 - HCP TERRAFORM

HCP Terraform: Workspaces, Projects, Variable Sets

The unit of organisation, the way to group it, and how values reach a run.

DOMAIN 8 of 8 COURSE WEIGHT 12% MODULE 22 of 27

admin@sthithapragna.com

A workspace is the unit of everything. A project is how you avoid managing two hundred of them one at a time.

What an HCP Terraform workspace holds

- **One Terraform configuration**, from a repository, an upload, or the CLI.
- **One state file**, versioned, with every previous version retained.
- **Its own variable values**, including secrets, stored by the service.
- **Its own run history**: every plan and apply, who started it, and who approved it.
- **Its own permissions**, so who can plan and who can apply are separate questions.

> ON THE EXAM
>
> Module eighteen's contrast: a CLI workspace is a state file, and an HCP workspace is all of this. Learn both definitions.

Workspace settings

- **Execution mode**: Remote, Local or Agent. Module twenty-one, and the largest consequences.
- **Terraform version**, pinned per workspace, so upgrades can be staged.
- **VCS connection** and branch, plus a **working directory** for monorepos.
- **Auto-apply**, which applies a successful plan with no approval. Rarely right for production.
- **Notifications**, and run tasks calling an external service during a run.

> ON THE EXAM
>
> The working directory setting is how a monorepo of configurations maps onto many workspaces from one repository.

Projects

- **Projects group workspaces**, so their access, variables and settings can be managed collectively.
- **Permissions apply at the project level**, so a team gets access to a group rather than to each workspace.
- A workspace belongs to **one project**, and can be moved between them in its settings.
- **Variable sets can be scoped to a project**, so every workspace in it inherits those values automatically.
- **Including future ones**, which is what makes self-service workflows possible.

> ON THE EXAM
>
> The 'and future ones' part is the point. A new workspace in the project inherits credentials and permissions without anyone configuring it.

Why that enables self-service

- Create a project for a team, and apply a **variable set with their cloud credentials** to it.
- Grant the team **permission to create workspaces** within that project.
- **Every workspace they create inherits the credentials and the permissions**, with no platform team involvement.
- HashiCorp names this directly: broader variable set scope **enables self-service workflows**.
- Without projects, each new workspace needs credentials and permissions configured by hand.

> ON THE EXAM
>
> This is objective 8c's real point. Projects are not just folders; they are the boundary that permissions and variables attach to.

Terraform variables or environment variables?

TERRAFORM VARIABLES

Inputs to the configuration

- Correspond to variable blocks in your code
- Referenced as var.name
- An HCL toggle allows complex types
- The equivalent of a .tfvars entry

ENVIRONMENT VARIABLES

Set in the run environment

- Exported into the shell running Terraform
- Used for provider credentials
- Also TF_LOG and other TF_ settings
- Not visible to var.name

THE TIEBREAKER

Both live in the same workspace screen and behave differently. Credentials belong in the environment category.

Sensitive variables

- Either kind can be marked **sensitive**, which is how credentials are stored in a workspace.
- **A sensitive value is write-only in the interface.** You can replace it; you cannot read it back.
- So the service holds the secret, rather than a file, a pipeline configuration, or somebody's laptop.
- **Dynamic credentials go further**, storing no long-lived secret at all. Module twenty-one.
- **The value still reaches state if a provider returns it.** Module four's rule has not changed.

> ON THE EXAM
>
> Storing a secret in a workspace solves distribution, not persistence. That distinction is module thirteen's, and it still applies here.

Variable sets

- **A reusable collection of Terraform and environment variables**, created once and applied to many places.
- Variables in a set **behave exactly like workspace variables**: available during runs, can be sensitive, support HCL types.
- The obvious use: **one set of cloud credentials**, applied wherever they are needed.
- Rotating a credential then means **editing one variable set**, not every workspace.
- They are created at the **organisation level**, then scoped to where they should apply.

> ON THE EXAM
>
> The value is in rotation and consistency. One set edited once, instead of the same secret pasted into forty workspaces.

The three variable set scopes

SCOPE	APPLIES TO
Global	All current and future workspaces in the organisation
Project-scoped	All current and future workspaces in the selected projects
Workspace-scoped	Only the workspaces you select

ON THE EXAM

Global and project scopes both include future workspaces. That is what makes them useful, and what makes them worth reviewing.

Precedence: the rules that matter

- **Workspace-specific variables always overwrite variables from variable sets** with the same key.
- **Workspace-scoped and project-scoped sets take precedence over global sets.**
- **Workspace-specific variables and workspace-scoped sets beat project-scoped sets.**
- The pattern: **the more specific the scope, the higher the precedence**.
- So a global default can be overridden for one workspace, which is the intended design.

> ON THE EXAM
>
> More specific wins. That is the opposite direction from module eight's file ordering, and the one exception is on the next slide.

When two sets of the same scope collide

- **Two variable sets at the same scope can define the same key.** Something has to break the tie.
- **HCP Terraform compares the variable set names and uses lexical order.**
- So a set named starting with A wins over one starting with B, for a conflicting key.
- **Not the most recently edited one.** Editing a set does not change which one wins.
- Ordering is by **Unicode code point**, so it is a genuine string comparison rather than anything cleverer.

> ON THE EXAM
>
> A conflict resolved by alphabetical order is invisible unless you know the rule. Prefer not creating the conflict.

Priority variable sets

- A variable set can be marked as a **priority** set, which inverts the usual direction.
- **Its values overwrite variables with the same key set at more specific scopes.**
- **Including command line flags, .auto.tfvars files and terraform.tfvars.**
- The use case: an organisation enforcing a value that individual workspaces must not override.
- **It does not guarantee the configuration uses the variable.** Somebody can hardcode a value instead.

> ON THE EXAM
>
> HashiCorp's own advice: for stricter enforcement use policy checks or run tasks. A priority variable set is not a control.

Run triggers

- **A run trigger links two workspaces**, so a successful apply in one queues a run in another.
- The use case: a networking workspace applies, and the workspaces that depend on it re-plan.
- Values usually flow between them through **remote state outputs**, as in module seven.
- **It queues a run; it does not apply one.** Approval still follows the destination workspace's settings.
- This is how a large estate is split into several workspaces without losing the ordering between them.

> ON THE EXAM
>
> Splitting one configuration into several workspaces was module five's advice about -target. Run triggers are what keeps them coordinated.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
One workspace should differ from the default	A workspace-specific variable. More specific wins
Rotate a credential used by forty workspaces	One variable set, edited once
Let a team create its own workspaces	A project, with a variable set scoped to it
Two sets define the same key	Lexical order of the set names decides
Engineers must not override this value	A priority variable set, with its caveat
Re-plan downstream when networking changes	A run trigger, plus remote state outputs

ON THE EXAM

Most questions here reduce to scope. Ask how many workspaces the value should reach, and whether it must be overridable.

> A platform team creates a global variable set defining instance_type as a small size, so every workspace gets a sensible default. Production workspaces need a larger size, so an engineer sets a workspace-specific variable with the same key on each of them.
>
> A colleague worries the global set will override the production values.

What actually happens?

A The global variable set wins, because it is defined at the organisation level

B The workspace-specific variable wins. Workspace variables always overwrite variable set values

C The run fails with a conflicting variable definition error

D Whichever was edited most recently wins

◆ PAUSE AND DECIDE

What actually happens?

A The global set wins

Backwards. Organisation level means broadest scope, and the broadest scope has the lowest precedence unless the set is marked as priority.

B The workspace variable wins

Correct. HashiCorp states workspace-specific variables always overwrite variables from variable sets with the same key.

C A conflict error

Overriding is the intended design, not an error. HCP Terraform shows which value it will use rather than refusing to run.

D Most recently edited

The common assumption, and wrong even where a tie-break is needed. Same-scope conflicts are broken by lexical order of set names.

THE SIGNAL

Set a default broadly, override it narrowly. That is what the precedence order is designed to allow.

> A new squad needs to manage its own infrastructure. They will create roughly one workspace per service, expected to reach thirty over a year. Each needs the same cloud credentials and the same team permissions.
>
> The platform team currently configures credentials and permissions on each new workspace by hand, which takes about a day per request.

What should the platform team set up?

A. A global variable set, so every workspace in the organisation gets the credentials

B. A project for the squad, a variable set scoped to it, and permission to create workspaces there

C. A workspace-scoped variable set listing all thirty workspaces in advance

D. A single shared workspace with CLI workspaces inside it for each service

◆ PAUSE AND DECIDE

What should the platform team set up?

A A global variable set

Would give one squad's credentials to every workspace in the organisation, including other teams'. Broad scope is the wrong tool for a bounded group.

B A project and a scoped set

Correct. Project scope covers current and future workspaces, so each new one inherits credentials and permissions with no ticket.

C List thirty in advance

They do not exist yet, and the list would need editing on every creation. It reintroduces the manual step it was meant to remove.

D CLI workspaces inside one

Module eighteen: a CLI workspace shares one configuration, one credential set and one permission boundary. It is not an environment or a service.

THE SIGNAL

'Current and future workspaces' is the phrase that decides this. Only global and project scopes include workspaces that do not exist yet.

> Two project-scoped variable sets are applied to the same project. One is named aws-shared and defines a region as one value. The other is named zz-overrides, created later specifically to change that region, and defines the same key with a different value.
>
> Runs continue to use the original region. The engineer who created zz-overrides edits it again, with no effect.

Why?

A Project-scoped sets cannot define the same key twice, so the second is ignored entirely

B Same-scope conflicts are resolved by lexical order of the set names, and aws-shared sorts first

C Variable sets are cached, and the project must be re-saved to pick up edits

D The newer set needs to be marked sensitive before it takes effect

◆ PAUSE AND DECIDE

Why?

A Second set ignored

It is not ignored; it is applied and then loses the tie. Its other, non-conflicting variables take effect normally.

B Lexical order decides

Correct. HCP Terraform compares variable set names and uses the one with lexical precedence. 'aws-shared' sorts before 'zz-overrides'.

C Caching

Invents a mechanism. Edits take effect on the next run; the edited value is simply losing the same tie every time.

D Mark it sensitive

Sensitivity controls whether a value can be read back in the interface. It has no bearing on precedence.

THE SIGNAL

Naming a set to sort last is exactly backwards. The correct fix is a workspace-specific variable, or a priority set.

> A compliance team must ensure every workspace tags resources with a cost centre from an approved list. Engineers currently pass their own value with -var, and some workspaces override the organisation default.
>
> The compliance team asks whether a priority variable set will guarantee the tag is correct everywhere.

What is the accurate answer?

A. Yes. A priority variable set overrides everything, including -var, so the value is guaranteed

B. It will override -var and tfvars files, but cannot stop somebody hardcoding a value in the configuration

C. No. Priority variable sets affect only environment variables, not Terraform variables

D. Yes, provided every workspace is moved into a single project first

◆ PAUSE AND DECIDE

What is the accurate answer?

A Guaranteed

Overstates it. HashiCorp says a priority set cannot guarantee the configuration uses the variable at all - only what the variable's value would be.

B Overrides inputs, not code

Correct, and it is HashiCorp's own caveat. For stricter enforcement they recommend policy checks or run tasks.

C Environment variables only

False. Variable sets hold both kinds, and priority applies to both.

D One project first

Scope affects reach, not the strength of the override. It also does not address the hardcoding gap.

THE SIGNAL

The gap is real: a priority set controls the variable's value, and a hardcoded literal never reads the variable. Policy is the control.

Module 22 recap

- **A workspace holds configuration, state, variables, run history and permissions.**
- **Projects group workspaces** so access, variables and settings are managed collectively.
- **Global and project scopes include future workspaces.** That is what enables self-service.
- **More specific scope wins**, and workspace variables always beat variable sets.
- **Same-scope conflicts are broken alphabetically**, not by which was edited last.
- **A priority set inverts the order**, and still cannot stop a hardcoded value.

> ON THE EXAM
>
> Run triggers queue a run in a downstream workspace after an upstream apply. They do not approve it.

MODULE 23 · DOMAIN 8 - HCP TERRAFORM

HCP Terraform: Governance, Policy and Editions

Rules the configuration author cannot edit, and the three editions the exam expects you to tell apart.

DOMAIN 8 of 8 COURSE WEIGHT 12% MODULE 23 of 27

admin@sthithapragna.com

Governance is a rule **the person writing the configuration cannot remove**. That is the whole distinction.

What governance covers here

- **Policy as code**, evaluated against a plan before an apply is permitted.
- **Run tasks**, which call an external service at defined points in a run.
- **Cost estimation**, shown between plan and apply.
- **Drift detection and health assessments**, which watch infrastructure between runs.
- **Teams, permissions, SSO and audit logging**, which govern who may do what.

> ON THE EXAM
>
> Every item here depends on remote execution, as module twenty-one established. Local execution mode disables most of this list.

Sentinel

- **HashiCorp's own policy as code framework**, with its own language, used across their products.
- Policies are evaluated **against the plan**, so they see what would happen before it happens.
- Typical rules: **required tags, permitted instance sizes, forbidden regions, mandatory encryption**.
- Policies are grouped into **policy sets** and applied to workspaces or projects.
- **They run between plan and apply.** A failing policy stops the apply, it does not undo anything.

> ON THE EXAM
>
> Sentinel sees the plan, not the configuration. It judges the proposed change rather than the code that produced it.

Open Policy Agent

- **OPA is the open source alternative**, using the Rego language, and HCP Terraform supports it.
- Attractive when an organisation **already uses OPA elsewhere**, for Kubernetes or APIs.
- It evaluates the same thing: the **plan**, before an apply is allowed.
- The two are **alternatives, not layers**. You would not normally run both against one workspace.
- The exam expects you to know both exist and that both are supported.

> ON THE EXAM
>
> Sentinel is HashiCorp's; OPA is the open source option. Knowing that both are supported is usually enough.

The three enforcement levels

LEVEL	WHAT A FAILURE DOES
Advisory	Logs a warning. The run continues and the apply proceeds
Soft-mandatory	Stops the run, and someone with permission may override it
Hard-mandatory	Stops the run. Nobody can override it; the configuration must change

ON THE EXAM

This is the most reliably tested item in Domain 8. Advisory warns, soft-mandatory can be overridden, hard-mandatory cannot.

Choosing a level

- **Advisory** for a new rule you are still testing, or guidance that should not block anyone.
- **Soft-mandatory** where exceptions are legitimate but should be deliberate and recorded.
- **Hard-mandatory** for genuine absolutes: regulatory boundaries, encryption, forbidden regions.
- The override on soft-mandatory is **recorded**, so an exception leaves a trail.
- **Making everything hard-mandatory** tends to produce a queue at the policy team, not better compliance.

> ON THE EXAM
>
> The question to ask is whether an exception could ever be legitimate. If yes, soft-mandatory. If never, hard-mandatory.

Run tasks

- **An integration point that calls an external service** during a run, at a defined stage.
- Stages include before and after the plan, and before and after the apply.
- Typical uses: **security scanning, cost tooling, change management, compliance checks**.
- The task can **pass or fail the run**, so a third-party tool gains the same gate a policy has.
- **Not the same as policy as code.** A run task calls out; a policy is evaluated inside HCP Terraform.

> ON THE EXAM
>
> Run tasks are how tools HashiCorp does not make get a say in whether an apply proceeds.

Cost estimation

- **Estimates the cost of resources in a plan**, hourly and monthly, with the monthly delta.
- **Disabled by default.** It is enabled at the organisation level, for all workspaces at once.
- When enabled, it appears as **an extra run phase, between plan and apply**.
- It supports resources in the **three major cloud providers**, and not everything is estimable.
- Estimates can be **fed into policy**, so a change above a threshold fails a check.

> ON THE EXAM
>
> It is an estimate. Some resources have no cost data, and usage-based pricing cannot be predicted from a plan.

Drift detection and health assessments

- **HCP Terraform can check workspaces on a schedule**, without anyone running anything.
- **Drift detection** compares state against reality and reports differences. Module seventeen's comparison, managed.
- **Continuous validation** re-runs your **check blocks** on the same schedule. Module twelve.
- Together these are called **health assessments**, and results appear on the workspace.
- **Detection, not remediation.** Nothing is applied automatically; a human still decides.

> ON THE EXAM
>
> Module twelve said check blocks warn rather than block. Continuous validation is what makes that warning useful over time.

The private module registry, revisited

- Module fifteen covered publishing. Its governance value is **controlling what modules exist**.
- A curated registry means teams compose from **approved building blocks** rather than starting from raw resources.
- Combined with policy, you get **defaults from modules and limits from policy**.
- **No-code provisioning** lets people deploy from a module without writing configuration at all.
- Curation is not enforcement. A module can be ignored; a policy cannot.

> ON THE EXAM
>
> Modules make the right thing easy. Policy makes the wrong thing impossible. Most organisations need both.

Teams, permissions and identity

- **Organisations contain projects, workspaces and teams.** Permissions attach to teams, not individuals.
- Permissions are granular: **read, plan, write, admin**, and separately for variables and state.
- **Being able to plan and being able to apply are different permissions**, which is the point.
- **SSO** connects an existing identity provider, so leavers lose access automatically.
- **Audit logging** records who did what, which is what most compliance requirements actually ask for.

> ON THE EXAM
>
> Separating plan from apply is the permission model's central idea. Everyone can see the change; few can make it.

The three editions

EDITION	WHAT IT IS
Community	The free CLI you download and run yourself. No service, no UI, no policy
HCP Terraform	HashiCorp's managed SaaS, in tiers, with governance features
Terraform Enterprise	The same capabilities, self-hosted in your own infrastructure

ON THE EXAM

The exam's own description says a candidate can identify Terraform Enterprise features and distinguish them from Community edition.

HCP Terraform or Terraform Enterprise?

HCP TERRAFORM

HashiCorp runs it

- Nothing to install or operate
- Upgrades handled for you
- Your state lives in their service
- The usual choice unless something forbids it

TERRAFORM ENTERPRISE

You run it

- Installed in your own infrastructure
- Full control of data residency
- Works in an isolated network
- You own operations and upgrades

THE TIEBREAKER

The deciding factors are data residency, network isolation and regulatory constraint. Not feature availability.

What Community edition does not have

- **No run environment.** Runs happen wherever you type the command.
- **No policy as code, no run tasks, no cost estimation.** Those are service features.
- **No teams, permissions, SSO or audit logging.** Those come from the platform's identity model.
- **No managed state.** You configure and operate a backend yourself. Module sixteen.
- **The language and the CLI are identical.** Everything in domains one to seven applies unchanged.

> ON THE EXAM
>
> The distinction is a service, not a language. A question about a feature that requires a run environment is asking about the paid editions.

Reading the scenario

THE WORDING SAYS	IT IS POINTING AT
This must never be bypassed	Hard-mandatory enforcement
Exceptions with approval and a record	Soft-mandatory enforcement
Warn us but do not block anyone	Advisory enforcement, or a check block
Our security scanner must gate the apply	A run task, not a policy
Data must stay inside our own network	Terraform Enterprise, self-hosted
Tell us when a certificate is expiring	Continuous validation, on check blocks

ON THE EXAM

Ask whether the requirement is a rule, an integration, or an observation. Each maps to a different feature.

> A security team defines three rules. Resources must never be created in an unapproved region, because of a legal data residency requirement. Instances should normally be within an approved size list, though a documented exception is occasionally justified. And a naming convention should be encouraged but must not block anyone.
>
> They ask which enforcement level applies to each.

What is the correct mapping?

A All three hard-mandatory, since all three are organisation policy

B Region hard-mandatory, instance size soft-mandatory, naming advisory

C Region soft-mandatory, instance size advisory, naming hard-mandatory

D All three advisory, with a run task to enforce the region rule

◆ PAUSE AND DECIDE

What is the correct mapping?

A — All hard-mandatory

Ignores the language. 'Occasionally justified' and 'must not block anyone' both describe exceptions, which hard-mandatory forbids entirely.

B — Hard, soft, advisory

Correct. Never means hard-mandatory; a justified exception means soft-mandatory; encouraged but non-blocking means advisory.

C — Inverted

Reverses the strongest and weakest requirements, making a legal boundary overridable and a naming convention absolute.

D — All advisory plus a run task

Advisory never blocks, so the residency rule would be unenforced. A run task could gate it, and Sentinel already does this natively.

THE SIGNAL

The scenario states each level in plain English. 'Never', 'occasionally justified', 'must not block' map one to one.

> A team already uses check blocks to assert that TLS certificates in their configuration have more than thirty days until expiry. The assertions run whenever somebody plans, which for some workspaces is once a quarter.
>
> They want to be told about an approaching expiry without waiting for the next plan.

What should they enable?

A. Continuous validation, which re-runs check blocks on a schedule

B. A hard-mandatory Sentinel policy on certificate expiry

C. Auto-apply, so workspaces run more often

D. A scheduled pipeline running terraform plan -refresh-only

◆ PAUSE AND DECIDE

What should they enable?

A Continuous validation

Correct. It re-runs exactly the check blocks they already wrote, on a schedule, and surfaces the results on the workspace.

B A hard-mandatory policy

Would block every apply once a certificate neared expiry, turning an advance warning into an outage. Module twelve's scenario 1, restated.

C Auto-apply

Applies successful plans automatically. It does not cause runs to happen, and it removes a safety step for no benefit here.

D Scheduled refresh-only

A sound drift detector from M17, and refresh-only detects state versus reality. It does not evaluate check blocks.

THE SIGNAL

They already wrote the logic. Continuous validation runs the same check blocks more often, without changing what failure does.

> A defence contractor must run all infrastructure tooling inside an isolated network with no internet egress. Regulation requires that state data never leaves their own facilities. They want policy enforcement, run history, teams and a private module registry.
>
> An architect asks which Terraform offering fits.

What should be recommended?

A Community edition, since the paid features cannot work in an isolated network

B Terraform Enterprise, self-hosted inside their own infrastructure

C HCP Terraform with agents, which keeps state inside their network

D HCP Terraform with Local execution mode, so nothing leaves the network

◆ PAUSE AND DECIDE

What should be recommended?

A Community edition

Gives up all four things they asked for. Isolation is a deployment constraint, not a reason to abandon the requirements.

B Terraform Enterprise

Correct. Self-hosted delivers the same capabilities inside their own boundary, which is exactly what data residency and isolation require.

C HCP Terraform with agents

*Agents solve reachability of infrastructure. **State would still live in HashiCorp's service**, which the regulation forbids.*

D Local execution mode

Module twenty-one: it removes policy enforcement and variable evaluation, and state still lives in the service anyway.

THE SIGNAL

Agents move execution; they do not move state. The requirement here is about where data rests, not where runs happen.

> A platform team wants every storage bucket to have encryption enabled. A module author proposes adding a precondition to their shared module that fails if encryption is off. The platform team proposes a Sentinel policy instead.
>
> Both would catch the case described. The team asks which is right.

What is the correct reasoning?

A. The precondition, because it fails earlier and gives a clearer message

B. The policy, because a module author can remove a precondition and cannot remove a policy

C. Either. They are equivalent mechanisms with different syntax

D. The precondition, because Sentinel cannot inspect resource arguments

◆ PAUSE AND DECIDE

What is the correct reasoning?

A Precondition, earlier

Both true, and neither decides it. A clear early failure is worth having; it is not an enforcement guarantee.

B Policy, because it cannot be removed

Correct. The distinction is ownership: a condition lives in the code it checks; a policy is owned by another team and evaluated outside.

C Equivalent

They catch the same case and differ in who controls the rule, which is the entire point of governance.

D Sentinel cannot inspect arguments

False. Sentinel evaluates the plan, which contains the resource arguments. That is precisely what it is for.

THE SIGNAL

Not either-or in practice. The module gives a fast, clear failure; the policy guarantees the rule survives someone deciding otherwise.

Module 23 recap

- **Advisory warns. Soft-mandatory can be overridden. Hard-mandatory cannot.**
- **Sentinel is HashiCorp's; OPA is the open source alternative.** Both evaluate the plan.
- **Run tasks call an external service** and can fail the run.
- **Health assessments** cover drift detection and continuous validation. Never remediation.
- **Community edition has the language and no service.**
- **Terraform Enterprise is self-hosted**, for data residency and isolated networks.

> ON THE EXAM
>
> Governance is a rule the configuration author cannot remove. That single test separates policy from every condition in module twelve.

MODULE 24 · EXAM STRATEGY - ALL DOMAINS

Confusion Pairs

Almost no new content. Everything here has been taught; the job now is telling them apart at speed.

COVERS All domains NEW CONTENT Almost none MODULE 24 of 27

admin@sthithapragna.com

Wrong answers are rarely wrong. They are usually **the right answer to a slightly different question**.

admin@sthithapragna.com

The one that costs most marks

CLI WORKSPACE

A state file selector

- Separate state inside one working directory
- One configuration, one backend, one credential set
- Switched with terraform workspace select
- Not appropriate for separate credentials

HCP TERRAFORM WORKSPACE

A managed environment

- Its own configuration and working directory
- Its own variables, credentials, permissions
- Its own run history and state versions
- The unit of organisation in HCP Terraform

THE TIEBREAKER

HashiCorp calls these 'distinctly different'. The word appears constantly in questions, so decide which kind is meant before reading the options.

The refresh family

WRITTEN AS	WHAT IT MEANS
`Default plan or apply`	Refreshes everything, then compares against configuration
`-refresh=false`	Skips reading reality. Faster, and the plan may be stale
`plan -refresh-only`	Reads reality and proposes no infrastructure changes
`apply -refresh-only`	Writes those findings into state. Adopts drift
`terraform refresh`	Deprecated since 0.15.4. No preview, no prompt

> ON THE EXAM
>
> -refresh=false ignores reality. -refresh-only looks at nothing else. Opposite meanings, one shared word.

Changing state without changing infrastructure

MECHANISM	WHAT IT DOES
`moved block`	Same object, new address. Terraform keeps managing it
`removed, destroy = false`	Terraform stops managing it. The object survives
`removed, destroy = true`	Terraform stops managing it and destroys it
`import block`	An existing object comes under management
`state mv / state rm`	The same jobs, immediately, with no plan and no review

ON THE EXAM

The lifecycle block inside removed is required, and its destroy argument is the difference between a refactor and a deletion.

Ordering, and forcing a rebuild

MECHANISM	WHAT IT ACTUALLY DOES
`An attribute reference`	Creates the dependency implicitly. Almost always the right answer
`depends_on`	Adds ordering only. Says after; never says again
`replace_triggered_by`	Replaces this resource when the referenced thing changes
`-replace=ADDRESS`	Replaces one instance now, visibly in the plan
`terraform taint`	Deprecated. Marked state directly with no preview

ON THE EXAM

depends_on says after. replace_triggered_by says again. -replace says now. Three different requests.

The four ways to assert something

MECHANISM	WHEN IT RUNS, AND WHAT FAILURE DOES
Variable validation	When variables are set. Blocks. Since 1.9 can see other variables
Precondition	Before the block is provisioned. Blocks. Cannot use self
Postcondition	After it is provisioned. Blocks. Can use self
Check block	Last step of plan or apply. WARNS, and continues
Sentinel or OPA policy	Between plan and apply. Owned by another team

ON THE EXAM

Only the check block lets the operation continue. Only a policy cannot be removed by the person writing the configuration.

The secrets family

MECHANISM	WHAT IT CONTROLS
`sensitive = true`	Display only. The value is still stored, in plain text
`ephemeral`	Lifetime. Never written to the plan artefact or state
`Write-only argument`	Lets an ephemeral value reach a managed resource. Not persisted
`_wo_version`	IS stored in state. Incrementing it accepts a new value
`Dynamic credentials`	No stored secret at all. Issued per run, expires with it

ON THE EXAM

State, saved plan files and TRACE logs all hold secrets in plain text. sensitive has never affected any of them.

Repeating things

MECHANISM	WHAT IT REPEATS, AND HOW IDENTITY WORKS
`count`	A resource. Identity is a position, from zero
`for_each`	A resource. Identity is a key, from a map or a set of strings
`dynamic block`	A nested block inside one resource. Not the resource itself
`for expression`	Builds a value. Creates no resources at all
`count = 0`	The conditional resource pattern. There is no if statement

ON THE EXAM

count numbers, for_each names. Keys and counts must be known at plan time, because they become addresses in the graph.

Three version numbers, three mechanisms

WHAT IS BEING PINNED	HOW
Terraform itself	required_version, in the terraform block
A provider	version, inside required_providers. Recorded in the lock file
A registry module	version, on the module block. NOT recorded anywhere
A Git-sourced module	The ref argument. version is rejected
A local path module	Nothing. Whatever is on disk is what runs

ON THE EXAM

The lock file records providers only. A module constraint is the entire pinning mechanism, which is why exact pins matter there.

The HCP Terraform family

THE WORD	WHAT IT ACTUALLY IS
Workflow	What starts a run: VCS, CLI or API
Execution mode	Where it runs: Remote, Local or Agent
Local execution mode	The workspace becomes a state store. Policies stop applying
Agent mode	Runs inside your network, and stays an HCP Terraform run
Terraform Enterprise	Self-hosted. The whole platform inside your boundary

ON THE EXAM

Agents move execution. They do not move state. Only Terraform Enterprise keeps state inside your own infrastructure.

Files that sound alike

FILE	COMMITTED?
`.terraform/`	No. A disposable working directory. Recreated by init
`.terraform.lock.hcl`	Yes. Records provider selections and checksums
`terraform.tfstate`	No. And treat it as sensitive data
`terraform.tfvars`	Auto-loaded. Never put secrets in it
`prod.tfvars`	NOT auto-loaded. Needs -var-file
`tfplan`	No. Holds the configuration and secrets in cleartext

ON THE EXAM

Two names differ by four characters and have opposite treatment: the .terraform directory is ignored, the lock file is committed.

Language pairs worth a second look

PAIR	THE DIFFERENCE
`resource vs data`	Ownership. Ask what should happen on destroy
`variable vs local vs output`	In, stays put, out. That table answers most design questions
`list vs set`	Ordered with duplicates, versus neither. Drives for_each
`try vs can`	try returns a value; can returns a boolean
`file vs templatefile`	file returns contents verbatim; templatefile renders values in
`validate vs plan`	Structure using local schemas, versus reality using credentials

ON THE EXAM

None of these is subtle once stated. All of them are easy to get backwards at speed, which is exactly what the exam measures.

Precedence, in both directions

CONTEXT	WHICH WAY IT RUNS
CLI variables	Weakest to strongest: default, TF_VAR_, tfvars, auto.tfvars, CLI
The trap there	TF_VAR_ is second weakest. Files beat environment variables
HCP variable sets	More specific scope wins. Workspace beats project beats global
The exception	A priority variable set overrides more specific scopes, including -var
Two sets, same scope	Lexical order of the set names. Not the most recently edited

ON THE EXAM

The CLI order and the HCP scope order are different systems. A priority variable set inverts the CLI order entirely.

> A workspace must have its own cloud credentials, its own list of who may apply, and a record of every run for audit.
>
> An engineer says this is what workspaces are for and proposes creating one.

Which kind of workspace satisfies this?

A. A CLI workspace, created with terraform workspace new

B. An HCP Terraform workspace

C. Either. Both isolate credentials and permissions

D. Neither. Credentials and permissions are always organisation-wide

◆ PAUSE AND DECIDE

Which kind of workspace satisfies this?

A — A CLI workspace

Isolates state and nothing else. Credentials, permissions and history are all shared with every other workspace in that directory.

B — An HCP workspace

Correct. Its own variables, credentials, permissions, run history and state versions is precisely the list of what an HCP workspace holds.

C — Either

The assumption the ambiguity exists to catch. HashiCorp calls the two distinctly different.

D — Neither

Overcorrects. HCP Terraform's permission model is built on exactly this separation.

THE SIGNAL

Three requirements, three things a CLI workspace does not have. The requirement list is the definition of an HCP workspace.

> An engineer wants to record that somebody resized a database by hand, so Terraform's state matches reality. She must not change any infrastructure.
>
> The configuration still specifies the old size.

Which command?

A terraform apply -refresh=false

B terraform apply -refresh-only

C terraform apply

D terraform refresh

◆ PAUSE AND DECIDE

Which command?

A **-refresh=false**

The opposite. It skips reading reality, so it cannot record what reality now is.

B **-refresh-only**

Correct. It compares state against reality, proposes no infrastructure changes, and writes the findings into state once approved.

C **Plain apply**

Would revert the change, shrinking the database back to the size in the configuration.

D **terraform refresh**

Deprecated since 0.15.4, and it does this with no preview and no prompt. -refresh-only replaced it.

THE SIGNAL

Adopting drift is only half the job. The configuration still says the old size, so the next normal apply reverts it.

> A storage bucket is being handed to another department. It must keep existing with all its contents, and Terraform must stop managing it.
>
> The change should be reviewed in a pull request like any other.

What should be written?

A. Delete the resource block and apply

B. A moved block pointing at the other department's configuration

C. A removed block with lifecycle destroy set to false

D. prevent_destroy on the resource, then delete the block

◆ PAUSE AND DECIDE

What should be written?

A Delete and apply

Terraform destroys what is no longer declared. The bucket and its contents would go.

B A moved block

moved renames within one configuration's state. It cannot move an object into a different configuration owned by someone else.

C removed, destroy = false

Correct. It forgets the binding, leaves the object alone, and appears in the plan so it goes through review.

D prevent_destroy then delete

The rule dies with the block. Deleting the resource block removes the protection along with it, and the destroy proceeds.

THE SIGNAL

The pair here is moved versus removed. moved keeps a binding and renames it; removed deletes the binding.

> A generated database password must not appear in the state file. The team is on Terraform 1.12 with a current provider.
>
> An engineer has already marked the input variable as sensitive and is surprised the password is still in state.

What actually keeps it out?

A Marking the output sensitive as well

B An ephemeral resource feeding a write-only argument

C Enabling encryption at rest on the backend

D Adding ignore_changes on the password attribute

◆ PAUSE AND DECIDE

What actually keeps it out?

A Sensitive output too

More of the mechanism that already failed. sensitive controls display, not storage, wherever it is applied.

B Ephemeral plus write-only

Correct. Ephemeral values are never persisted, and a write-only argument is the door that lets one reach a managed resource.

C Encryption at rest

Worth doing, and it protects against stolen storage. Anyone with legitimate read access still sees plain text.

D ignore_changes

Stops Terraform proposing updates to an attribute. It does not stop the value being recorded.

THE SIGNAL

sensitive is display, ephemeral is lifetime. This pair appeared in five modules, which is a fair measure of how often it is tested.

> In a CLI-driven run against HCP Terraform, an engineer passes an instance size with -var. The organisation has a priority variable set defining the same key.
>
> She expects her command line value to win, since the command line has the highest precedence.

What happens?

A Her -var value wins. The command line always has highest precedence

B The priority variable set wins. It overrides more specific scopes, including -var

C The run fails with a conflicting variable error

D The values are merged, and the larger size is selected

◆ PAUSE AND DECIDE

What happens?

A **-var wins**

True in the CLI-only precedence order from module eight. A priority variable set is the documented exception that inverts it.

B **The priority set wins**

Correct. Priority sets overwrite variables with the same key at more specific scopes, including command line flags and tfvars files.

C **A conflict error**

Overriding is the intended behaviour throughout Terraform. HCP Terraform shows which value it will use.

D **Merged**

Terraform never merges scalar values. The highest-precedence source simply wins.

THE SIGNAL

Two precedence systems. Module eight's order is correct for the CLI, and a priority variable set inverts it inside HCP Terraform.

> A pipeline runs in an environment with no cloud credentials by design. It must reject a pull request that references an argument name which does not exist on a resource type.
>
> It must not require access to any cloud account.

What should the pipeline run?

A terraform plan, which is the only command that checks resource schemas

B terraform init -backend=false, then terraform validate

C terraform fmt -check -recursive

D A check block asserting the argument exists

◆ PAUSE AND DECIDE

What should the pipeline run?

A plan

Would catch it, and needs credentials and state. The requirement explicitly rules that out.

B init -backend=false, then validate

Correct. validate checks attribute names and types using locally installed schemas, and -backend=false avoids needing the backend.

C fmt -check

A formatter, not a linter. It will happily format a configuration that references arguments which do not exist.

D A check block

Warns rather than blocking, so it could not reject anything. It also runs during a plan, which needs credentials.

THE SIGNAL

validate needs init but not a backend, and no credentials at all. That combination is exactly what a module's CI wants.

Module 24 recap

- **Decide which kind of workspace a question means** before reading the options.
- **-refresh=false ignores reality. -refresh-only looks at nothing else.**
- **moved renames, removed forgets, import adopts.** The destroy argument decides the rest.
- **sensitive is display; ephemeral is lifetime.** Five modules made this point.
- **Only a check block warns. Only a policy cannot be deleted by the author.**
- **Two precedence systems.** The CLI order, and HCP scope with its priority exception.

> ON THE EXAM
>
> A distractor is usually a correct statement about something adjacent. Identify the pair, then answer the question you were asked.

MODULE 25 · EXAM STRATEGY - ALL DOMAINS

Version Currency: 1.12 Against Today

What the exam tests, what you will actually download, and the rules that changed in between.

COVERS Cross-cutting EXAM VERSION Terraform 1.12 MODULE 25 of 27

admin@sthithapragna.com

The exam is a **snapshot**. The documentation is not. Where they disagree, the exam wins.

What the exam is pinned to

- **HashiCorp states the exam tests Terraform 1.12.** It is on the certification page, not inferred.
- **The official exam content list links to /v1.12.x/ documentation paths**, deliberately.
- So the pinning is not incidental. HashiCorp is telling you which documentation to revise from.
- **developer.hashicorp.com serves the latest version by default.** You have to select 1.12 yourself.
- That version selector is the single most useful thing on the site while revising.

> ON THE EXAM
>
> Following a link from the exam content list gives you 1.12 docs. Searching the site gives you current docs. Two different answers.

The timeline

VERSION	RELEASED, AND WHAT IT MEANS HERE
1.10	Ephemeral variables, outputs and resources. Examinable
1.11	Write-only arguments. S3 native locking GA. Examinable
1.12	The version this exam tests
1.13 and 1.14	Beyond the exam. 1.14 added list resources and terraform query
1.15	Dynamic module sources, variable deprecation, backend validation
1.16	Import blocks inside modules, a terraform_data store block

ON THE EXAM

Four minor versions separate the tested version from the current one. Terraform 1.16.1 was released on 2 September 2026.

Things you will meet that are not examinable

- **terraform query** and **list resources**, written in .tfquery.hcl files. Terraform 1.14.
- **actions blocks**, invoked with a flag. Terraform 1.14.
- **Dynamic module sources**, using a const attribute on a variable. Terraform 1.15.
- **Variable and output deprecation**, with a deprecated attribute. Terraform 1.15.
- **A store block in terraform_data**, holding ephemeral values across plan and apply. Terraform 1.16.

> ON THE EXAM
>
> These are real features you will see in current documentation. None of them can appear as a correct answer on this exam.

Rules this course taught that later versions relaxed

- **Module fourteen:** a module source cannot use variables. **Relaxed in 1.15** with the const attribute.
- **Module nineteen:** import blocks sit in the root module. **Relaxed in 1.16**, which allows them inside modules.
- **Module six:** validate checks configuration structure. **Extended in 1.15** to validate the backend block too.
- **All three statements remain correct for 1.12**, which is what the exam asks about.
- This is what a version-pinned exam feels like from the inside. Answer for 1.12, and know why.

> ON THE EXAM
>
> Three absolute-sounding rules in this course have exceptions in versions you can download today. None of them applies to the exam.

Things that ARE examinable and older material misses

- **Provider-defined functions**, called with a namespaced syntax. Terraform 1.8.
- **Cross-object variable validation**, referencing other variables and data sources. Terraform 1.9.
- **Ephemeral values** (1.10) and **write-only arguments** (1.11).
- **S3 native locking** with use_lockfile, GA in 1.11.
- Plus the four topics HashiCorp names as new in 004, which are modules eleven, twelve, thirteen and twenty-two.

> ON THE EXAM
>
> This is the more dangerous direction. A candidate revising from a 003-era guide will simply not have seen most of this.

Replaced, and still widely taught

OLDER MATERIAL SAYS	CURRENT ANSWER
terraform taint	-replace on plan or apply. Deprecated since 0.15.2
terraform refresh	-refresh-only mode. Deprecated since 0.15.4
S3 plus a DynamoDB lock table	use_lockfile, writing a .tflock object. DynamoDB deprecated
The remote backend for Terraform Cloud	The cloud block. Since Terraform 1.1
Terraform Cloud	HCP Terraform. Renamed in 2024
terraform state mv and state rm	moved and removed blocks, which appear in a plan

ON THE EXAM

Every left-hand entry still works. That is exactly why they survive in runbooks and study guides years after being superseded.

Ownership, naming and licensing

- **IBM completed its acquisition of HashiCorp in June 2024.** Operations transitioned in 2025.
- HashiCorp states **no features or functionality changed** as a result.
- **Terraform Cloud became HCP Terraform** in 2024. The 004 objectives use the new name.
- **Terraform is under the Business Source Licence** since 2023. Source-available, not open source.
- **OpenTofu** is the Linux Foundation fork. Not examinable, and worth knowing exists.

> ON THE EXAM
>
> The exam uses HCP Terraform throughout. A question or a guide saying Terraform Cloud is describing the same product by an older name.

How to revise from the documentation safely

- **Start from the official exam content list**, and follow its links. They are pinned to 1.12.
- **Use the version selector** when you land on a page from a search. The default is current.
- **Watch for version notes.** Pages say when a feature arrived, which tells you if it is in scope.
- **Be sceptical of undated blog posts.** Most Terraform writing on the internet predates 1.10.
- **Check the changelog for anything you learn late.** One line tells you which release introduced it.

> ON THE EXAM
>
> A documentation page that says a feature was added in 1.14 has answered the only question that matters for this exam.

Reading the scenario

YOU ENCOUNTER	WHAT IT MEANS FOR THE EXAM
A feature note saying 'added in 1.14'	Out of scope. It cannot be a correct answer
A guide recommending a DynamoDB lock table	Written before 1.11. Still works, not current
A page saying 'Terraform Cloud'	Older naming. The product is HCP Terraform
A claim that Terraform has no custom functions	Out of date since 1.8. Provider functions exist
A module source referencing a variable	Valid from 1.15. Not valid in 1.12
taint, or the refresh command	Deprecated. Recognise them; do not choose them

ON THE EXAM

The question to ask about any fact is which release it belongs to. Before 1.12 is in scope; after is not.

> A candidate revising from developer.hashicorp.com reads about list resources and the terraform query command, which discover existing infrastructure and can generate import configuration in bulk.
>
> It looks directly relevant to the import objective, and they plan to memorise it.

What should they do?

A. Memorise it. It is in the official documentation, so it is in scope

B. Note it as useful at work, and skip it. It arrived in 1.14, after the tested version

C. Memorise it, since import is an exam objective and this is part of importing

D. Skip the import objective entirely, since the documentation has changed

◆ PAUSE AND DECIDE

What should they do?

A Official, so in scope

The documentation covers the current release. Being official says nothing about which version a page describes.

B Note it, skip it

Correct. It arrived in 1.14, two releases after the tested version, so it cannot be a correct answer on this exam.

C Import is an objective

The objective is examined against 1.12 behaviour. A newer mechanism for the same task is still newer than the exam.

D Skip the objective

Overcorrects badly. The 1.12 import mechanisms - the command, import blocks, config generation - are all squarely examinable.

THE SIGNAL

Being in the documentation and being in scope are different tests. The version note on the page answers the second one.

admin@sthithapragna.com

> A study guide bought second-hand teaches: use terraform taint to force replacement, create a DynamoDB table alongside an S3 bucket for locking, connect the CLI to Terraform Cloud with the remote backend, and notes that Terraform has no user-defined functions.
>
> A candidate asks which of these are still correct for the exam.

What is the accurate assessment?

A All four are correct. They are core Terraform behaviours

B All four are superseded, though each still describes something that works or once worked

C Only the DynamoDB advice is out of date. The rest are current

D All four are wrong and were never correct

◆ PAUSE AND DECIDE

What is the accurate assessment?

A All four correct

Each was correct at some point, which is why the guide says them. Taint and the remote backend are deprecated, DynamoDB locking is deprecated, and provider functions exist since 1.8.

B All four superseded

Correct. taint gave way to -replace, DynamoDB to use_lockfile, the remote backend to the cloud block, and 1.8 added provider-defined functions.

C Only DynamoDB

Picks the best-known of the four. The other three are equally superseded, and the naming change alone dates the guide.

D Never correct

Unfair and unhelpful. Each was the documented recommendation in its day, which is exactly why they persist.

THE SIGNAL

The fourth item is the subtlest. 'No user-defined functions in HCL' is still true; 'no custom functions at all' stopped being true in 1.8.

admin@sthithapragna.com

> A candidate sits the exam having also used Terraform 1.16 at work. A question asks how to make a module's source address vary by environment.
>
> At work they would use a variable marked const, which 1.15 introduced for exactly this.

How should they answer?

A Choose the const-based answer, since it is the modern approach

B Answer for 1.12: module sources take no variables, so use separate module blocks or a different structure

C Choose whichever option mentions a variable, since the question implies one is needed

D Skip the question, since the correct answer depends on the version

◆ PAUSE AND DECIDE

How should they answer?

A The modern approach

Correct at work on 1.15 or later, and wrong here. The exam tests 1.12, where a module source cannot reference a variable at all.

B Answer for 1.12

Correct. Knowing the newer feature exists is useful; answering with it on a version-pinned exam is not.

C Anything with a variable

Pattern-matching on a keyword rather than reading the options. It would likely select a distractor.

D Skip it

There is no negative marking and the 1.12 answer is knowable. Skipping a question you can answer is a guaranteed loss.

THE SIGNAL

Real experience with a newer version is an advantage everywhere except the few questions where the behaviour changed.

admin@sthithapragna.com

Module 25 recap

- **The exam tests Terraform 1.12**, and the objectives link to 1.12 documentation deliberately.
- **The current release is four minor versions ahead.** Use the version selector when you search.
- **query, actions, const module sources and deprecation attributes are all out of scope.**
- **Provider functions, cross-object validation, ephemeral values and native S3 locking are in scope.**
- **taint, terraform refresh, DynamoDB locking and the remote backend are superseded**, and still widely taught.
- **Where current behaviour and 1.12 disagree, answer for 1.12**, and know that you are choosing to.

> ON THE EXAM
>
> Three rules this course stated as absolute have exceptions in later versions. All three still hold for the exam.

MODULE 26 · EXAM STRATEGY - ALL DOMAINS

Exam Mechanics and Strategy

What HashiCorp publishes, what it does not, and how to spend sixty minutes.

COVERS The exam itself PRODUCT CONTENT None MODULE 26 of 27

admin@sthithapragna.com

Sixty minutes, one attempt, no free retake. **Pace is part of the exam**, not something that happens around it.

What HashiCorp publishes

FACT	VALUE
Exam name	HashiCorp Certified: Terraform Associate (004)
Terraform version tested	1.12
Duration	60 minutes
Format	Online proctored, delivered via Certiverse
Question types	Multiple choice, multiple answer, and true/false
Fee, language, validity	$70.50 plus local taxes. English only. Valid 2 years

ON THE EXAM

No free retake is included. A second attempt costs the full fee again, which is worth knowing before you book.

What HashiCorp does not publish

- **The number of questions.** Widely quoted figures of 57 or 60 do not come from HashiCorp.
- **The passing score.** Estimates of around seventy per cent are third-party guesses.
- **Domain weights.** This course's percentages are derived, and are stated as such.
- So do not build a strategy on a target score, because nobody outside HashiCorp knows what it is.
- **You do get an immediate pass or fail**, and an objective-level report shortly afterwards.

> ON THE EXAM
>
> Any source quoting a precise question count or pass mark is quoting somebody's estimate. Treat it as such.

The three question types

- **Multiple choice.** One correct answer from several. The most common form.
- **Multiple answer.** Two or three correct, and the question tells you how many.
- **True or false.** A single statement to judge.
- **The question bank behind this course is about one fifth true/false**, which is a fair guide to expect.
- There are **no labs, no code editors and no free text.** Everything is a selection.

> ON THE EXAM
>
> Nothing here requires you to write Terraform. It requires you to read it, and to judge statements about it.

Multiple answer questions

- **The question states how many to choose.** Read that number before reading the options.
- **Selecting the wrong count is a wasted question**, however good your reasoning was.
- Treat each option as its own true-or-false question, then count the trues.
- If you get more trues than the question wants, **one of them is subtly wrong**. Find it.
- If you get fewer, you have **ruled something out too quickly**. Reconsider your weakest rejection.

> ON THE EXAM
>
> Choose two means exactly two. The count is a constraint you can use, not just an instruction to follow.

True or false questions

- **The whole statement must be true.** One wrong clause makes the whole thing false.
- Read compound statements clause by clause, and stop at the first one that fails.
- Watch absolute words: **always, never, all, only, must, cannot**. They are often where a true statement breaks.
- **But absolutes are not automatically false.** Plenty of Terraform rules genuinely are absolute.
- Example: state **always** holds attribute values in plain text. True, and absolute.

> ON THE EXAM
>
> The old test-taking advice that absolutes are usually false does not hold here. Judge the claim, not the wording.

Reading a scenario question

- **Find the requirement first.** It is usually the last sentence, and it is what the options are judged against.
- **Planted details are constraints.** 'Must not cause downtime', 'no credentials available', 'nine workspaces'.
- **Several options often work.** The question asks which fits the stated requirement, not which is possible.
- **Eliminate rather than select.** Rejecting three options is usually easier than justifying one.
- If two survive elimination, **re-read the requirement**. One of them fails a detail you skimmed.

> ON THE EXAM
>
> This course's scenarios were written the same way. Every one has a sentence that eliminates an otherwise correct option.

Spending the hour

- **An hour is not long.** Whatever the question count, that is roughly a minute each.
- **Answer every question as you reach it**, even if you flag it. Never leave a blank behind you.
- **An unanswered question cannot earn a mark. A guess can.** Leaving one blank is a certain zero.
- **Flag and move on after about ninety seconds.** The next question may be one you know instantly.
- **Reserve the last five minutes** for flagged questions and for checking every item has an answer.

> ON THE EXAM
>
> HashiCorp does not publish scoring detail, and no scheme rewards a blank more than a guess. Answer everything.

The proctored setup

- **Delivered remotely via Certiverse**, with a live proctor, system checks and a room scan.
- **A valid government ID is required**, and the name must match your certification profile.
- **Run the system check days beforehand**, not on the day. Camera, microphone, browser, network.
- **Clear the desk**, and expect to show the room. No notes, no second screen, no phone.
- Accessibility accommodations are available, and need **requesting in advance**.

> ON THE EXAM
>
> Most preventable failures on the day are logistical: a mismatched name, an untested camera, a cluttered desk.

Afterwards, and staying certified

- **You get an immediate pass or fail**, with an objective-level report following shortly.
- **That report is worth reading even if you passed**, because it shows where your weakest domain was.
- **The credential is valid for two years.**
- Renew by **retaking the Associate exam**, from six months before expiry, or by passing the Professional exam.
- **There is no discounted renewal at Associate level.** A retake is the full fee.

> ON THE EXAM
>
> Passing the higher Terraform exam extends the Associate credential, which is worth knowing if you plan to continue.

The strategy, in one table

SITUATION	WHAT TO DO
A scenario question	Read the last sentence first, then eliminate
Two options both look right	Re-read the requirement. One fails a detail
A multiple answer question	Read the count, judge each option separately, then count
A true/false compound statement	Clause by clause. Stop at the first failure
Ninety seconds gone, still unsure	Answer it anyway, flag it, move on
Five minutes left	Revisit flags, and confirm nothing is unanswered

ON THE EXAM

None of this is about knowing more Terraform. It is about converting what you already know into marks.

> A question describes a team needing to stop Terraform managing a resource while keeping the object. Two options both achieve that: a removed block with destroy set to false, and terraform state rm.
>
> The last sentence of the scenario says the change must be reviewed in a pull request.

How should a candidate proceed?

A. Pick either. Both achieve the stated outcome

B. Re-read the requirement. Only one of the two is reviewable in a pull request

C. Pick the older mechanism, since exams usually test fundamentals

D. Flag it and guess later, since two options are technically correct

◆ PAUSE AND DECIDE

How should a candidate proceed?

A **Pick either**

Both achieve the outcome, and only one satisfies the requirement. That gap is what the last sentence exists to create.

B **Re-read the requirement**

Correct. A removed block appears in a plan and lives in configuration; state rm runs from a terminal with no review. The requirement decides it.

C **Prefer the older mechanism**

A guessing heuristic, not reasoning. It also happens to be wrong here, since the newer mechanism is the reviewable one.

D **Flag and guess**

Reasonable when genuinely stuck, and premature here. The deciding information is in the question and takes seconds to use.

THE SIGNAL

When two options both work, the requirement sentence is doing the work. This is how nearly every scenario in this course was built.

> A true or false question reads: marking a variable as sensitive prevents its value from being written to the state file, and suppresses it in plan output.
>
> A candidate knows the second half is true and is unsure about the first.

What is the answer, and why?

A. True. The second clause is correct, and the first is the intended meaning of sensitive

B. False. The whole statement must be true, and sensitive does not affect state

C. True, provided the backend is encrypted

D. Cannot be determined without knowing the Terraform version

◆ PAUSE AND DECIDE

What is the answer, and why?

A — True, second clause correct

A compound statement is true only if every clause is true. One correct half does not rescue an incorrect one.

B — False, on the first clause

Correct. sensitive controls display and never storage, so the first clause fails and the whole statement is false.

C — True if encrypted

Encryption protects the file at rest. The value is still written, and anyone with read access sees it.

D — Version-dependent

This has been true throughout. It is not a behaviour that changed between releases.

THE SIGNAL

This is the course's most-repeated misconception, dressed as a compound statement. Half-true reads as true unless you check each clause.

> A candidate has twelve minutes left and eleven questions remaining. Four of the remaining questions are long scenarios. They have three earlier questions flagged for review.
>
> They are considering going back to the flagged ones first, since they have already read them.

What is the better plan?

A. Revisit the three flagged questions first, since they are partly understood

B. Work forward through the eleven unseen questions, then return to flags with whatever remains

C. Answer the seven short questions, skip the four scenarios, then revisit flags

D. Slow down and answer carefully, accepting that some will go unanswered

◆ PAUSE AND DECIDE

What is the better plan?

A. Flags first

Spends scarce time on questions that already have answers, while eleven questions have none. Familiarity is not the same as value.

B. Unseen first, flags after

Correct. An unseen question is worth a mark you do not have; a flagged one already has an answer that may well be right.

C. Skip the scenarios

Skipping guarantees zero on four questions. Even a rushed read of a scenario beats not reading it.

D. Slow down

Accepts unanswered questions as a cost of care. With eleven left and twelve minutes, that trade loses more than it gains.

THE SIGNAL

A flagged question already has an answer. An unseen one has nothing. Under time pressure, unseen questions are worth more.

> A question asks which two statements about the dependency lock file are correct, and offers five options. A candidate is confident about three of them.
>
> They consider selecting all three, reasoning that partial credit is likely.

What should they do?

A. Select all three. Partial credit is standard on multiple answer questions

B. Select exactly two. One of their three is subtly wrong, and finding it is the question

C. Select one, to avoid the risk of a wrong selection

D. Leave it blank and return to it, since three seem correct

◆ PAUSE AND DECIDE

What should they do?

A Select all three

The question asks for two. Selecting three is not a partially correct answer to that question; it is an invalid one.

B Select exactly two

Correct. The count is a constraint, and having one option too many means one of them fails on a detail. Finding it is the question.

C Select one

Also the wrong count, and it discards a selection they were confident about to avoid a risk that does not exist.

D Leave it blank

A blank is a certain zero. Two of the three, chosen on the best available reasoning, is strictly better.

THE SIGNAL

Having one option too many is information. It tells you exactly where to look, because one of your three has a flaw.

Module 26 recap

- **60 minutes, online proctored, $70.50 plus tax, valid two years, no free retake.**
- **HashiCorp publishes no question count, no passing score and no domain weights.**
- **Read the requirement first, then eliminate.** Two workable options means a detail decides it.
- **A compound true/false statement is false if any clause fails.**
- **Answer everything as you reach it.** A blank is a certain zero; a guess is not.
- **Run the system check days ahead**, and match your ID to your profile name.

> ON THE EXAM
>
> None of this adds knowledge. All of it converts knowledge you already have into marks you might otherwise lose.

MODULE 27 · EXAM STRATEGY - ALL DOMAINS

Practice Exam 1 of 3

Twenty questions, mixed domains, nine seconds to answer each.

QUESTIONS 20 HOLD 9 seconds PAPER 1 of 3

admin@sthithapragna.com

How to use this paper

- **Answer within the nine seconds.** If you pause to think, you are not practising the thing that fails people.
- **Write your answers down.** A letter you said in your head is not a letter you can score.
- **Note which domain each miss came from.** That matters more than the total.
- **Do not look up anything mid-paper.** Sit it as an exam, then review afterwards.
- The mix is weighted to this course's derived domain percentages, verified across all sixty questions.

> ON THE EXAM
>
> The score is the least useful number here. Where your misses cluster is what tells you what to revise.

> A team replaces servers on every release by building a new machine image and swapping the fleet, rather than patching in place.

What is the principal advantage?

A Upgrades complete more quickly

B Upgrades require no automation

C Upgrades become less complex, because instances cannot diverge

D Upgrades can be applied without any downtime

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the principal advantage?

A Quicker

Usually the opposite. Building an image and cycling a fleet takes longer than patching in place.

B No automation

Unrelated. Immutable replacement is normally more automated, not less.

C Less complex

Correct. Every instance is built identically, so the class of problem caused by divergence stops occurring.

D No downtime

Describes what create_before_destroy aims at. Replacement itself can interrupt service.

ON THE EXAM

Domain 1. 'Quicker' is the most attractive wrong answer in this domain, and immutable is often slower.

> An engineer adds a required_providers entry with the local name dns and a version constraint, but no source argument. terraform init succeeds, and every resource type she uses is then rejected as unknown.

What happened?

A The provider requires manual installation because it is a community provider

B Terraform implied the hashicorp namespace and installed a different provider

C The version constraint excluded every published release

D The lock file pinned an incompatible version

◆ PAUSE AND DECIDE

What happened?

A Manual installation

True before 0.13 and false since. Official, partner and community providers all install identically.

B Implied hashicorp namespace

Correct. With no source, Terraform assumes the public registry, the hashicorp namespace, and the local name as the type.

C Constraint excluded everything

That fails init with a version error. Init succeeded, so something was installed.

D Lock file pinned wrongly

A lock file constrains a provider it already recorded. It cannot cause a different provider to be selected.

ON THE EXAM

Domain 2. Read the two symptoms together: init succeeded AND the resource types are unknown. That means the wrong provider, not a failed install.

> A developer adds a new module block to a configuration and runs terraform plan. Terraform stops and says the module is not installed.

What must be run, and what will it affect?

A terraform init, which touches no existing infrastructure

B terraform get -update, which also refreshes state

C terraform apply, which installs modules as part of the run

D terraform init -reconfigure, to rebuild the backend as well

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What must be run, and what will it affect?

A init, affects nothing

Correct. Init handles backend, providers and modules. It creates and destroys nothing.

B get -update

terraform get does install modules, and it does not refresh state. The second half of the option is wrong.

C apply installs modules

Neither plan nor apply installs modules. That is one of init's three responsibilities.

D init -reconfigure

Discards the backend configuration for no reason. The backend did not change; a module was added.

ON THE EXAM

Domain 3. Init's three responsibilities are backend, providers and modules, and none of them reaches a running resource.

> A plan shows a managed database with the symbol -/+ and the header 'must be replaced'. One line inside the block reads 'forces replacement'.

What does this mean?

A The database will be updated in place with a brief interruption

B Terraform cannot decide and will prompt during apply

C The database will be created first, then the old one removed

D The database will be destroyed and then recreated

◆ PAUSE AND DECIDE

What does this mean?

A Updated in place

That is the tilde symbol. Minus slash plus is not an update.

B Terraform will prompt

The plan has already decided. Apply carries it out; it does not reconsider.

C Created first

That ordering only happens with create_before_destroy set, and nothing here indicates it.

D Destroyed then recreated

Correct. Minus slash plus means destroy followed by create, which for a database is a very different conversation.

ON THE EXAM

Domain 3. The 'forces replacement' line names the single attribute responsible, and is usually the thing you can change to avoid it.

> A configuration passes a list of team names directly to a resource's for_each argument. terraform plan fails immediately.

Why?

A for_each requires at least two elements

B for_each accepts a map or a set of strings, not a list

C for_each cannot be used on this resource type

D The list must be sorted before use

◆ PAUSE AND DECIDE

Why?

A **Needs two elements**

Invents a rule. One element is fine, and an empty collection is valid and produces no instances.

B **Map or set, not a list**

Correct. Wrap it in toset, which is why that function appears so often in real configuration.

C **Not available here**

for_each is a meta-argument, available on every resource type regardless of provider.

D **Must be sorted**

Sorting changes order, and a set has no order. It would not make a list acceptable.

ON THE EXAM

Domain 4. This is the first mistake almost everyone makes with for_each, and toset is the fix.

> A module author wants callers to be able to switch prevent_destroy on and off, so she sets it from an input variable.

What happens?

A Terraform errors. Lifecycle settings accept only literal values

B It works, and the variable must be marked nullable = false

C It works only when the variable has a default

D It works, but only in the root module

◆ PAUSE AND DECIDE

What happens?

A Literal values only

Correct. Lifecycle settings shape the dependency graph, so they are processed before expressions can be evaluated.

B Needs nullable = false

Invents a dependency. nullable governs whether null may be passed; it has nothing to do with lifecycle.

C Needs a default

A default changes when a value is required, not whether an expression can appear in a lifecycle block.

D Root module only

The restriction is about evaluation order, not module position. It applies everywhere.

ON THE EXAM

Domain 4. The same principle governs plan-time for_each keys and dynamic blocks not generating meta-arguments.

> A team adds a check block asserting that no storage bucket is publicly readable. Three weeks later a public bucket is deployed. The pipeline run is marked successful and the warning appears in the log.

What went wrong?

A The assertion was written incorrectly

B Check blocks run only during plan, not apply

C The check block needed a scoped data source

D A check block reports a warning and lets the operation continue

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What went wrong?

A Written incorrectly

The warning is in the log, which proves the assertion evaluated false and fired exactly as designed.

B Plan only

A check block executes as the last step of both plan and apply. Neither was blocked, because warnings never block.

C Needed a data source

Would change what the check can see, not what a failure does. A better-informed warning is still a warning.

D It warns and continues

Correct. Enforcement requires a mechanism that halts: a precondition, a postcondition, or a policy.

ON THE EXAM

Domain 4. The mechanism worked perfectly. The choice of mechanism was the error.

> An engineer feeds an ephemeral random password into a database's password_wo argument, with password_wo_version set to 1. Security asks for a rotation. She reruns apply and the plan reports no changes.

Why?

A The ephemeral resource cached its value in state

B Write-only arguments can only be set at creation

C The version argument is unchanged, so no new value is accepted

D The database must be replaced to change its password

◆ PAUSE AND DECIDE

Why?

A It cached the value

Ephemeral resources persist nothing by definition, and they execute on every plan and apply. A new password genuinely was generated.

B Create-only

False. Updating is exactly what the version mechanism exists to support.

C Version unchanged

Correct. Terraform cannot see the value, so the version stored in state is the only signal it has. Increment it.

D Replace the database

Would work and rebuild a production database to change one argument. The version increment updates it in place.

ON THE EXAM

Domain 4. Ephemeral resources run every time, so without the version argument a password would appear to change on every apply.

> A shared module reads a startup script that ships alongside its Terraform files. It works when the author tests it from the module's own directory, and fails for every consumer.

What is the most likely cause?

A Modules may only contain .tf files

B The module uses path.root, which resolves to the caller's directory

C Consumers must run terraform get before the file is available

D File contents must be passed in as a variable

◆ PAUSE AND DECIDE

What is the most likely cause?

A Only .tf files

Modules routinely ship scripts, templates and policy documents. Nothing restricts the directory.

B path.root

Correct. In the author's own directory path.root and path.module are identical, so the bug hides exactly where it would be caught.

C Run terraform get

Init already installs modules, and the module was clearly installed since Terraform reached the point of reading the file.

D Pass it in

Would work and pushes a detail onto every consumer. Modules can read their own files perfectly well.

ON THE EXAM

Domain 4. path.module for anything a module ships. Testing from the module's own directory cannot catch this class of bug.

> A configuration in the same repository references a sibling directory as a module source, written as modules/vpc with no prefix. terraform init fails, reporting that it cannot find the module in the registry.

What is wrong?

A. A local path must begin with ./ or ../

B. Local modules require a version argument

C. The module must be published before it can be used

D. Sibling directories cannot be used as modules

◆ PAUSE AND DECIDE

What is wrong?

A Needs ./ or ../

Correct. That prefix is how Terraform distinguishes a filesystem path from a registry address. Without it, the string is a registry address.

B Needs a version

Backwards. The version argument works only with registry sources and would be rejected on a local path.

C Must be published

Local modules are the normal way to factor out repeated code within one repository.

D Siblings not allowed

A relative path going up and across is entirely valid, written with dot dot slash.

ON THE EXAM

Domain 5. The error message names the registry, which tells you Terraform read the string as a registry address rather than a path.

> A regulated team commits the dependency lock file and pins providers with pessimistic constraints. They apply the same pessimistic constraints to their registry modules. Two environments then deploy different module versions.

What should change?

A Commit the lock file in both environments

B Run terraform init -upgrade before every apply

C Switch the modules to Git sources with no ref

D Use exact version constraints for the modules

◆ PAUSE AND DECIDE

What should change?

A Commit the lock file

They already do, and it would not help. The lock file records provider selections only.

B -upgrade every time

The opposite of what they need. It re-resolves to the newest matching release, maximising divergence.

C Git with no ref

Removes even the constraint they had. An unpinned Git source takes whatever is on the default branch.

D Exact module versions

Correct. Modules are never recorded in the lock file, so the constraint is the entire pinning mechanism.

ON THE EXAM

Domain 5. A pessimistic constraint is safe for providers because the lock file pins underneath it. Modules have nothing underneath.

> A team is provisioning state storage on AWS. Their three-year-old runbook specifies a versioned, encrypted S3 bucket plus a DynamoDB table for locking.

Which part is out of date, and what replaces it?

A Versioning, replaced by state snapshots

B Encryption, now applied automatically by the backend

C DynamoDB locking, replaced by use_lockfile

D Nothing. The runbook is current

◆ PAUSE AND DECIDE

Which part is out of date, and what replaces it?

A **Versioning**

Still recommended, and the real protection against losing state.

B **Encryption**

Still configured deliberately, and still recommended alongside restricted access.

C **DynamoDB, use_lockfile**

Correct. dynamodb_table is deprecated. Native S3 locking writes a .tflock object beside the state file.

D **Nothing**

The honest trap. It still works, which is exactly why nobody has revisited it. Working and current differ.

ON THE EXAM

Domain 6. This is the most reliably out-of-date pattern in Terraform writing, including material published recently.

> A team moves state to a bucket in a different account. They edit the backend block, run init, misread the prompt, and rerun with -reconfigure. The next plan proposes to create every resource. Nothing has been destroyed.

What happened?

A -reconfigure discarded the old configuration without copying state

B The resources were destroyed during the migration

C The new bucket needs versioning enabled first

D Apply will detect and adopt the existing resources

◆ PAUSE AND DECIDE

What happened?

A -reconfigure copied nothing

Correct. It starts the new backend fresh. -migrate-state is the flag that brings existing state across.

B Destroyed

The question says nothing was destroyed, and migration never destroys infrastructure. Only apply changes anything.

C Versioning first

Versioning protects against losing state. It is not a precondition for reading it.

D Apply adopts them

The most dangerous option. Terraform never adopts existing objects by matching them; it would build duplicates.

ON THE EXAM

Domain 6. The infrastructure is fine. What was lost is Terraform's knowledge of it, which is why the next apply looks reasonable.

> An engineer runs terraform import for a manually created bucket, using the address she intends to use. The command reports success. The next plan proposes to destroy the bucket.

Why?

A The import silently failed and left a partial state entry

B No resource block exists at that address

C The bucket must be tagged before Terraform recognises it

D terraform import requires a second invocation to complete

◆ PAUSE AND DECIDE

Why?

A **Silent failure**

It reported success and succeeded. State now holds a correct binding, which is why the plan can see something to destroy.

B **No resource block**

Correct. Import writes state and never writes configuration. A binding with nothing declared reads as a resource you deleted.

C **Tagging**

Terraform identifies objects by what state records. Tags were tried as an identity mechanism and abandoned.

D **Two invocations**

One completes the import. What is missing is the other half of the job, which is yours to write.

ON THE EXAM

Domain 7. Import creates a binding in state. The configuration is still your job, and this is the most common import mistake.

> An engineer is asked to capture verbose logs. She exports TF_LOG_PATH pointing at a file and runs terraform apply. The apply completes and the file exists, completely empty.

What is wrong?

A The path must be relative to the working directory

B Logs are written only when a command fails

C TF_LOG_PATH was superseded by a command-line flag

D TF_LOG must also be set, or nothing is logged

◆ PAUSE AND DECIDE

What is wrong?

A Must be relative

Absolute paths work. The file was created where she asked, which proves Terraform read the variable.

B Only on failure

Logging is level-based, not outcome-based. A successful TRACE run produces a great deal of output.

C Superseded by a flag

There is no such flag. Terraform's logging is configured entirely through environment variables.

D TF_LOG is the switch

Correct. Even when TF_LOG_PATH is set, TF_LOG must be set for any logging to be enabled.

ON THE EXAM

Domain 7. The empty file is the clue. Terraform read the path and created the file, then had no logging enabled to write into it.

> An organisation enforces Sentinel policies on every workspace. One team switched their workspace's execution mode to Local for easier debugging. An audit later finds policy-violating resources deployed from that workspace, with no policy failures recorded.

What is the explanation?

A The policies were misconfigured and passed everything

B Sentinel applies only to VCS-driven workspaces

C Local execution mode disables the features that depend on remote operations

D The team used -auto-approve, which bypasses policy checks

◆ PAUSE AND DECIDE

What is the explanation?

A Misconfigured

They worked everywhere else. One workspace differing points at a workspace setting, not at the policies.

B VCS-driven only

Policies apply across workflows. Execution mode is what matters, not what triggered the run.

C Local disables them

Correct. HashiCorp lists policy enforcement, cost estimation and notifications as requiring remote execution.

D -auto-approve

Skips the interactive confirmation, not the policy stage. A failing policy would still stop the run.

ON THE EXAM

Domain 8. Nothing errored and nothing was recorded. The policies were not failing; they were never evaluated.

> A security team writes a policy for a legal data residency requirement. Deployments outside approved regions must never be permitted, under any circumstances, by anyone.

Which enforcement level applies?

A Advisory

B Hard-mandatory

C Soft-mandatory

D Advisory, with a run task to block the apply

◆ PAUSE AND DECIDE

Which enforcement level applies?

A Advisory

Logs a warning and lets the run continue. The deployment would proceed.

B Hard-mandatory

Correct. It stops the run and nobody can override it, which is what 'never, by anyone' requires.

C Soft-mandatory

Stops the run and permits an override by someone with the right permission. The requirement forbids any override.

D Advisory plus a run task

A run task could gate it, and it is an elaborate route to something one enforcement level does natively.

ON THE EXAM

Domain 8. Advisory warns, soft-mandatory can be overridden, hard-mandatory cannot. The requirement states its level in plain English.

> A configuration must deploy resources into two regions of the same cloud account, in a single plan, with the dependency between them visible.

How should this be configured?

A. A second provider block with an alias, and the provider meta-argument on the second region's resources

B. A second entry in required_providers, one per region

C. A CLI workspace per region

D. Two root modules, applied separately

◆ PAUSE AND DECIDE

admin@sthithapragna.com

How should this be configured?

A Alias plus meta-argument

Correct. One plugin, two configurations, both regions in one plan with a real dependency between them.

B Two required_providers entries

That block holds one entry per local name and says which plugin to install. Region is not part of a plugin's identity.

C A workspace per region

Only one workspace is active at a time, so the two regions could never appear in one plan.

D Two root modules

Works, and splits the plan in two. The requirement is a single plan with the dependency visible.

ON THE EXAM

Domain 2. 'Single plan, dependency visible' is the sentence that eliminates the two options which would otherwise work.

> A team applies with -target daily so they only touch the service they are working on. Over six months, three incidents were traced to resources nobody targeted. They ask how to make targeting safer.

What should they be told?

A Target the parent module instead, so dependencies are included

B Run an untargeted apply weekly to catch stragglers

C Stop routine targeting and split the configuration, sharing values via data sources

D Increase parallelism so full applies are fast enough

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should they be told?

A **Target the module**

Widens the selection and is still targeting. The undetected drift HashiCorp warns about is unchanged.

B **Weekly full apply**

Reduces the window without closing it, and means a weekly apply carrying six days of unreviewed change.

C **Split the configuration**

Correct, and the alternative HashiCorp names: smaller configurations applied independently, connected by data sources.

D **Raise parallelism**

Treats a design problem as throughput. Provider rate limits usually bite first.

ON THE EXAM

Domain 3. The question asks how to make targeting safer, and the documented answer is to stop needing it.

> A change board approves a plan on Monday. It is saved with -out. On Thursday an unrelated team applies an incident fix to the same configuration. On Friday the saved plan is applied.

What should the team expect?

A Terraform will prompt, because the plan is no longer current

B Terraform will regenerate the plan, since saved plans expire

C Terraform will merge Monday's plan with Thursday's changes

D The apply may fail or produce an unintended result

◆ PAUSE AND DECIDE

What should the team expect?

A It will prompt

Apply never prompts for a saved plan. The approval is treated as having happened when the plan was generated.

B Plans expire

Invents a lifetime. A plan file is stale the moment anything changes, and it never announces that.

C It merges

Terraform has no merge concept. A plan is a fixed set of actions computed against one moment.

D May fail or misfire

Correct. HashiCorp warns that intervening changes can make the final effect differ, and advises re-checking before applying.

ON THE EXAM

Domain 3. A saved plan guarantees that what was approved is what runs. It cannot guarantee that what runs still makes sense.

Scoring paper 1

- **The answer key, in order:** C B A D B A D C B A D C A B D C B A C D.
- **Now group your misses by domain**, using the label at the bottom of each walkthrough.
- **Two misses in one domain matters more than six spread evenly.** A cluster is a gap; a scatter is noise.
- **Do not re-sit this paper.** You would be testing recall of these questions, not of Terraform.
- Papers two and three cover different topics, and one question in each deliberately inverts one from this paper.

> ON THE EXAM
>
> Revisit the module for any domain where you missed more than one, then take paper two. The score itself tells you very little.

MODULE 28 · EXAM STRATEGY - ALL DOMAINS

Practice Exam 2 of 3

Twenty new questions. Three of them deliberately invert a question from paper one.

QUESTIONS 20 HOLD 9 seconds PAPER 2 of 3

admin@sthithapragna.com

> A team is arguing for adopting Infrastructure as Code over provisioning through the cloud console.

What is the headline advantage?

A. Resources become cheaper to run

B. Configuration can be versioned, reused and shared

C. Infrastructure becomes automatically highly available

D. Credentials are secured by the tooling

◆ PAUSE AND DECIDE

What is the headline advantage?

A Cheaper

The same instance costs the same however it was provisioned. Cost is a property of what you built.

B Version, reuse, share

Correct, and it is HashiCorp's own phrasing for the advantage of code over a graphical interface.

C Highly available

Availability comes from architecture. Declaring it in code does not create it.

D Credentials secured

Backwards. Code creates a new way to leak credentials, which is why secrets never belong in a repository.

ON THE EXAM

Domain 1. Every wrong option here is a genuine benefit of something else, which is what makes them attractive.

> A director tells the board that adopting Terraform will make it impossible for engineers to change infrastructure outside the approved process.

How should this be corrected?

A It is accurate once state is stored remotely

B It is accurate for immutable resources only

C It is accurate once the configuration is in version control

D Terraform surfaces such changes as drift, but cannot prevent them

◆ PAUSE AND DECIDE

How should this be corrected?

A **Remote state fixes it**

Remote state solves sharing. It has no authority over what somebody does in a cloud console.

B **Immutable only**

Invents a rule. Immutability describes how you choose to change things, not whether the provider accepts edits.

C **Version control fixes it**

Adds history and review to the configuration, and reaches nothing outside it.

D **Detects, cannot prevent**

Correct. Prevention needs provider-side permissions or policy. Terraform reports the difference; it has no authority to stop it.

ON THE EXAM

Domain 1. IaC detects unmanaged change. Preventing it means taking away the permission to make it.

admin@sthithapragna.com

> A configuration must install a specific provider version, and separately must configure that provider with a particular region.

Where does each belong?

A Both in the provider block

B Both in required_providers

C Version in required_providers; region in the provider block

D Version in the provider block; region in required_providers

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Where does each belong?

A Both in provider

The provider block configures behaviour. It does not decide which version is installed.

B Both in required_providers

That block names the plugin and constrains its version. Region is a provider-specific setting, not part of a plugin's identity.

C Version then region

Correct. required_providers declares and installs; the provider block parameterises what was installed.

D Reversed

Inverts both. This is the exact confusion the question is testing.

ON THE EXAM

Domain 2. Declaration installs, configuration parameterises. Two blocks, two jobs, constantly conflated.

admin@sthithapragna.com

> A team needs a provider published by a third-party vendor, listed in the Terraform Registry as a Community provider.

How is it installed?

A By terraform init, exactly like an official provider

B By downloading the binary and placing it manually

C Only through a private registry

D By terraform init, but only with an explicit -upgrade

◆ PAUSE AND DECIDE

How is it installed?

A init, like any other

Correct. Since 0.13, official, partner and community providers all install automatically from the registry.

B Manual placement

True before 0.13 and false since. This is the most persistent out-of-date claim about providers.

C Private registry only

Community providers are on the public registry. A private registry is for things you host yourself.

D Needs -upgrade

-upgrade re-resolves versions against constraints. It is not required for a first installation.

ON THE EXAM

Domain 2. The tier describes who supports a provider, not how it installs. All three tiers install identically.

> A pipeline runs on pull requests in an environment with no cloud credentials, by design. It must reject a change that references an argument name which does not exist on a resource type.

What should it run?

A terraform plan, granting it read-only credentials

B terraform fmt -check -recursive

C A check block asserting the argument exists

D terraform init -backend=false, then terraform validate

◆ PAUSE AND DECIDE

What should it run?

A plan with credentials

Would catch it, and grants credentials to a pipeline deliberately built without any, for a problem that never needed them.

B fmt -check

A formatter, not a linter. It will happily format a configuration referencing arguments that do not exist.

C A check block

Warns rather than blocking, so it could not reject anything, and it runs during a plan.

D init -backend=false, validate

Correct. validate checks attribute names and types using locally installed schemas, with no remote access at all.

ON THE EXAM

Domain 3. An invalid argument name is a language error. validate needs init but not a backend, and no credentials.

> A pipeline must generate a plan, have it approved by a human in a separate step minutes later, then apply exactly what was approved, with no possibility of a different set of actions running.

What supports this?

A Run plan and apply back to back with -auto-approve

B terraform plan -out, then apply that plan file

C terraform apply -refresh=false, to freeze the inputs

D terraform plan -detailed-exitcode, then a second plan

◆ PAUSE AND DECIDE

What supports this?

A Back to back

Removes the approval step entirely. The requirement is that a human approves between plan and apply.

B Saved plan file

Correct. Apply executes a saved plan exactly as written, with no prompt, which is why the two-step workflow exists.

C -refresh=false

Skips reading reality. It does not preserve a set of approved actions, and makes the plan less accurate.

D A second plan

A second plan is a new plan. What was approved and what runs could differ, which is the thing being ruled out.

ON THE EXAM

Domain 3. Paper one question 20 had a saved plan go stale over five days. Here the gap is minutes, and the saved plan is exactly right.

admin@sthithapragna.com

> A virtual machine is unresponsive after a failed in-guest upgrade. Its configuration is unchanged and correct, and Terraform plans no changes because the provider reports the instance as healthy.

What should be run?

A terraform apply -replace for that instance

B terraform apply -refresh-only

C terraform destroy, then terraform apply

D terraform state rm, then terraform apply

◆ PAUSE AND DECIDE

What should be run?

A -replace

Correct. This is the documented use: a degraded object rebuilt identically, with the replacement visible in the plan.

B -refresh-only

Records what the provider reports, and the provider reports a healthy instance. It would find nothing.

C destroy then apply

Destroys everything in the configuration to rebuild one sick machine.

D state rm then apply

Terraform forgets the instance, then builds a second one. The unresponsive machine keeps running, unmanaged.

ON THE EXAM

Domain 3. Terraform cannot see in-guest failure. -replace is how you tell it something is wrong that no API can report.

> Fourteen buckets are created from a list of team names using count. A new team is added alphabetically first. The plan proposes to destroy and recreate thirteen existing buckets.

What is the cause?

A Bucket names must be globally unique

B The list must be sorted before use

C count identifies instances by position, so every index shifted

D The provider requires buckets to be created in order

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the cause?

A Global uniqueness

True of bucket names and irrelevant. Only one name was added; the other thirteen are unchanged.

B Sort it

The list is already alphabetical, and sorting cannot rescue an identity scheme based on position.

C Positional identity

Correct. Inserting at the front shifts every index, so Terraform sees thirteen resources with different names.

D Ordering requirement

Ordering is not the problem. Terraform believes thirteen different buckets are now required.

ON THE EXAM

Domain 4. for_each over a set keys instances by value, so inserting a name affects nothing else. Appending to a list causes no churn, which is why this hides for years.

> An application instance reads its configuration from a storage object at boot only. The team wants the instance rebuilt automatically whenever that object changes. No argument on the instance references the object.

Which mechanism does this?

A depends_on referencing the storage object

B create_before_destroy on the instance

C ignore_changes on the instance's user data

D replace_triggered_by referencing the storage object

◆ PAUSE AND DECIDE

Which mechanism does this?

A depends_on

Orders operations only. The object would be updated first, and the instance would be left alone.

B create_before_destroy

Changes the order of a replacement that is already happening. It never causes one.

C ignore_changes

Suppresses reactions to change. The opposite direction entirely.

D replace_triggered_by

Correct. It replaces this resource when a referenced resource or attribute changes, which is precisely the requirement.

ON THE EXAM

Domain 4. depends_on says after. replace_triggered_by says again. Both take references, which is why they blur.

admin@sthithapragna.com

> A platform team wants to be alerted when any TLS certificate is within thirty days of expiry. The alert must appear in the run output and must never cause an unrelated deployment to fail.

Which mechanism fits?

A A postcondition on the certificate resource

B A check block with an assert on the expiry date

C A hard-mandatory Sentinel policy

D A variable validation comparing expiry to the current date

◆ PAUSE AND DECIDE

Which mechanism fits?

A A postcondition

Blocks, and a postcondition failure also prevents changes to dependent resources. An expiry notice would become an outage.

B A check block

Correct. A failed assertion reports a warning and the operation continues, which is exactly the stated requirement.

C Hard-mandatory policy

Cannot be overridden by anyone, so an approaching expiry would stop every deployment until the certificate was renewed.

D Variable validation

Runs before any resource exists, so it cannot see a certificate's expiry date, and it blocks.

ON THE EXAM

Domain 4. Paper one question 7 had a check block as the WRONG answer, because enforcement was required. Here the requirement forbids blocking.

> A module author needs to assert that a resource, once created, has a property that can only be read from the resource itself.

Which mechanism can express this?

A A variable validation block

B A precondition in the resource's lifecycle block

C A postcondition, which can use self

D A check block, which is the only mechanism with self

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Which mechanism can express this?

A Variable validation

Runs when variables are set, before any resource exists. There is nothing to inspect.

B A precondition

Runs before the block is provisioned, and cannot use self. The object does not exist yet.

C A postcondition with self

Correct. It runs after provisioning and can reference the enclosing object's own attributes.

D A check block

Check blocks are top-level and do not carry self. The claim inside the option is what makes it wrong.

ON THE EXAM

Domain 4. If the condition must inspect the object itself, only a postcondition can see it. That is the practical tell.

> A pipeline exports TF_VAR_environment as staging. The repository contains a committed terraform.tfvars setting environment to production. The deployment behaves as production.

Why?

A tfvars files have higher precedence than environment variables

B TF_VAR_ names are case-sensitive and must match exactly

C Environment variables are read only when no default exists

D The variable must also be declared with nullable = false

◆ PAUSE AND DECIDE

Why?

A tfvars outranks env vars

Correct. Environment variables sit second from the bottom, above only the default. Every tfvars file beats them.

B Case sensitivity

Names are case-sensitive, and a mismatch means the variable is unset, producing a prompt or an error rather than a wrong value.

C Only without a default

Environment variables are always consulted. They simply lose to anything above them in the order.

D nullable = false

Governs whether null may be passed. It has no bearing on precedence.

ON THE EXAM

Domain 4. Most tools treat an environment variable as an override of last resort. Terraform is the opposite, which is why this is reliably missed.

> A root configuration needs the public IP of a NAT gateway that a child module creates. An engineer writes a reference to the resource inside the module and Terraform rejects it.

What is required?

A. Use terraform_remote_state to read the module's resources

B. Declare an output in the module, then reference it

C. Move the NAT gateway into the root module

D. Read the value with terraform state show

◆ PAUSE AND DECIDE

What is required?

A Remote state

Reads another configuration's root outputs. This module is in the same configuration, and outputs are still the only interface.

B Declare an output

Correct. A module's outputs are its entire public interface. Undeclared internals are unreachable by design.

C Move it to root

Dismantles the module to read one value. Modules exist so internals can stay internal.

D state show

An inspection command that prints for a human. It cannot supply a value to configuration.

ON THE EXAM

Domain 5. There is no syntax for reaching inside a module. Encapsulation here is deliberate, not incidental.

> An engineer prepares a module for the public registry. The repository is public, named correctly, follows the standard structure and has a description. She tags the release release-2026-q1. The registry accepts the repository but lists no versions.

What is wrong?

A A module needs at least three versions before any are shown

B The tag must be signed

C The repository must be named with a terraform prefix

D Release tags must be a semantic version, optionally prefixed with v

◆ PAUSE AND DECIDE

What is wrong?

A Three versions

Invents a rule. One release tag is enough, and at least one is required to publish initially.

B Signed tags

Provider releases involve signing. Module publishing has no such requirement.

C Naming

A real requirement, and the question says the repository was named correctly and accepted.

D Not a semantic version

Correct. The registry identifies versions from x.y.z tags. Anything else is ignored silently.

ON THE EXAM

Domain 5. The registry accepted the repository and found no versions. That split clears the naming rule and points at the tags.

admin@sthithapragna.com

> A team converts nine production databases from count to for_each, with unchanged names. The plan proposes to destroy all nine and create nine replacements. The refactor must touch no infrastructure.

What should they add?

A A moved block per instance, mapping each old index to its new key

B prevent_destroy on each database

C ignore_changes so the address change is not detected

D A removed block per database, then re-import each one

◆ PAUSE AND DECIDE

What should they add?

A moved blocks

Correct, and exactly what moved exists for. One per instance, because what changed is how instances are keyed.

B prevent_destroy

Blocks the plan rather than fixing it. Terraform would refuse to apply and the refactor would stay stuck.

C ignore_changes

Ignores attribute changes, not addresses. Instance identity is not an attribute.

D Remove then re-import

Reaches the same state by a longer route, with a window where nine production databases are unmanaged.

ON THE EXAM

Domain 6. Terraform compares addresses, not intentions. A refactor that changes addresses is indistinguishable from a deletion unless you say otherwise.

> A storage bucket is being handed to another department. It must keep existing with all its contents, Terraform must stop managing it, and the change must be reviewable in a pull request.

What should be written?

A. Delete the resource block and apply

B. Run terraform state rm on the bucket

C. A removed block with lifecycle destroy set to false

D. A moved block pointing at the other department's configuration

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should be written?

A Delete and apply

Terraform destroys what is no longer declared. The bucket and its contents would go.

B state rm

Achieves the outcome, immediately, from a terminal, with no plan and no review. The last requirement rules it out.

C removed, destroy = false

Correct. It forgets the binding, leaves the object alone, and appears in a plan so it goes through review.

D A moved block

moved renames within one configuration's state. It cannot move an object into a configuration owned by somebody else.

ON THE EXAM

Domain 6. Two options achieve the outcome. The requirement about review is what decides between them.

> One configuration is deployed to three environments, each with its own state bucket. An engineer adds a variable for the bucket name and references it from the backend block. terraform init fails.

What is the correct approach?

A Define the bucket name as a local value instead

B Move the backend block to its own file so it parses later

C Maintain three copies of the configuration

D Use partial configuration, supplying values at init time

◆ PAUSE AND DECIDE

What is the correct approach?

A Use a local

Locals are named values too. The restriction covers input variables, locals and data source attributes alike.

B A separate file

Filenames carry no meaning. Terraform reads all .tf files as one configuration, so ordering is unaffected.

C Three copies

Solves it by duplicating everything, reintroducing exactly the divergence problem modules exist to prevent.

D Partial configuration

Correct. A minimal backend block, with a per-environment settings file supplied via -backend-config at init.

ON THE EXAM

Domain 6. The restriction is about evaluation order, not syntax. The backend is resolved before variables, locals or data sources exist.

> An engineer needs to adopt one manually created resource quickly, as a one-off repair. A colleague says the terraform import command is deprecated and must not be used.

Is the colleague correct?

A. No. The command is not deprecated and suits a one-off adoption

B. Yes. It was deprecated when import blocks arrived in 1.5

C. Yes. It was deprecated alongside taint and the refresh command

D. No, but it only works for resources that support config generation

◆ PAUSE AND DECIDE

Is the colleague correct?

A Not deprecated

Correct. import blocks are preferable for anything reviewed or repeated, and the command remains valid for a one-off.

B Deprecated in 1.5

1.5 added import blocks as a better option. Adding an alternative is not the same as deprecating the original.

C With taint and refresh

Those two genuinely are deprecated, which is what makes this attractive. The import command was not.

D Config generation only

Invents a restriction. Config generation is a separate feature of the block-based workflow.

ON THE EXAM

Domain 7. Paper one question 12 rewarded spotting an out-of-date pattern. This one punishes assuming that older always means wrong.

> A workspace must hold its own cloud credentials, its own list of who may apply, and a record of every run for audit.

Which kind of workspace provides this?

A A CLI workspace, created with terraform workspace new

B An HCP Terraform workspace

C Either. Both isolate credentials and permissions

D Neither. Credentials are always organisation-wide

◆ PAUSE AND DECIDE

Which kind of workspace provides this?

A A CLI workspace

Isolates state and nothing else. Configuration, backend, credentials and permissions are all shared.

B An HCP workspace

Correct. Its own variables, credentials, permissions, run history and state versions is the definition of one.

C Either

The assumption the ambiguity exists to catch. HashiCorp calls the two distinctly different.

D Neither

Overcorrects. HCP Terraform's permission model is built on exactly this separation.

ON THE EXAM

Domain 8. Decide which kind of workspace a question means before reading the options. The correct answers have nothing in common.

> A regulated team wants their Git repository to be the single source of truth. Every change must be reviewed as a pull request with a plan attached, and nobody should be able to start an apply from a workstation. They currently use the CLI-driven workflow with remote execution.

What should change?

A. Nothing. Remote execution already prevents applying from a workstation

B. Switch execution mode to Agent

C. Move to the VCS-driven workflow

D. Move to the API-driven workflow

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should change?

A Nothing needed

Remote execution moves where a run happens, not who can start one. An engineer can still type terraform apply.

B Agent mode

Execution mode again, answering a question about triggering. An agent changes where a run executes, not what starts it.

C VCS-driven workflow

Correct. Runs trigger from the repository, pull requests receive speculative plans, and the branch becomes the source of truth.

D API-driven

Requires building tooling for no benefit here, and approvals are available across workflows anyway.

ON THE EXAM

Domain 8. Workflow is what starts a run. Execution mode is where it runs. Two of these options answer the wrong question.

Scoring paper 2

- **The answer key, in order:** B D C A D B A C D B C A B D A C D A B C.
- **Three questions inverted paper one:** 6 against 20, 10 against 7, and 18 against 12.
- **Missing an inverted pair is the most useful miss on the paper.** It means you learned a mechanism as a rule.
- **Compare your domain misses with paper one.** A domain missed on both is a real gap.
- Paper three covers the remaining topics and closes by reading the trend across all three.

> ON THE EXAM
>
> A mechanism is never simply right or wrong. It fits a requirement or it does not, which is what the inverted pairs are for.

MODULE 29 · EXAM STRATEGY - ALL DOMAINS

Practice Exam 3 of 3

The last twenty questions, and a look at what three papers have told you.

QUESTIONS 20 HOLD 9 seconds PAPER 3 of 3

admin@sthithapragna.com

> A team is told Terraform's multi-cloud support means their AWS configuration will work against Azure with minimal change.

How should this be corrected?

A It is accurate, provided only official providers are used

B It is accurate once the configuration is written with modules

C It is accurate for compute resources but not storage

D Terraform offers one workflow across providers, not portable configuration

◆ PAUSE AND DECIDE

How should this be corrected?

A **With official providers**

Provider tier has nothing to do with it. Resource types differ between clouds regardless of who maintains the provider.

B **With modules**

A module wraps resource types; it does not translate between them. The resources inside are still cloud-specific.

C **Compute but not storage**

Invents a distinction. No category of resource is portable between clouds.

D **One workflow, not portable config**

Correct. The promise is a common write, plan, apply cycle across providers. Configuration is written against specific resource types.

ON THE EXAM

Domain 1. 'Write once, run on any cloud' is the most common misreading of what multi-cloud means here.

> A bank has written its own provider for an internal system. Security forbids publishing it anywhere outside the bank, and the platform team would rather not operate another internal service.

How should it be distributed?

A Give it a source address on a bank-controlled hostname and use a filesystem mirror

B Publish it to the public registry under a private namespace

C Omit it from required_providers and place the binary manually

D Rewrite it as a module, since providers must be published

◆ PAUSE AND DECIDE

How should it be distributed?

A Own hostname, mirror

Correct. A source address is an identifier first; the hostname need not resolve. A mirror avoids running a registry.

B Private namespace

The public registry has no private namespaces. A namespace is an organisation name, not an access control.

C Omit and place manually

Omitting source implies the hashicorp namespace. Providers must be declared to be used.

D Rewrite as a module

A module composes resource types that already exist. Only a provider can create a new one.

ON THE EXAM

Domain 2. A source address looks like a URL, so people assume it must resolve. It does not.

admin@sthithapragna.com

> Available provider releases are 1.4.0, 1.4.7, 1.5.2 and 2.0.1. A configuration specifies a constraint of ~> 1.4.0.

Which version is selected?

A 1.4.0

B 1.4.7

C 1.5.2

D 2.0.1

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Which version is selected?

A 1.4.0

The floor, not the selection. Terraform takes the newest version satisfying the constraint.

B 1.4.7

Correct. Three components written means the rightmost may rise, so patch releases only. 1.5.0 and above are excluded.

C 1.5.2

What ~> 1.4 with two components would select. The third component changes the meaning entirely.

D 2.0.1

What >= 1.4.0 with no ceiling would select. The pessimistic operator always implies a ceiling.

ON THE EXAM

Domain 2. Count the components before anything else. Three allow patch; two allow minor. Same operator, different answer.

> A team wants every engineer and every pipeline to install exactly the same provider versions, without pinning each one to an exact number in configuration.

What achieves this?

A Add .terraform to version control

B Run terraform init -upgrade in the pipeline

C Commit the .terraform.lock.hcl file

D Use a plugin cache directory shared across machines

◆ PAUSE AND DECIDE

What achieves this?

A Commit .terraform

A disposable working directory holding provider binaries. It belongs in your ignore file, not your repository.

B -upgrade in the pipeline

The opposite. It re-resolves to the newest matching versions every run, maximising divergence.

C Commit the lock file

Correct. It records the exact version selected and its checksums, so everyone selects identically.

D A shared plugin cache

Saves downloading. It does not decide which version each configuration selects.

ON THE EXAM

Domain 2. Two filenames differ by four characters and have opposite treatment: ignore the directory, commit the lock file.

> A repository has passed terraform fmt -check -recursive for two years. Nobody has touched the Terraform code this week. After the pipeline image was rebuilt with a newer Terraform CLI, the check now fails on forty files.

What is the most likely explanation?

A The newer CLI applies additional formatting rules, which is expected

B -recursive now scans directories it previously skipped

C A third-party tool reformatted the repository

D fmt -check is non-deterministic and should be replaced

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the most likely explanation?

A New rules in a new CLI

Correct. HashiCorp states the canonical format may change in minor ways between versions, and new rules are not a breaking change.

B -recursive changed

Plausible-sounding and unsupported. Recursive has meant subdirectories since it was introduced.

C A third-party tool

The image rebuild changed the tool, not the repository. Nothing in a pipeline writes back to source.

D Non-deterministic

Entirely deterministic for a given version, which is the whole point of a canonical format.

ON THE EXAM

Domain 3. Inputs constant, output changed, therefore the tool changed. Run fmt once, commit, carry on.

> A pipeline must raise an alert only when a plan finds pending changes, and must treat a genuine error differently from a plan with changes.

What supports this?

A Parse the plan output for the summary line

B terraform plan -json, and count the change entries

C terraform plan -detailed-exitcode

D terraform plan -refresh=false, which exits non-zero on changes

◆ PAUSE AND DECIDE

What supports this?

A **Parse the output**

Works until the wording changes. Terraform provides a supported mechanism, so parsing human output is unnecessary.

B **-json and count**

Would work and is far more effort. It also still needs logic to distinguish an error from a change.

C **-detailed-exitcode**

Correct. Zero means no changes, one means an error, two means changes present - exactly the three-way distinction required.

D **-refresh=false**

Skips reading reality. It has no effect on exit codes at all.

ON THE EXAM

Domain 3. Without this flag a successful plan exits zero whether or not it found changes, which is why pipelines need it.

> A deployment script captures a load balancer hostname with terraform output and passes it to a health check. Every check fails, with the hostname wrapped in double quotation marks.

What should the script use?

A terraform show, filtered to the output name

B terraform output -raw

C terraform console, piped the output name

D terraform state show on the load balancer

◆ PAUSE AND DECIDE

What should the script use?

A show, filtered

show prints whole state or a whole plan file. It does not take an output name.

B output -raw

Correct. -raw prints the bare value with no quoting, which is what a shell variable needs.

C console piped

Console can evaluate the expression, and wrapping an interactive tool in a script is the wrong instrument.

D state show

Prints a resource's recorded attributes for a human to read, not a single value for a script.

ON THE EXAM

Domain 3. The quotes are not a bug. Default output is formatted for a terminal; -raw is formatted for another program.

> A database's generated administrator password is supplied through a variable declared with sensitive = true. Plan output correctly hides it. An auditor opens the remote state file and reads the password in plain text.

What is the explanation?

A sensitive applies to outputs, not variables

B The backend should encrypt sensitive attributes individually

C The plan was run with -refresh=false

D sensitive affects displayed output only. State stores values in plain text

◆ PAUSE AND DECIDE

What is the explanation?

A Outputs only

Both variables and outputs accept sensitive, and it behaves the same way in both. The marking worked as designed.

B Per-attribute encryption

No backend encrypts individual attributes. Encryption at rest protects the whole file from theft, not from authorised readers.

C -refresh=false

Controls whether attributes are re-read. It has no bearing on whether they are stored.

D Display only

Correct. sensitive suppresses display. HashiCorp's guidance is to treat state itself as sensitive data.

ON THE EXAM

Domain 4. This course's most repeated misconception. Nothing malfunctioned; the mental model was wrong.

> A module author writes a dynamic block labelled lifecycle, so callers can switch prevent_destroy on by passing a value. Terraform rejects the configuration.

Why?

A Dynamic blocks cannot generate meta-argument blocks such as lifecycle

B Dynamic blocks require a non-empty for_each

C The label must be quoted and needs an iterator

D prevent_destroy is unavailable inside modules

◆ PAUSE AND DECIDE

Why?

A Cannot generate meta-arguments

Correct. Terraform processes meta-arguments before it is safe to evaluate expressions, so they cannot be produced dynamically.

B Needs non-empty for_each

An empty collection is valid and simply generates no blocks. That is a normal pattern.

C Quoting and iterator

Invents a rule. No keyword becomes valid by adding an iterator argument.

D Not available in modules

prevent_destroy works anywhere a lifecycle block is valid. Module scope is irrelevant.

ON THE EXAM

Domain 4. The same principle governs plan-time for_each keys and literal-only lifecycle values: what the engine needs early cannot come from what runs late.

> A module takes a boolean enable_logging and a string log_bucket_name. If logging is on, the bucket name must not be empty. The team runs Terraform 1.12.

Where should this rule live?

A A null_resource carrying a precondition, the documented workaround

B A check block, which can see both variables

C Nowhere. Terraform cannot express a rule spanning two variables

D A validation block on the variable, which since 1.9 may reference others

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Where should this rule live?

A null_resource precondition

The documented pattern before 1.9. On 1.12 it adds a placeholder resource to state for no benefit.

B A check block

Warns rather than blocking, so an invalid combination would deploy anyway, and it runs last.

C Cannot be expressed

True before 1.9 and false since. This is the belief the question is testing.

D Cross-object validation

Correct. Since 1.9 a validation condition may reference other input variables, data sources and local values.

ON THE EXAM

Domain 4. Option A catches experienced people rather than beginners, because it was the correct answer for several years.

> A team reads database credentials from Vault using the Vault provider's data sources. A security review finds the passwords in plain text in the remote state file. They are on Terraform 1.12 with a current provider.

What should they change?

A Mark the data source output as sensitive

B Enable encryption at rest on the backend

C Replace the data source with an ephemeral resource

D Move the credentials into TF_VAR_ environment variables

◆ PAUSE AND DECIDE

What should they change?

A Mark it sensitive

Display only, as ever. The value would still be written to state.

B Encryption at rest

Worth doing, and it does not resolve this. The reviewer had legitimate read access.

C An ephemeral resource

Correct. Same secret, same source, and the value is never written to the plan artefact or state.

D Environment variables

Moves the secret out of Vault into pipeline configuration, losing central rotation, and it would still land in state.

ON THE EXAM

Domain 4. Fetching from a secrets manager solved distribution and never solved persistence. Ephemeral resources closed that gap.

> A team proposes publishing a module containing exactly one storage bucket resource, with eleven input variables each passed straight through to the corresponding resource argument.

What should a reviewer say?

A Approve. Every resource should be wrapped for consistency

B Probably not. It adds a layer and a release cycle while encapsulating no decision

C Approve once count is added so it can be made conditional

D Reject. The registry does not accept single-resource modules

◆ PAUSE AND DECIDE

What should a reviewer say?

A Wrap everything

Consistency for its own sake, and it restricts callers to the eleven arguments chosen where the resource offers more.

B Encapsulates nothing

Correct. Eleven pass-through variables is a rename, not an abstraction. A module should save the caller from knowing something.

C Add count first

Improves a module that should not exist. Conditional creation works equally well on the resource directly.

D Registry rejects it

Invents a rule. A single-resource module is fine when it encodes an opinion, such as enforced encryption or naming.

ON THE EXAM

Domain 5. A single-resource module is not automatically wrong. Pure pass-through is what makes this one questionable.

> A platform team distributes modules from a private Git repository. Every consuming team needs a deploy key and every pipeline needs repository credentials, so onboarding takes days. They already use HCP Terraform.

What removes most of that burden?

A Make the module repository readable by everyone

B Vendor the modules into each repository with a sync job

C Switch consumers to local path sources

D Publish the modules to the HCP Terraform private registry

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What removes most of that burden?

A Open the repository

Widens visibility to solve a distribution problem, and every pipeline still needs VCS credentials.

B Vendor with a sync job

Automates divergence. Every repository still holds its own copy, now with a script to maintain.

C Local paths

Local paths cannot cross repository boundaries reliably, and they accept no version argument.

D Private registry

Correct. The registry stores the source and serves it via API tokens, so consumers need no access to the module repository at all.

ON THE EXAM

Domain 5. Versioning is the obvious benefit of a private registry. Decoupling consumption from repository permissions is the quieter one.

> An engineer runs terraform state rm on a resource so Terraform stops managing it, and leaves the resource block in the configuration. The next plan proposes to create the resource.

Why?

A The resource is declared with no binding in state, so Terraform plans a create

B state rm destroyed the object, so it must be rebuilt

C The state file needs refreshing before the plan is accurate

D state rm requires a matching removed block to take effect

◆ PAUSE AND DECIDE

Why?

A Declared, no binding

Correct. Terraform compares configuration with state, finds a declaration and no binding, and concludes it must create it.

B It was destroyed

state rm destroys nothing. The object keeps running; Terraform simply forgets it.

C Needs refreshing

Refreshing re-reads objects Terraform knows about. It has no record of this one to re-read.

D Needs a removed block

The block is the declarative alternative to the command, not a companion to it.

ON THE EXAM

Domain 6. Applying here builds a duplicate while the original keeps running, unmanaged. Delete the resource block too.

> During an incident, an engineer increased a database's storage through the console. The change was correct and should be kept, and Terraform should manage it correctly from now on. The configuration still specifies the old size.

What should be done?

A Run terraform apply to reconcile state with reality

B Run apply -refresh-only only. State then matches reality

C Run apply -refresh-only, then update the configuration

D Use a removed block so Terraform stops managing the database

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What should be done?

A Just apply

Reverts the change. A normal apply makes reality match configuration, and the configuration still says the old size.

B Refresh-only alone

State matches reality and configuration does not, so the next routine apply shrinks the database back.

C Adopt, then codify

Correct. Adopt the change into state, then write it into configuration so the next normal apply proposes nothing.

D A removed block

Stops Terraform managing the database entirely, throwing away management to avoid editing one attribute.

ON THE EXAM

Domain 6. Three responses to drift: revert, adopt, codify. Adopting alone leaves the next apply to undo it, possibly weeks later.

> An engineer writes an import block using for_each over a map of twelve existing databases, then runs terraform plan with -generate-config-out to produce the resource configuration. Terraform errors before generating anything.

What is the cause?

A for_each on import blocks requires Terraform 1.9 or later

B Configuration generation cannot be used with for_each

C The resources must already exist in state

D Import blocks cannot target more than one resource

◆ PAUSE AND DECIDE

What is the cause?

A Needs 1.9

for_each on import blocks arrived in 1.7, and the exam tests 1.12 regardless. Version is not the constraint.

B Generation and for_each

Correct. The flag cannot be used with import blocks that use for_each, nor with resources using for_each or count.

C Must be in state

Backwards. Generation exists for resources not yet in state; once imported, no generation is needed.

D One resource only

Import blocks handle many at once, which is one of their advantages over the command.

ON THE EXAM

Domain 7. Bulk import and configuration generation are the two headline import features, and they are mutually exclusive.

> An apply fails intermittently with an uninterpretable message from the cloud provider. A TRACE log runs to hundreds of megabytes, dominated by Terraform's internal graph output. The team needs the API request and response for the failing call.

What is the better approach?

A TF_LOG_PROVIDER=TRACE

B TF_LOG=DEBUG, for less noise

C TF_LOG_CORE=TRACE

D apply -parallelism=1, so the log covers one operation at a time

◆ PAUSE AND DECIDE

What is the better approach?

A TF_LOG_PROVIDER=TRACE

Correct. It keeps full detail on the half they care about and removes core's graph output entirely.

B DEBUG instead

Reduces volume by lowering detail, risking the loss of the request and response bodies they specifically need.

C TF_LOG_CORE=TRACE

Exactly the output they are already drowning in. Core produced the noise, not the API calls.

D -parallelism=1

Makes the log easier to follow and does nothing about its size or which half is talking.

ON THE EXAM

Domain 7. Narrow by component before narrowing by level. Lowering the level discards the detail you need.

> A policy requires instances to come from an approved size list. Occasionally a genuine exception is justified, and when one is granted it must be a deliberate decision with a record of who approved it.

Which enforcement level applies?

A Advisory

B Hard-mandatory

C Soft-mandatory

D Hard-mandatory, with a separate advisory policy for exceptions

◆ PAUSE AND DECIDE

Which enforcement level applies?

A Advisory

Logs a warning and lets the run continue, so nothing would be deliberate and no approval would be required.

B Hard-mandatory

Permits no override at all. The requirement explicitly allows justified exceptions.

C Soft-mandatory

Correct. It stops the run, permits an override by someone with the right permission, and the override is recorded.

D Hard plus advisory

Two policies that between them still cannot grant an exception. An advisory policy blocks nothing.

ON THE EXAM

Domain 8. Paper one question 17 was 'never, by anyone', which is hard-mandatory. Here an exception is legitimate, so the level differs.

> A team manages clusters in a private data centre with no route from HCP Terraform's runners. They want to keep policy enforcement, run history and centrally managed variables.

What should they use?

A Local execution mode, running on a jump host inside the network

B Open inbound firewall rules for HCP Terraform's runners

C The API-driven workflow, which needs no network access

D An HCP Terraform agent running inside the network

◆ PAUSE AND DECIDE

What should they use?

A Local execution mode

Reaches the clusters and gives up all three things they asked for. Local mode disables policies, notifications and workspace variables.

B Open inbound rules

Removes a networking constraint on a private data centre, which is the opposite of what agents exist to avoid.

C API-driven

Confuses workflow with execution mode. An API-driven run still executes on HCP Terraform's infrastructure by default.

D An agent

Correct. The agent runs inside the network with an outbound connection, and runs stay HCP Terraform runs with everything intact.

ON THE EXAM

Domain 8. Local and Agent both move execution off HashiCorp's runners. Only the agent keeps the run under HCP Terraform's control.

> A global variable set defines an instance size for the whole organisation. A production workspace sets a workspace-specific variable with the same key and a larger size.

Which value does the production workspace use?

A The global variable set, because organisation scope wins

B The workspace-specific variable

C Neither. The run fails with a conflicting variable error

D Whichever was edited most recently

◆ PAUSE AND DECIDE

Which value does the production workspace use?

A The global set

Backwards. Organisation scope is the broadest scope, and breadth means lowest precedence unless the set is marked priority.

B The workspace variable

Correct. Workspace-specific variables always overwrite variables from variable sets with the same key.

C A conflict error

Overriding is the intended design. HCP Terraform shows which value it will use rather than refusing to run.

D Most recently edited

The common assumption, and wrong even for same-scope ties, which are broken by lexical order of the set names.

ON THE EXAM

Domain 8. More specific scope wins. The one exception is a priority variable set, which inverts the order deliberately.

Scoring paper 3

- **The answer key, in order:** D A B C A C B D A D C B D A C B A C D B.
- **Question 18 completed the fourth inverted pair**, against paper one's question 17 on hard-mandatory.
- **Across all three papers there were four inverted pairs.** If you missed both halves of one, revisit that module.
- **Sixty questions, weighted to this course's derived domain percentages**, verified across the whole set.
- Now read the trend, on the next slide.

> ON THE EXAM
>
> Every question across the three papers was original. None came from any question bank.

Reading the trend

- **Lay the three domain breakdowns side by side.**
- **A domain missed on all three is the only signal worth acting on.**
- **A rising score means little.** Later papers were not easier; you had seen more.
- **A falling score means little too.** Paper three carried the hardest misconceptions.
- **Where you missed matters more than how many.**

> ON THE EXAM
>
> The exam reports at objective level too. Practising that habit here means the real report tells you something you can use.

Before you sit the real thing

- **Revisit the modules your misses clustered in**, not the whole course.
- **Re-watch module twenty-four**, the confusion pairs, close to the day. It is pure recognition practice.
- **Check module twenty-five** if you use Terraform 1.15 or 1.16 at work. Three rules changed after 1.12.
- **Run the Certiverse system check days ahead**, and match your ID to your certification profile name.
- **Answer every question as you reach it.** A blank is a certain zero; a guess is not.

> ON THE EXAM
>
> Sixty minutes, one attempt, no free retake. Pace is part of the exam, and these three papers were where you practised it.